Octobre 2026

Segmentation & Hardening

SARR Pape Aldiouma

L3 Cybersécurité · Module : Sécurité des systèmes d'exploitation

Enterprise Network Segmentation, Hardening & Incident Response. Rapport de laboratoire : OPNsense, Kali Linux, Debian, Fail2Ban, Auditd, Lynis, OpenVAS et ClamAV pour l'entreprise fictive SenTech Solutions.





République du Sénégal, Un Peuple, Un But, Une Foi
Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation
Direction Générale de l'Enseignement Supérieur Privé
Institut Supérieur de Management
Année académique 2026-2027

SOMMAIRE

Introduction : Contexte et objectifs du laboratoire....3
Les objectifs....3
Les machines et les outils....3
Partie I : Conception de l'architecture du laboratoire....4
1. Vue d'ensemble....4
2. Plan d'adressage IP définitif....5
Partie II : Mise en place de l'infrastructure QEMU/KVM....6
1. Création des réseaux virtuels libvirt....6
2. Création des machines virtuelles....8
Partie III : Installation et configuration d'OPNsense....9
1. Installation et assignation des interfaces réseau....9
2. Vérification depuis la console....9
3. Configuration réseau statique de Kali Linux....10
4. Configuration réseau statique de Debian Serveur et Debian Client....11
5. Vérification de la connectivité des machines....11
6. Tableau récapitulatif des accès....12
Partie IV : Administration de base....14
Partie 1 : Gestion des utilisateurs....14
Partie 2 : Permissions et ACL....16
Partie 3 : Sécurisation par SSH....18
Partie V : Hardening Linux : durcissement de Debian Serveur....24
Les outils utilisés....24
Partie 1 : Audit initial, état des lieux avant hardening....24
Partie 2 : Désactivation des services inutiles....26
Partie 3 : Configuration du pare-feu local....26
Partie 4 : Déploiement de Fail2Ban....32
Partie 5 : Configuration d'Auditd....37
Partie 6 : Durcissement de PAM et sudo....41
Partie 7 : Hardening système complémentaire (sysctl)....45
Partie 8 : Vérification finale et comparaison....46
Partie VI : Audit, analyse et réponse locale....51
Rappel théorique....51
Partie 1 : Audit Lynis, mesure de la posture de sécurité....52
Partie 2 : Analyse approfondie des journaux système....59
Partie 3 : Scan de vulnérabilités avec OpenVAS/Greenbone (sur Kali)....62
Partie 4 : Sauvegarde et restauration système....66
Partie 5 : Rédaction du rapport de remédiation....75
Partie VII : Pilotage de ClamAV via l'interface graphique ClamUI....77
Présentation de ClamUI....77
Étape 0 : Installation de ClamAV (sur Debian Client)....77
Étape 1 : Installation de ClamUI....77
Étape 2 : Découverte de l'interface et premier scan....78
Étape 3 : Test avec un fichier EICAR....79
Étape 4 : Relancer le scan....80
Étape 5 : Vérifier la quarantaine....81
Étape 6 : Protection en temps réel (On-Access)....81
Étape 7 : Analyse automatique des périphériques USB....82
Étape 8 : Planification des analyses....86
Étape 9 : Intégration au gestionnaire de fichiers (Nautilus)....87
Étape 10 : Éditeur de configuration....87
Questions de synthèse....88
Conclusion : Bilan et perspectives....89
Perspectives d'amélioration....89
Annexe : Glossaire....90

INTRODUCTION

Contexte et objectifs du laboratoire

SenTech Solutions est une entreprise fictive de taille moyenne qui compte trois équipes : les développeurs, l'équipe finance et les administrateurs système. Sa direction souhaite protéger ses serveurs Linux contre les attaques courantes, c'est-à-dire les scans de ports, les tentatives de connexion par force brute, les erreurs de configuration et les logiciels malveillants. Elle veut aussi pouvoir prouver, avec des mesures chiffrées, que la sécurité s'est réellement améliorée.

Ce rapport décrit le laboratoire que nous avons construit pour répondre à ce besoin. Il suit la démarche d'un administrateur sécurité : **concevoir** un réseau segmenté, **configurer** les accès et les permissions, **durcir** un serveur Debian, **auditer** le résultat, **sauvegarder** et **restaurer** après un incident, puis **protéger** un poste de travail avec un antivirus.

POUR COMPRENDRE **Comment lire ce rapport**

Chaque étape est présentée de la même façon : le **principe** (pourquoi on le fait), les **commandes** (comment on le fait), puis la **vérification** (preuve que cela fonctionne, avec une capture d'écran). Les encadrés « Pour comprendre » expliquent les notions utiles à un lecteur qui n'est pas spécialiste. Les encadrés « Attention » signalent les erreurs fréquentes et les risques de blocage.

Les objectifs

- Construire un réseau d'entreprise virtuel protégé par un pare-feu périmétrique (OPNsense), avec la virtualisation QEMU/KVM.
- Créer des comptes, des groupes, des permissions et des ACL selon le principe du moindre privilège.
- Sécuriser l'accès SSH par clés et durcir Debian Serveur : pare-feu iptables, Fail2Ban, Auditd, PAM, sudo, paramètres du noyau.
- Mesurer la sécurité avec Lynis (audit interne) et OpenVAS (scan de vulnérabilités depuis le réseau).
- Mettre en place des sauvegardes automatiques et les tester lors d'un incident simulé.
- Détecter les logiciels malveillants avec ClamAV et son interface ClamUI.

Les machines et les outils

Élément	Version / rôle	Usage dans le laboratoire
Hôte physique	Ubuntu, QEMU/KVM, libvirt, virt-manager	Héberge les quatre machines virtuelles et les deux réseaux virtuels
OPNsense	Pare-feu / routeur (FreeBSD)	Frontière entre « Internet » et le réseau interne
Kali Linux	Distribution d'audit	Poste d'administration, scans Nmap, serveur OpenVAS
Debian Serveur	Debian 13 « Trixie »	Machine cible, durcie et auditée
Debian Client	Debian 13 « Trixie »	Serveur de sauvegarde et poste équipé de ClamAV

REMARQUE

Tous les noms, comptes et adresses de ce rapport appartiennent à un laboratoire isolé. Les attaques simulées (scans, force brute, fichier de test EICAR) n'ont été réalisées que sur nos propres machines virtuelles.

PARTIE I

Partie I : Conception de l'architecture du laboratoire

1. Vue d'ensemble

L'architecture reproduit, à l'échelle d'un laboratoire, la structure d'un petit réseau d'entreprise protégé par un pare-feu périmétrique. Elle comprend deux segments réseau distincts, séparés par OPNsense :

- un segment **WAN** (accès Internet), qui relie OPNsense à notre machine physique Ubuntu à travers un réseau NAT géré par libvirt ;
- un segment **LAN** (192.168.10.0/24), isolé du reste du monde, sur lequel se trouvent Kali, Debian Serveur et Debian Client.

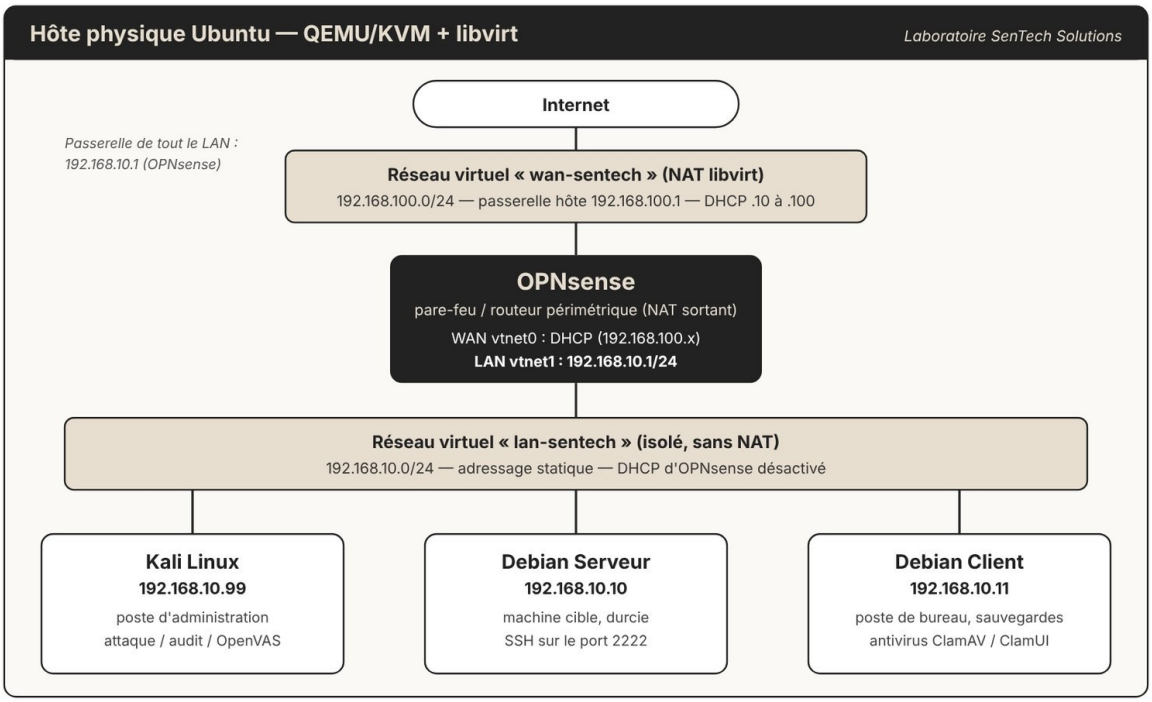


Figure 1 : *Architecture du laboratoire : deux réseaux virtuels séparés par le pare-feu OPNsense*

> POUR COMPRENDRE **WAN, LAN, NAT**
> Le **WAN** (Wide Area Network) est le côté « extérieur » du pare-feu, celui qui mène à Internet. Le **LAN** (Local Area Network) est le réseau interne à protéger. Le **NAT** (Network Address Translation) permet à plusieurs machines internes de sortir sur Internet derrière une seule adresse : les machines du LAN ne sont donc pas directement joignables depuis l'extérieur.

Cette architecture apporte trois avantages pédagogiques :

1. **Un vrai périmètre réseau.** OPNsense matérialise la frontière entre « Internet » (ou notre machine physique) et le réseau interne de SenTech Solutions, exactement comme un pare-feu d'entreprise réel. Les règles iptables que nous configurerons sur Debian Serveur deviennent alors une deuxième ligne de défense, derrière le pare-feu périmétrique : c'est le concept de sécurité en couches (« defense in depth »).
2. **Un accès Internet maîtrisé.** `apt update`, `freshclam` (ClamAV), la synchronisation NTP et le téléchargement des signatures OpenVAS/Greenbone ont tous besoin d'Internet. OPNsense fournit cet accès à toutes les machines du LAN par une seule sortie NAT contrôlée, plutôt que de donner à chaque machine une carte réseau supplémentaire directement exposée.

3. **Un poste d'administration réaliste.** Dans une architecture avec pare-feu périmétrique, la machine physique n'a normalement pas d'accès direct au réseau interne protégé : c'est justement le rôle du pare-feu de l'empêcher. Kali Linux, qui se trouve déjà sur le LAN, reprend donc ce rôle de poste d'administration, ce qui est d'ailleurs l'usage réel d'une distribution comme Kali en entreprise (poste de test et d'audit dédié).

2. Plan d'adressage IP définitif

Machine	Rôle	Interface	Adresse IP	Passerelle
OPNsense	Pare-feu / routeur	WAN (vtnet0)	DHCP (réseau NAT libvirt, 192.168.100.0/24)	192.168.100.1
OPNsense	Pare-feu / routeur	LAN (vtnet1)	192.168.10.1/24	non applicable
Kali Linux	Administration, attaque et audit	eth0	192.168.10.99/24	192.168.10.1
Debian Serveur	Serveur cible	enp1s0	192.168.10.10/24	192.168.10.1
Debian Client	Sauvegarde et poste de bureau avec antivirus	enp1s0	192.168.10.11/24	192.168.10.1

Toutes les adresses du LAN sont **statiques** et le service DHCP d'OPNsense reste désactivé sur l'interface LAN. Choisir des adresses fixes permet d'écrire des règles de pare-feu précises (par exemple « autoriser SSH uniquement depuis 192.168.10.0/24 ») et de retrouver toujours chaque machine à la même adresse.

PARTIE II

Partie II : Mise en place de l'infrastructure QEMU/KVM

1. Création des réseaux virtuels libvirt

Deux réseaux virtuels sont nécessaires : `wan-sentech` (NAT, pour l'accès Internet d'OPNsense) et `lan-sentech` (isolé, le réseau interne 192.168.10.0/24). Chaque réseau est décrit dans un fichier XML, puis déclaré, démarré et marqué en démarrage automatique avec `virsh`.

1-1 Réseau WAN (NAT)

Sur l'hôte Ubuntu

```
mkdir -p ~/isos
cat > ~/isos/wan-sentech.xml << 'EOF'
<network>
  <name>wan-sentech</name>
  <forward mode='nat'/>
  <bridge name='virbr-wan' stp='on' delay='0'/>
  <ip address='192.168.100.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='192.168.100.10' end='192.168.100.100'/>
    </dhcp>
  </ip>
</network>
EOF

virsh net-define ~/isos/wan-sentech.xml
virsh net-start wan-sentech
virsh net-autostart wan-sentech
```

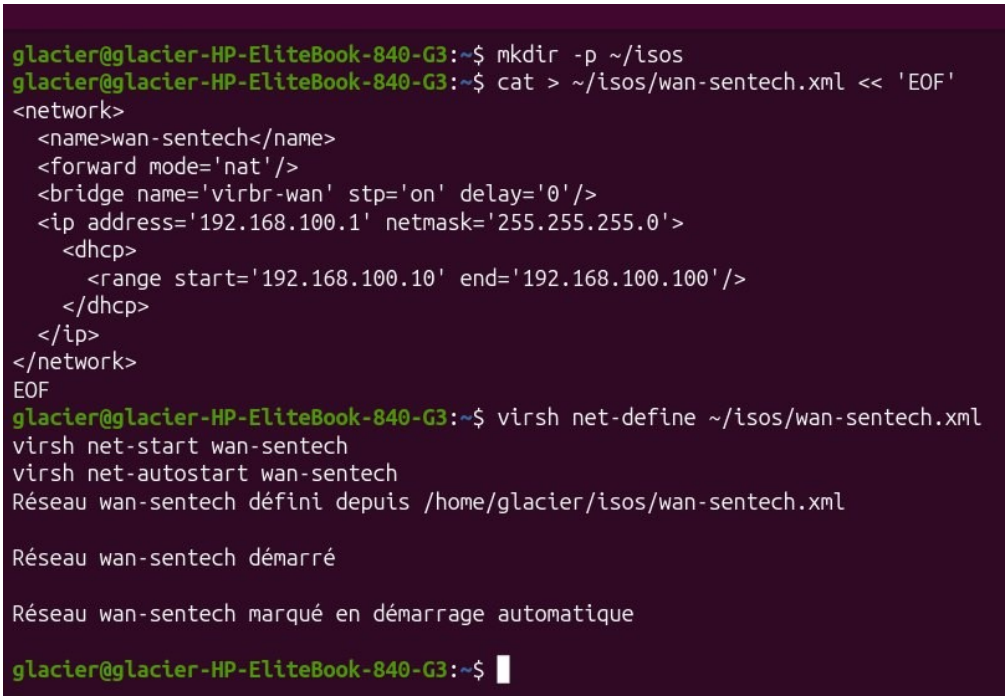


Figure 2 : *Définition, démarrage et démarrage automatique du réseau wan-sentech*

Ce réseau fournit une adresse IP par DHCP (plage 192.168.100.10 à 192.168.100.100) à l'interface WAN d'OPNsense, et effectue le NAT vers la machine physique pour sortir vers Internet.

POUR COMPRENDRE **les trois commandes virsh**

`net-define` enregistre le réseau à partir du fichier XML. `net-start` l'active immédiatement. `net-autostart` le réactive à chaque démarrage de l'hôte. Sans cette dernière commande, il faudrait relancer les réseaux à la main après chaque redémarrage de l'ordinateur.

1-2 Réseau LAN (isolé, 192.168.10.0/24)

Ce réseau n'a ni `<forward>` ni adresse IP côté hôte : c'est un simple « commutateur virtuel » qui relie les machines entre elles et à OPNsense. L'hôte physique n'y a donc aucun accès, ce qui reproduit bien un réseau interne protégé.

Sur l'hôte Ubuntu

```
cat > ~/isos/lan-sentech.xml << 'EOF'
<network>
  <name>lan-sentech</name>
  <bridge name='virbr-lan' stp='on' delay='0'/>
</network>
EOF

virsh net-define ~/isos/lan-sentech.xml
virsh net-start lan-sentech
virsh net-autostart lan-sentech
```



Figure 3 : *Définition et démarrage du réseau lan-sentech*

Vérification des deux réseaux :

Sur l'hôte Ubuntu

```
virsh net-list --all
```

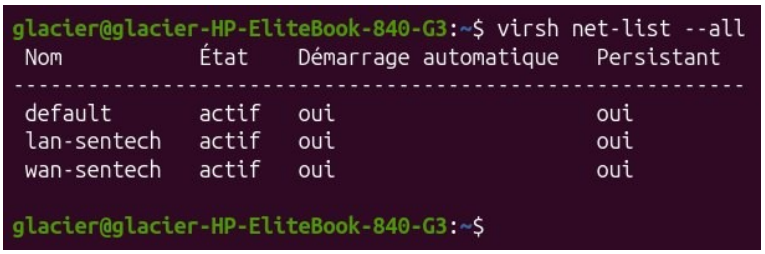


Figure 4 : *Les réseaux wan-sentech et lan-sentech sont actifs et en démarrage automatique*

Le réseau `default` est créé automatiquement par libvirt à l'installation ; nous ne l'utilisons pas.

2. Création des machines virtuelles

Les quatre machines virtuelles sont créées avec virt-manager (assistant « Nouvelle machine virtuelle »). Le point essentiel est le **branchement des cartes réseau** : OPNsense reçoit deux cartes, les autres machines une seule, reliée au réseau isolé.

Machine	Système	Mémoire	Disque	Cartes réseau
OPNsense	OPNsense (ISO)	2 Go	20 Go	1re carte : wan-sentech (vtnet0) ; 2e carte : lan-sentech (vtnet1)
Kali Linux	Kali (ISO)	4 Go	40 Go	lan-sentech uniquement
Debian Serveur	Debian 13 (ISO)	2 Go	20 Go	lan-sentech uniquement
Debian Client	Debian 13 (ISO)	2 Go	20 Go	lan-sentech uniquement

REMARQUE

Ces valeurs de mémoire et de disque sont indicatives : elles sont suffisantes pour le laboratoire. Le type de carte réseau « virtio » est recommandé car il est le plus performant ; c'est lui qui donne les noms `vtnet0` et `vtnet1` côté OPNsense.

PARTIE III

Partie III : Installation et configuration d'OPNsense

1. Installation et assignation des interfaces réseau

L'ISO d'OPNsense est installée sur sa machine virtuelle (identifiant d'installation `installer`, puis choix de l'installation sur le disque virtuel). Au premier démarrage après installation, OPNsense affiche un menu console numéroté. C'est ici que l'on indique quelle carte réseau virtuelle joue le rôle de WAN et laquelle joue le rôle de LAN (option 1 du menu, « Assign interfaces »).

WAN interface : vtnet0 LAN interface : vtnet1

L'option 2 du menu (« Set interface IP address ») permet ensuite de donner à l'interface LAN l'adresse **192.168.10.1/24** et de **refuser** le service DHCP sur le LAN, puisque toutes nos adresses sont statiques.

> ATTENTION **quelle carte est quelle interface ?**
> Si les deux cartes sont inversées, OPNsense n'aura pas d'accès Internet et les machines du LAN ne le verront pas. Pour contrôler, comparez l'adresse MAC affichée par OPNsense avec celle de chaque carte dans virt-manager.

2. Vérification depuis la console

Depuis la console d'OPNsense (option 8, « Shell »), nous vérifions que l'interface LAN porte bien l'adresse 192.168.10.1 et que le pare-feu atteint Internet :

Sur OPNsense (shell)
```
ifconfig vtnet1
ping -c 3 8.8.8.8
```

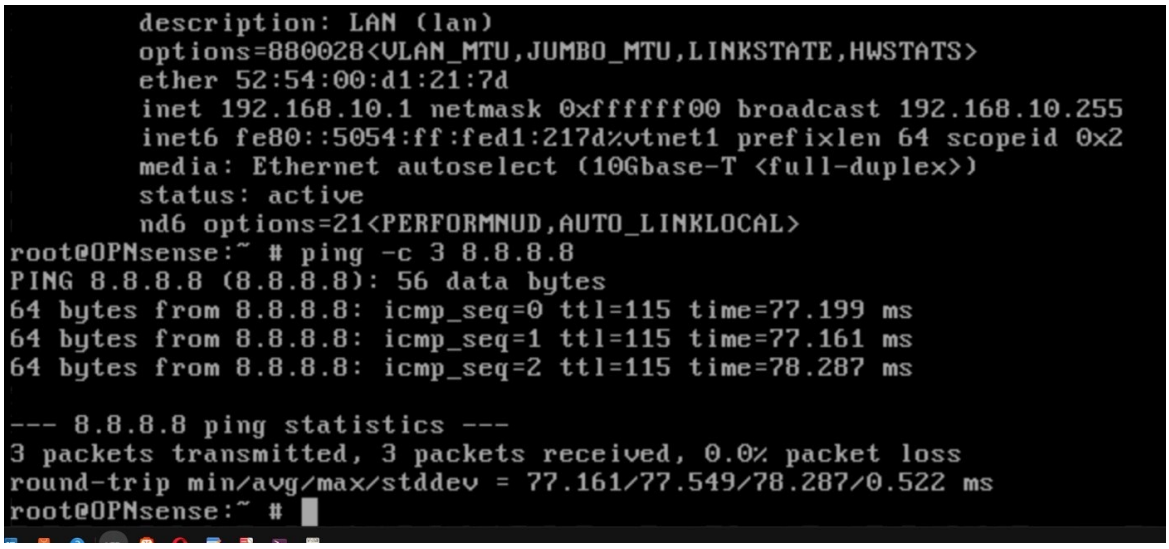


Figure 5 : *OPNsense : le LAN est configuré en 192.168.10.1 et Internet répond (0 % de perte)*

L'interface web d'administration d'OPNsense est ensuite accessible depuis Kali à l'adresse `https://192.168.10.1` (utilisateur `root`). C'est la seule machine du LAN qui sert de poste d'administration.

3. Configuration réseau statique de Kali Linux

Kali utilise NetworkManager par défaut sur l'image desktop. La méthode la plus simple est `nmcli`, en ligne de commande, depuis un terminal Kali :

Sur Kali Linux

```
nmcli connection show
sudo nmcli connection modify "Wired connection 1" \
  ipv4.addresses 192.168.10.99/24 \
  ipv4.gateway 192.168.10.1 \
  ipv4.dns "192.168.10.1 1.1.1.1" \
  ipv4.method manual
sudo nmcli connection up "Wired connection 1"
```

Vérification :

Sur Kali Linux

```
ip addr show
ping -c 3 192.168.10.1
ping -c 3 8.8.8.8
sudo apt update && sudo apt full-upgrade -y
```

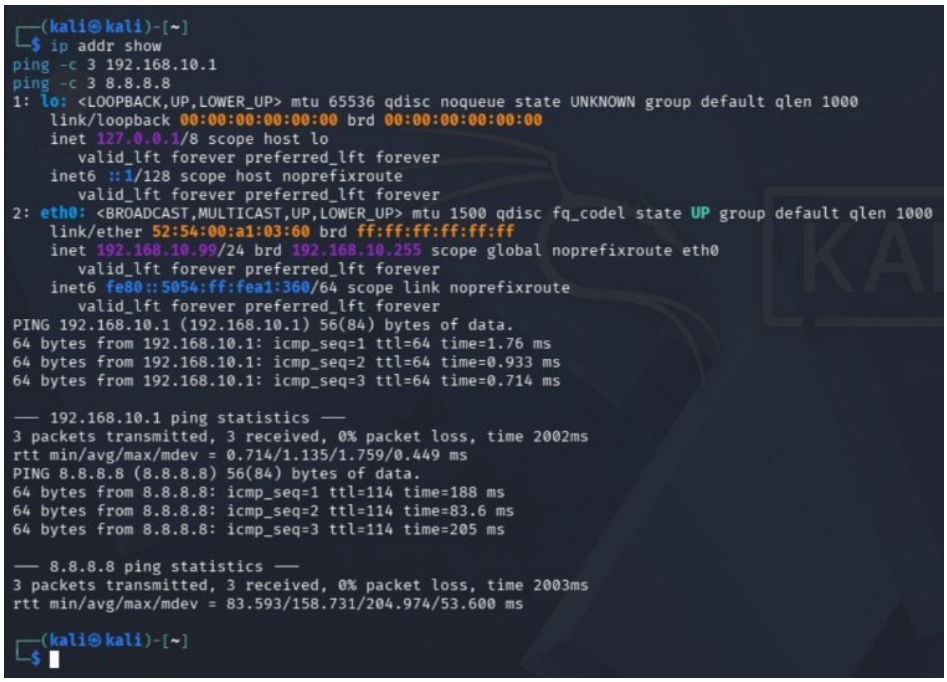


Figure 6 : *Kali : adresse 192.168.10.99/24, la passerelle et Internet répondent*

POUR COMPRENDRE **les trois pings**

Le premier ping (192.168.10.1) prouve que la machine voit le pare-feu. Le second (8.8.8.8, un serveur DNS public) prouve que le trafic traverse OPNsense et sort sur Internet. Si le premier échoue, le problème est local (adresse, câblage virtuel). Si seul le second échoue, le problème vient d'OPNsense ou du réseau WAN.

4. Configuration réseau statique de Debian Serveur et Debian Client

Les deux machines Debian utilisent le fichier `/etc/network/interfaces`. Le nom de l'interface (ici `enp1s0`) peut être vérifié avec `ip -br link`. Seule l'adresse change d'une machine à l'autre : 192.168.10.10 pour le serveur, 192.168.10.11 pour le client.

Sur Debian Serveur et Debian Client (en root)
```
nano /etc/network/interfaces
```

Contenu du fichier sur **Debian Client** (remplacer l'adresse par 192.168.10.10 sur Debian Serveur) :

/etc/network/interfaces
```
auto lo
iface lo inet loopback

auto enp1s0
iface enp1s0 inet static
    address 192.168.10.11
    netmask 255.255.255.0
    gateway 192.168.10.1
    dns-nameservers 192.168.10.1 1.1.1.1
```

Appliquer la configuration, puis tester :

Sur Debian Serveur et Debian Client
```
systemctl restart networking
ping -c 3 192.168.10.1
ping -c 3 8.8.8.8
apt update && apt upgrade -y
```

5. Vérification de la connectivité des machines

Chaque machine doit pouvoir joindre les trois autres et Internet. Les captures montrent les pings depuis Kali, depuis Debian Client et depuis Debian Serveur.

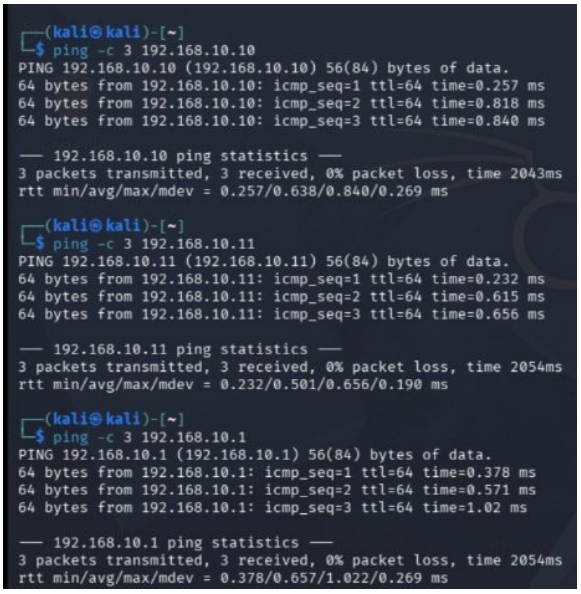


Figure 7 : *Ping depuis Kali vers 192.168.10.10, 192.168.10.11 et 192.168.10.1*

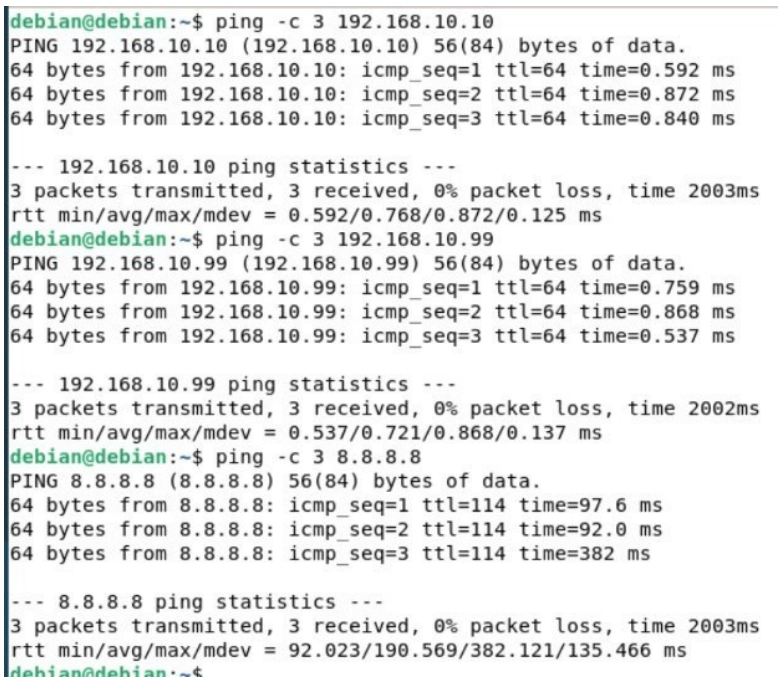


Figure 8 : *Ping depuis Debian Client vers le serveur, Kali, Internet et OPNsense*

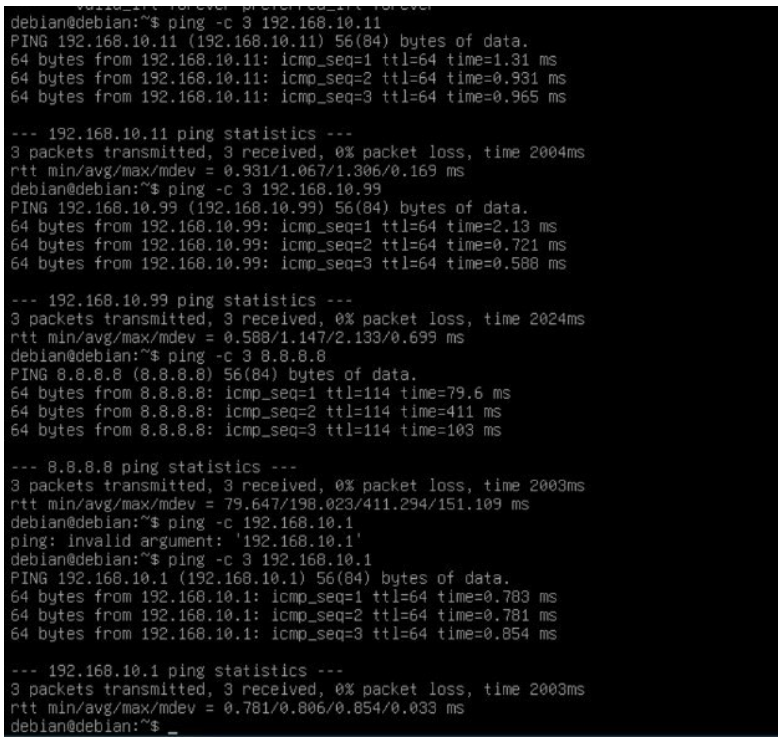


Figure 9 : *Ping depuis Debian Serveur vers le client, Kali, Internet et OPNsense*

6. Tableau récapitulatif des accès

Machine	IP	Utilisateur admin	Mot de passe	Accès initial
OPNsense	192.168.10.1	root	défini à l'installation	Interface web https://192.168.10.1
Debian Serveur	192.168.10.10	debian	défini à l'installation	Console virt-manager ; SSH (port 2222 après durcissement)
Debian Client	192.168.10.11	debian	défini à l'installation	Console graphique ; SSH port 22
Kali Linux	192.168.10.99	kali	défini à l'installation	Console graphique

ATTENTION **les mots de passe**

Les mots de passe ne sont volontairement pas reproduits dans ce rapport. Dans un vrai projet, ils sont conservés dans un gestionnaire de mots de passe, jamais dans un document.

PARTIE IV

Partie IV : Administration de base

Partie 1 : Gestion des utilisateurs

SenTech Solutions compte trois équipes : les développeurs, l'équipe finance et les administrateurs système. Chaque équipe doit avoir ses propres accès, ni plus ni moins. C'est le **principe du moindre privilège** : on n'accorde que ce qui est strictement nécessaire.

Création des utilisateurs

Les commandes suivantes sont exécutées sur **Debian Serveur**. L'option `-m` crée le dossier personnel, `-s /bin/bash` donne un interpréteur de commandes, et `-c` ajoute le nom complet dans le champ de commentaire.

Sur Debian Serveur
```
# alice : développeuse
sudo useradd -m -s /bin/bash -c "Alice Diallo" alice
# bob : comptable
sudo useradd -m -s /bin/bash -c "Bob Sow" bob
# charlie : administrateur système
sudo useradd -m -s /bin/bash -c "Charlie Ndiaye" charlie

# Définir les mots de passe (saisie interactive)
sudo passwd alice
sudo passwd bob
sudo passwd charlie
```

Vérification : affichons les trois nouveaux utilisateurs dans le fichier de base des comptes.

Sur Debian Serveur
```
grep -E "alice|bob|charlie" /etc/passwd
```

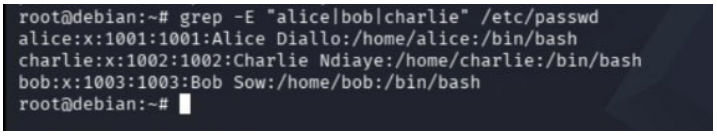


Figure 10 : *Les trois comptes apparaissent dans /etc/passwd*

Création des groupes

Chaque équipe a son propre groupe Linux, ce qui permet d'attribuer des permissions à plusieurs utilisateurs d'un coup.

Sur Debian Serveur
```
sudo groupadd dev_team
sudo groupadd finance
sudo groupadd admins_sys
```

Vérifier leur création :

Sur Debian Serveur
```
getent group dev_team
getent group finance
getent group admins_sys
```

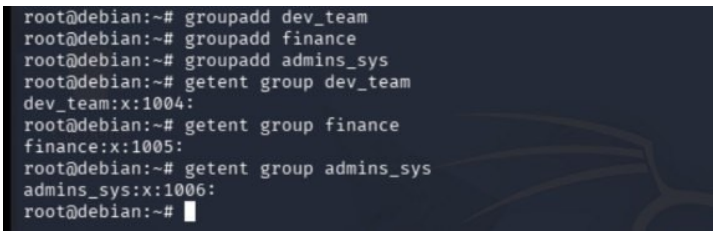


Figure 11 : *Les trois groupes existent (encore vides)*

Affectation des utilisateurs aux groupes

L'option `-aG` **ajoute** l'utilisateur à un groupe sans le retirer de ses autres groupes. Sans le `-a`, l'utilisateur perdrait tous ses groupes secondaires.

Sur Debian Serveur

```
# Alice rejoint l'équipe développement
sudo usermod -aG dev_team alice
# Bob rejoint l'équipe finance
sudo usermod -aG finance bob
# Charlie rejoint l'équipe administration
sudo usermod -aG admins_sys charlie
```

Vérification :

Sur Debian Serveur

```
getent group dev_team finance admins_sys
```

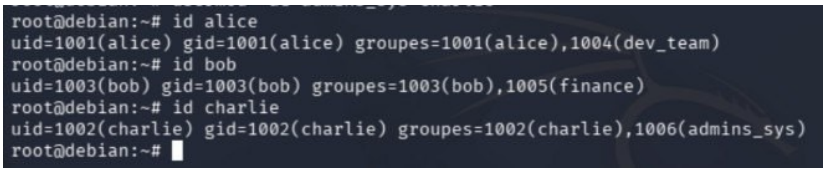


Figure 12 : *Chaque utilisateur est membre de son groupe*

Exploration des fichiers de comptes

Linux stocke les comptes dans trois fichiers, chacun avec un niveau de confidentialité différent :

Fichier	Lisible par	Contenu
/etc/passwd	tous	Comptes : nom, identifiant (UID), dossier personnel, shell
/etc/shadow	root uniquement	Mots de passe hachés (jamais en clair) et leur durée de validité
/etc/group	tous	Groupes et liste de leurs membres

Sur Debian Serveur

```
# Fichier des comptes (lisible par tous)
cat /etc/passwd | grep -E "alice|bob|charlie"
# Fichier des mots de passe hachés (root uniquement)
sudo cat /etc/shadow | grep -E "alice|bob|charlie"
# Fichier des groupes
cat /etc/group | grep -E "dev_team|finance|admins_sys"
```

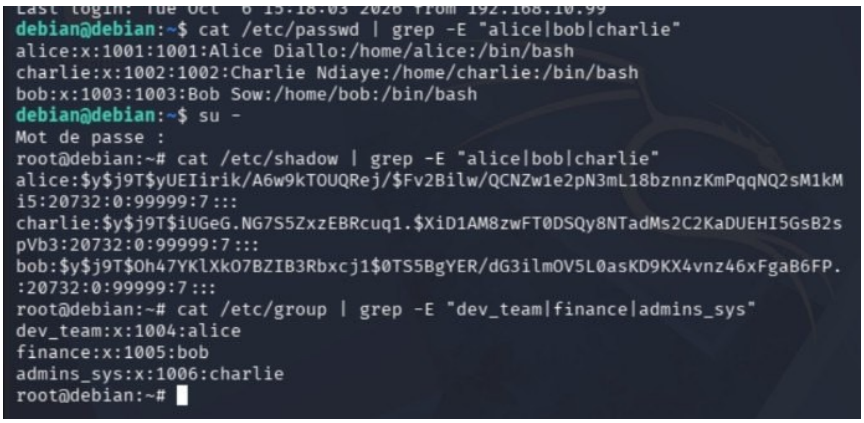


Figure 13 : *Contenu de /etc/passwd, /etc/shadow et /etc/group pour nos trois utilisateurs*

Partie 2 : Permissions et ACL

Contexte métier. SenTech Solutions stocke ses données dans `/srv/sentech/`. Chaque équipe doit accéder uniquement à son dossier. Le dossier public est accessible à tous en lecture. Le dossier backup est réservé à root.

Création de l'arborescence

Sur Debian Serveur

```
sudo mkdir -p /srv/sentech/{dev,finance,public,backup}
ls -la /srv/sentech/
```

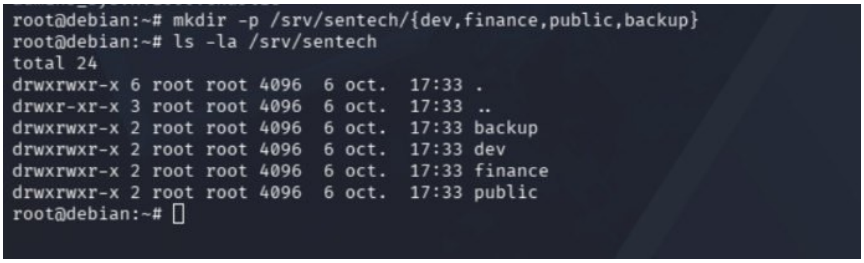


Figure 14 : *Les quatre dossiers existent ; tous appartiennent à root:root avec les permissions par défaut (755)*

Application des permissions

Les permissions Unix s'écrivent avec trois chiffres : propriétaire, groupe, autres. Chaque chiffre additionne **4** (lecture, `r`), **2** (écriture, `w`) et **1** (exécution ou traversée, `x`).

Valeur	Écriture	Signification
770	rwxrwx---	Propriétaire et groupe : tout. Autres : rien
755	rwxr-xr-x	Propriétaire : tout. Groupe et autres : lecture et traversée
700	rwx------	Propriétaire uniquement

Sur Debian Serveur

```
# dev : propriété de root:dev_team, rwxrwx--- (770)
sudo chown root:dev_team /srv/sentech/dev
sudo chmod 770 /srv/sentech/dev

# finance : propriété de root:finance, rwxrwx--- (770)
sudo chown root:finance /srv/sentech/finance
sudo chmod 770 /srv/sentech/finance

# public : lisible et traversable par tous, rwxr-xr-x (755)
sudo chmod 755 /srv/sentech/public

# backup : root uniquement, rwx------ (700)
sudo chmod 700 /srv/sentech/backup
```

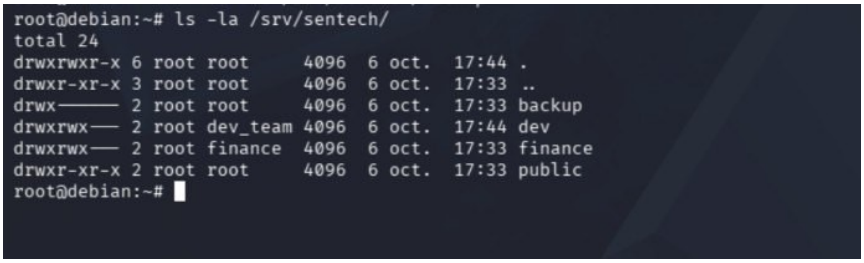


Figure 15 : *Permissions appliquées sur les quatre dossiers*

Les ACL (Access Control List)

Les permissions Unix classiques ont une limite : on ne peut définir des droits que pour un propriétaire, un groupe et les autres. Que faire si alice doit avoir accès à un fichier dans finance sans être membre du groupe finance ? C'est là qu'interviennent les **ACL** : elles permettent des permissions fines, fichier par fichier ou utilisateur par utilisateur.

Sur Debian Serveur
```
sudo apt install acl -y
```

Scénario : le directeur souhaite qu'alice puisse lire les rapports financiers (lecture seule), sans être membre du groupe finance.

Sur Debian Serveur
```
# Créer un fichier test dans finance
echo "Rapport Q4 2026 — Confidentiel" | sudo tee /srv/sentech/finance/rapport_Q4.txt

# 1. Autoriser alice à lire le fichier
sudo setfacl -m u:alice:r-- /srv/sentech/finance/rapport_Q4.txt
# 2. Autoriser alice à traverser le dossier parent (sans pouvoir y lister ni écrire)
sudo setfacl -m u:alice:x /srv/sentech/finance

# Vérifier les ACL appliquées
getfacl /srv/sentech/finance/rapport_Q4.txt
```

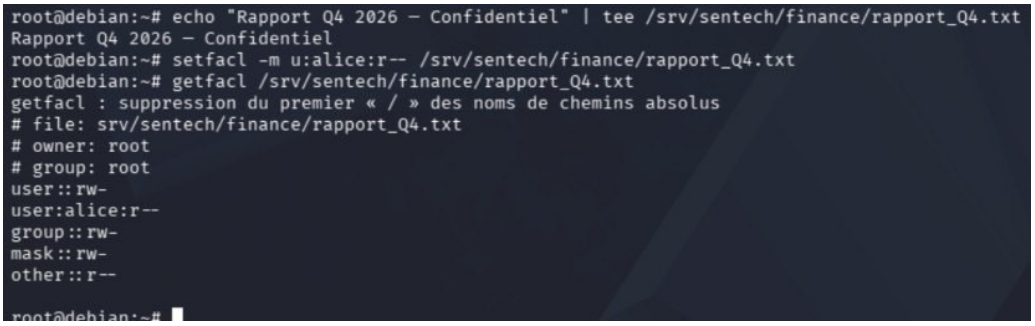


Figure 16 : *ACL du fichier : une ligne user:alice:r-- a été ajoutée*

Test : alice lit le fichier, alors qu'elle n'est pas membre du groupe finance.

Sur Debian Serveur
```
su - alice
cat /srv/sentech/finance/rapport_Q4.txt
```

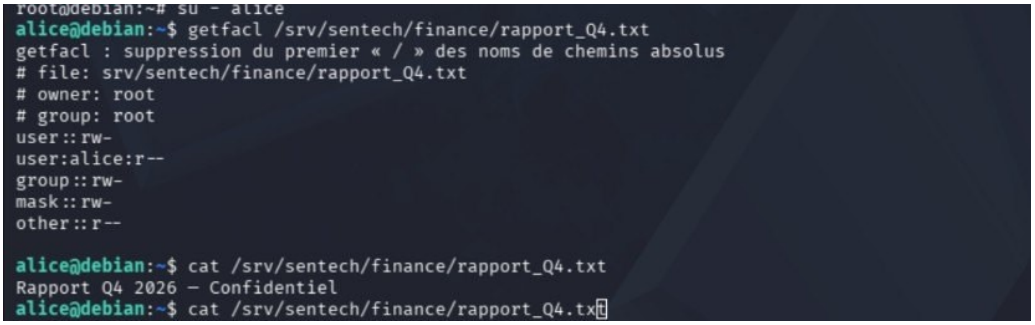


Figure 17 : *Alice peut lire le rapport grâce à l'ACL*

> POUR COMPRENDRE **pourquoi la deuxième commande ?**
> Pour lire un fichier, il ne suffit pas d'avoir le droit sur le fichier : il faut aussi pouvoir **traverser** (droit x) chaque dossier du chemin. Sans le x sur `/srv/sentech/finance`, alice recevrait « Permission refusée » même avec l'ACL sur le fichier. On accorde x seulement : alice peut atteindre le fichier qu'on lui a désigné, mais pas lister le contenu du dossier.

Partie 3 : Sécurisation par SSH

Contexte métier. Les administrateurs de SenTech doivent gérer les serveurs à distance. Utiliser un mot de passe pour SSH est risqué : les journaux de n'importe quel serveur exposé montrent des centaines de tentatives d'intrusion par jour. La bonne pratique en entreprise est l'**authentification par paire de clés**.

Nous générons donc la paire de clés depuis Kali Linux, qui joue ici le rôle de poste d'administration de SenTech Solutions, exactement comme le ferait un administrateur système utilisant une machine d'audit dédiée.

> POUR COMPRENDRE **la paire de clés**
> La **clé privée** reste secrète sur le poste de l'administrateur. La **clé publique** est déposée sur les serveurs. À la connexion, le serveur envoie un défi que seule la clé privée peut résoudre : aucun mot de passe ne circule sur le réseau, et il n'y a plus rien à deviner par force brute.

Génération de la paire de clés

Sur Kali Linux

```
# Créer le dossier .ssh s'il n'existe pas
mkdir -p ~/.ssh
chmod 700 ~/.ssh

# Générer une paire de clés ed25519 (algorithme recommandé)
ssh-keygen -t ed25519 -C "admin@sentech" -f ~/.ssh/sentech_ed25519
```

Chaque option a un rôle précis :

Option	Rôle
-t ed25519	Type d'algorithme (moderne, rapide, sûr)
-C "admin@sentech"	Étiquette (commentaire) qui identifie la clé
-f ~/.ssh/sentech_ed25519	Emplacement où enregistrer la clé

La commande demande une **passphrase** (phrase de passe). C'est une couche de sécurité supplémentaire : même si quelqu'un vole notre fichier de clé privée, il ne pourra pas l'utiliser sans connaître la passphrase.

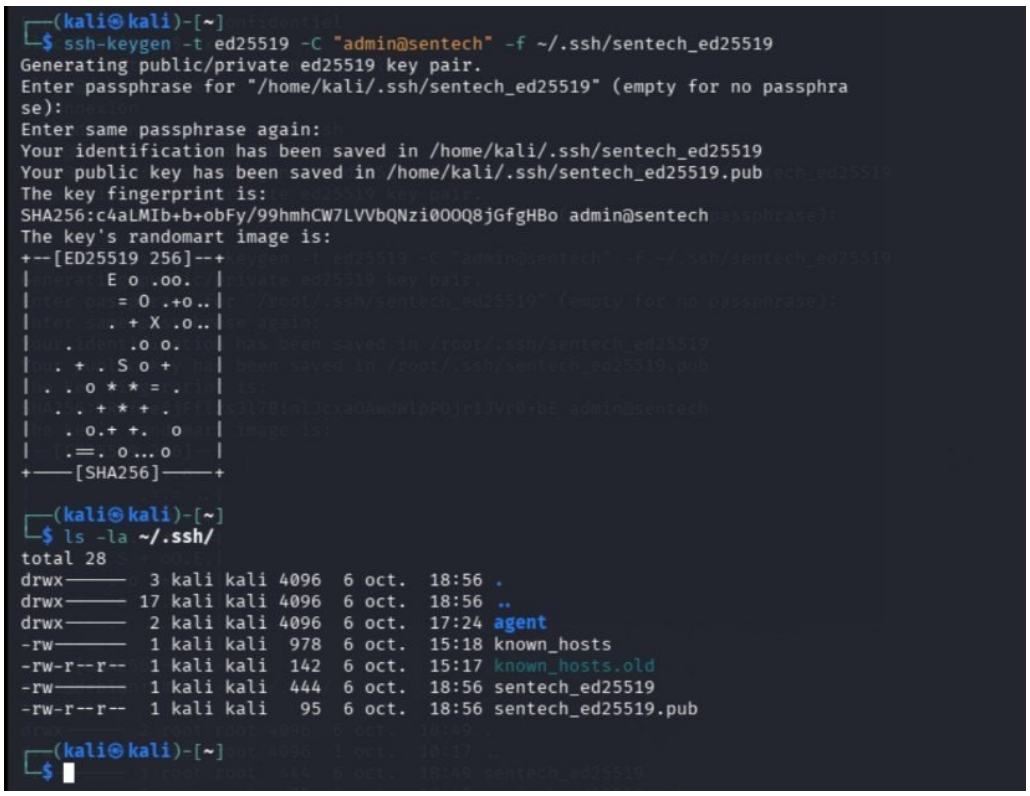


Figure 18 : *Génération de la clé ed25519 sur Kali*

Deux fichiers sont créés : `sentech_ed25519` est notre **clé privée** (à ne jamais partager) et `sentech_ed25519.pub` est notre **clé publique** (à déposer sur les serveurs).

Copie de la clé publique vers le serveur

À ce stade SSH écoute encore sur le port 22 et accepte les mots de passe : c'est ce qui permet de déposer la clé une première fois.

Sur Kali Linux
```
ssh-copy-id -i ~/.ssh/sentech_ed25519.pub debian@192.168.10.10
```

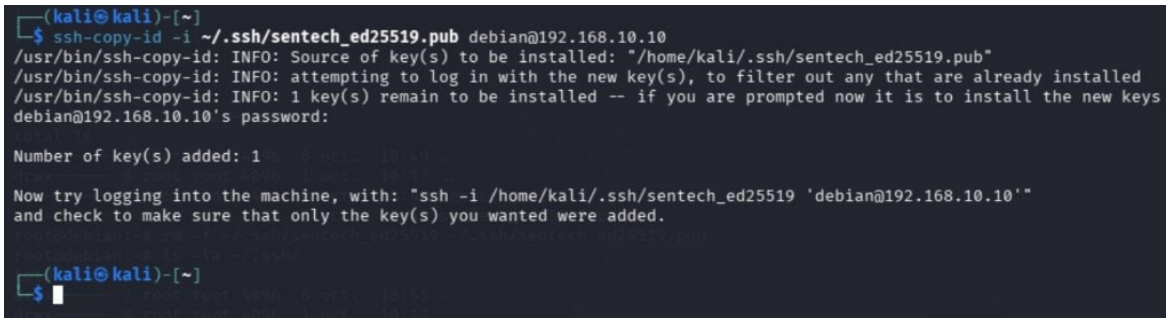


Figure 19 : *La clé publique est installée dans ~/.ssh/authorized_keys du serveur*

Vérification : connectons-nous en SSH avec notre clé (sans mot de passe de compte cette fois ; seule la passphrase de la clé est demandée).

Sur Kali Linux
```
ssh -i ~/.ssh/sentech_ed25519 debian@192.168.10.10
```

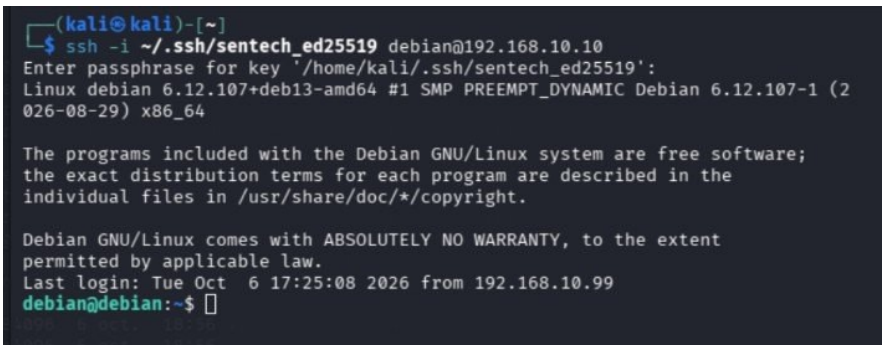


Figure 20 : *Connexion par clé réussie*

Durcissement de la configuration SSH

Sur Debian Serveur, nous éditons le fichier de configuration du service SSH.

Sur Debian Serveur
```
sudo nano /etc/ssh/sshd_config
```

Nous localisons et modifions les paramètres suivants (les lignes qui commencent par `#` sont des commentaires) :

/etc/ssh/sshd_config
```
# Changer le port par défaut (22 -> 2222)
# Réduit le bruit dans les logs : les scanners automatiques ciblent le port 22
Port 2222

# Interdire la connexion directe en root
# Si un attaquant obtient un accès root direct, c'est terminé
PermitRootLogin no

# Désactiver l'authentification par mot de passe
# Seules les clés SSH seront acceptées
PasswordAuthentication no

# Activer explicitement l'authentification par clé
PubkeyAuthentication yes

# Limiter les tentatives d'authentification
MaxAuthTries 3

# Réduire le délai de connexion (défaut : 120 secondes)
LoginGraceTime 20

# Autoriser uniquement les utilisateurs spécifiques
AllowUsers debian alice charlie

# Désactiver les fonctionnalités non nécessaires
X11Forwarding no
AllowAgentForwarding no
AllowTcpForwarding no

# Afficher un message d'avertissement à la connexion
PrintMotd yes
```

ATTENTION **ne pas s'enfermer dehors**

`AllowUsers` doit contenir **le compte avec lequel vous administrez** le serveur (ici `debian`). Si ce compte est absent de la liste, vous serez refusé dès le prochain redémarrage de SSH. De même, `PasswordAuthentication no` ne doit être activé qu'**après** avoir vérifié que la connexion par clé fonctionne. Gardez toujours la console de virt-manager ouverte pendant ces manipulations : elle reste utilisable même si SSH est cassé.

Test de la configuration sans redémarrage du service :

Sur Debian Serveur
```
sudo sshd -t
```

Si aucune erreur n'est affichée, la syntaxe est correcte. On redémarre alors le service (sur Debian il s'appelle `ssh`) :

Sur Debian Serveur
```
sudo systemctl restart ssh
```

Bannière d'avertissement

Le fichier `/etc/issue.net` contient le message affiché **avant** l'authentification (c'est la directive `Banner` de SSH). À ne pas confondre avec le « message du jour » (MOTD, fichier `/etc/motd`) qui s'affiche après la connexion.

Sur Debian Serveur
```
sudo nano /etc/issue.net
```

/etc/issue.net
```
**********************************************************
**** SenTech Solutions - Systeme d'information interne ****
**********************************************************
* Acces reserve aux utilisateurs autorises uniquement.
* Toute connexion est enregistree et peut faire l'objet
* de poursuites en cas d'utilisation non autorisee.
**********************************************************
```



Figure 21 : *Fichier /etc/issue.net*

Activation de la bannière dans sshd_config : ajouter (ou décommenter) la ligne `Banner /etc/issue.net`, puis redémarrer le service.

Sur Debian Serveur
```
sudo nano /etc/ssh/sshd_config
# Ajouter ou décommenter :
#   Banner /etc/issue.net
sudo systemctl restart ssh
```

Test final de la configuration SSH

Connexion par clé sur le nouveau port (doit réussir) :

Sur Kali Linux
```
ssh -p 2222 -i ~/.ssh/sentech_ed25519 debian@192.168.10.10
```

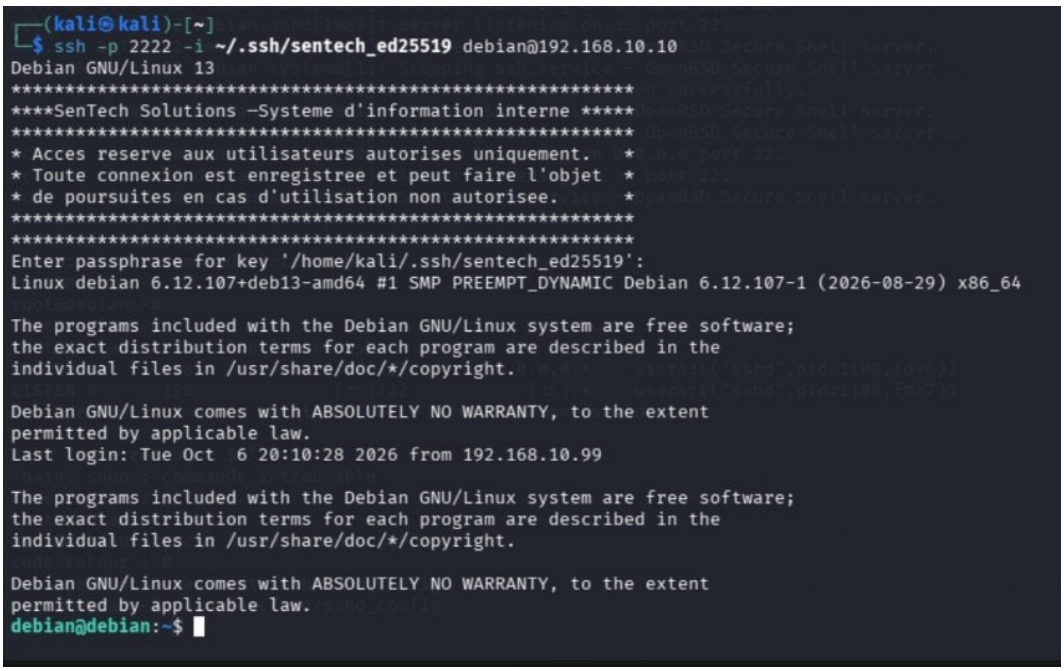


Figure 22 : *Connexion par clé sur le port 2222 : la bannière s'affiche puis l'accès est accordé*

Test de l'authentification par mot de passe (doit échouer) :

Sur Kali Linux

```
ssh -p 2222 -o PreferredAuthentications=password -o PubkeyAuthentication=no
debian@192.168.10.10
```

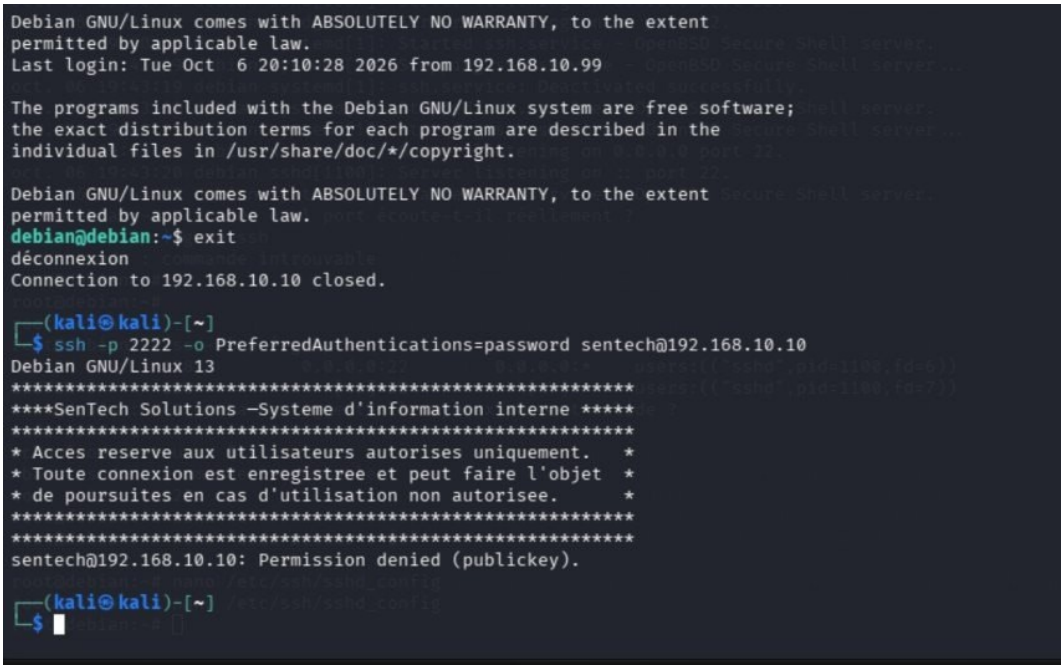


Figure 23 : *Refus : « Permission denied (publickey) », seule la clé est acceptée*

Vérification que le port 22 est fermé et que le port 2222 est ouvert :

Sur Kali Linux

```
nmap -p 22,2222 192.168.10.10
```

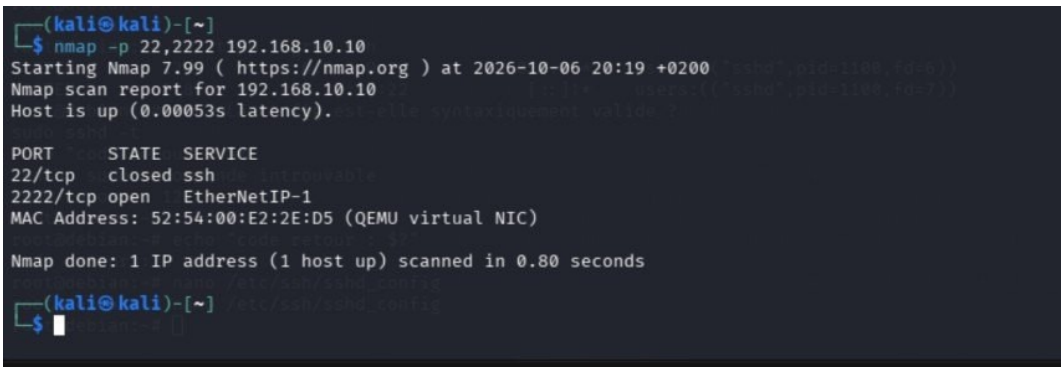


Figure 24 : *Nmap : port 22 fermé, port 2222 ouvert*

Le nom de service affiché par Nmap pour le port 2222, `EtherNetIP-1`, est trompeur : Nmap devine le nom du service à partir d'une base de ports connus, et le port 2222 est associé par convention à un autre protocole industriel. Le service réellement présent est bien OpenSSH, comme le confirmera le scan avec détection de version (`nmap -sV`) réalisé plus loin.

PARTIE V

Partie V : Hardening Linux : durcissement de Debian Serveur

Le **hardening** (durcissement) consiste à désactiver les services qui ne servent à rien, mettre en place un pare-feu, bloquer automatiquement les attaquants, enregistrer toutes les actions importantes et renforcer l'authentification. Le principe fondamental reste le moindre privilège : ce qui n'est pas nécessaire est supprimé ou interdit.

Les outils utilisés

iptables et UFW. Le noyau Linux intègre Netfilter, le système de filtrage des paquets. `iptables` configure ces règles directement, avec une syntaxe puissante mais complexe. UFW (Uncomplicated Firewall) est une interface simplifiée au-dessus d'iptables.

Fail2Ban lit les journaux en temps réel, détecte des échecs répétés et bannit automatiquement l'adresse IP de l'attaquant au moyen d'une règle de pare-feu temporaire. Trois paramètres le gouvernent :

Paramètre	Signification	Valeur recommandée
maxretry	Nombre d'échecs avant bannissement	3 à 5
findtime	Fenêtre de temps pour compter les échecs	600 s (10 min)
bantime	Durée du bannissement	3600 s (1 h) ou -1 (permanent)

Auditd est la « boîte noire » du système : il enregistre des événements précis (modification de `/etc/passwd`, commandes root, changements de permissions…), indispensables en cas d'investigation après un incident.

Partie 1 : Audit initial, état des lieux avant hardening

On ne peut pas mesurer une amélioration sans connaître le point de départ. Nous relevons donc les services actifs et les ports ouverts **avant** toute modification.

Sur Debian Serveur (via SSH depuis Kali)

```
systemctl list-units --type=service --state=active --no-pager
systemctl list-units --type=service --state=active --no-pager | wc -l
systemctl list-unit-files --type=service --state=enabled --no-pager
ss -tlnp
ss -ulnp
```

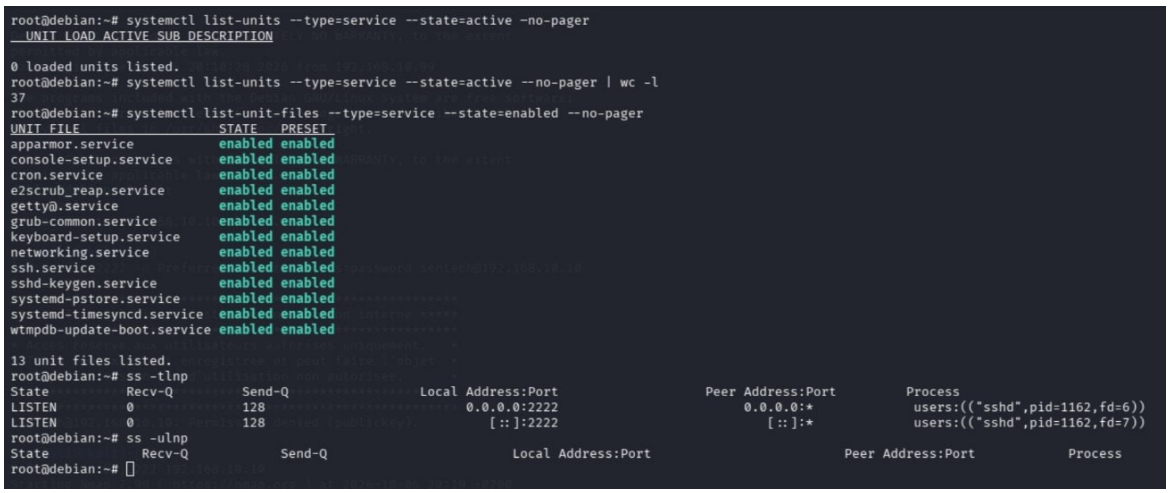


Figure 25 : *Services actifs et ports en écoute avant durcissement*

POUR COMPRENDRE **ss -tlnp**

`ss` liste les connexions réseau. Les options `-t` (TCP), `-u` (UDP), `-l` (en écoute), `-n` (adresses numériques) et `-p` (programme propriétaire) donnent la liste des « portes ouvertes » du serveur et du programme qui se cache derrière chacune.

Scan externe depuis Kali : c'est le point de vue d'un attaquant qui découvre la machine.

Sur Kali Linux

```
nmap -sV 192.168.10.10 -oN /tmp/scan_avant_hardening.txt
ls -la /tmp/scan_avant_hardening.txt
cat /tmp/scan_avant_hardening.txt
```

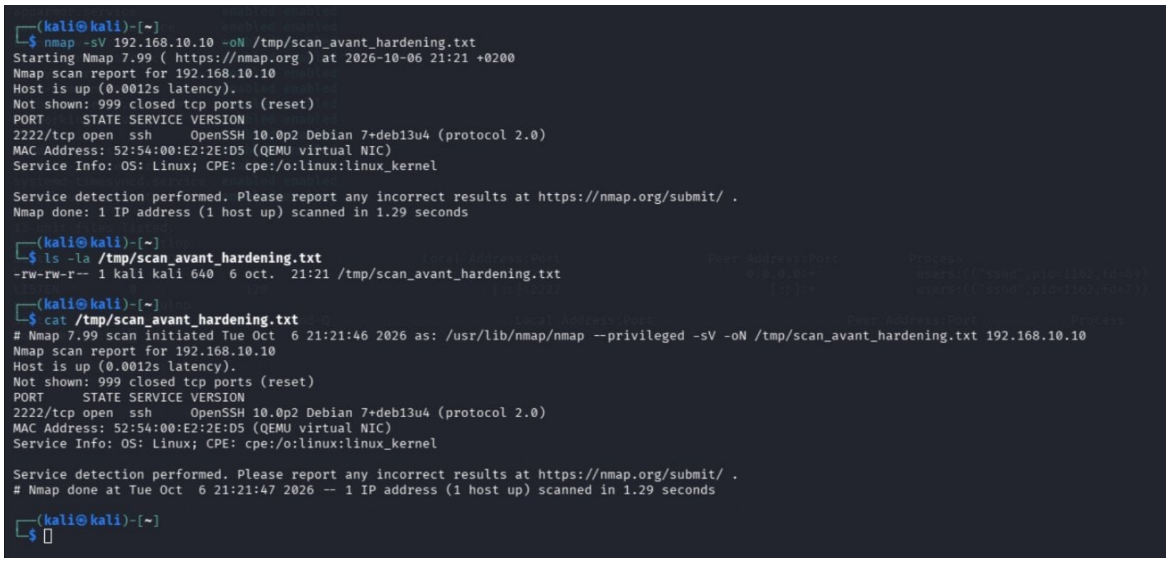


Figure 26 : *Scan Nmap avant hardening (sauvegardé pour la comparaison finale)*

Vérification de l'état du pare-feu :

Sur Debian Serveur

```
sudo ufw status
sudo iptables -L -n -v
```

Dans un état par défaut, UFW n'est pas encore installé ni actif et iptables n'a aucune règle de filtrage : toutes les connexions entrantes sont acceptées.

Partie 2 : Désactivation des services inutiles

Chaque service actif est une porte d'entrée potentielle. Nous vérifions l'état de ceux dont une machine de ce type n'a pas besoin (impression, découverte réseau, gestionnaire de modems), puis nous les désactivons. `2>/dev/null` masque simplement le message d'erreur lorsque le service n'est pas installé.

Sur Debian Serveur

```
systemctl status cups 2>/dev/null
systemctl status avahi-daemon 2>/dev/null
systemctl status snapd 2>/dev/null
systemctl status ModemManager 2>/dev/null

sudo systemctl disable --now cups 2>/dev/null
sudo systemctl disable --now cups-browsed 2>/dev/null
sudo systemctl disable --now avahi-daemon 2>/dev/null
sudo systemctl disable --now ModemManager 2>/dev/null

sudo systemctl is-enabled cups 2>/dev/null
sudo systemctl is-active cups 2>/dev/null
```

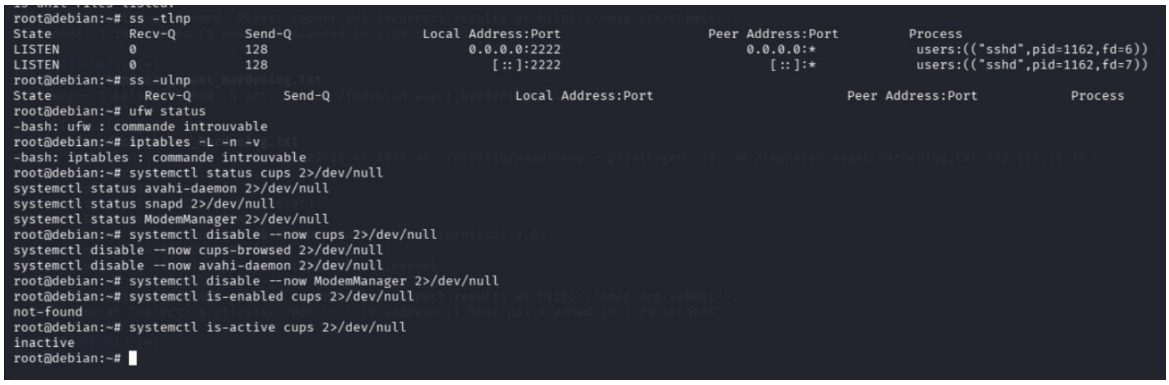


Figure 27 : *Services désactivés ou absents*

Si le paquet est installé, on peut aller plus loin et le supprimer complètement :

Sur Debian Serveur

```
dpkg -l | grep cups
sudo apt purge cups cups-browsed -y 2>/dev/null
sudo apt autoremove -y
systemctl list-units --type=service --state=active --no-pager | wc -l
```

Partie 3 : Configuration du pare-feu local

Cette partie raconte un cheminement réel, avec un obstacle rencontré : nous avons d'abord utilisé UFW, puis constaté qu'il était incompatible avec le paquet `iptables-persistent`, et nous avons finalement construit le pare-feu directement avec iptables.

3-1 Premier essai avec UFW

Sur Debian Serveur

```
sudo apt install ufw -y
sudo ufw version
```

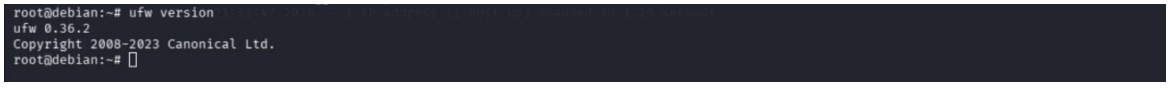


Figure 28 : *Installation d'UFW*

ATTENTION **ordre des opérations critique**

Définissez toujours vos règles **avant** d'activer UFW. Si vous activez UFW avec la politique « deny all » sans avoir autorisé SSH au préalable, vous serez immédiatement déconnecté et ne pourrez plus accéder au serveur à distance.

Sur Debian Serveur

```
# Politique par défaut : tout refuser en entrée, tout autoriser en sortie
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw status verbose

# Autoriser SSH sur le port 2222, restreint au sous-réseau interne
sudo ufw allow from 192.168.10.0/24 to any port 2222 proto tcp comment 'SSH interne SenTech'
sudo ufw show added

# Activer (confirmer avec 'y' quand demandé)
sudo ufw enable
sudo ufw status verbose
```



Figure 29 : *Règles UFW ajoutées et pare-feu activé*

Blocage du ping (ICMP). Avec UFW, les règles ICMP sont particulières : selon la version, `ufw deny proto icmp` peut bloquer tout ICMP (y compris les messages d'erreur nécessaires) ou être ignoré silencieusement. La méthode recommandée est de modifier la règle explicite dans `/etc/ufw/before.rules` :

Sur Debian Serveur

```
sudo nano /etc/ufw/before.rules
# Chercher le bloc :
#   -A ufw-before-input -p icmp --icmp-type echo-request -j ACCEPT
# Remplacer ACCEPT par DROP :
#   -A ufw-before-input -p icmp --icmp-type echo-request -j DROP
sudo ufw reload
```

Test depuis Kali : le SSH sur le port 2222 doit fonctionner (Kali est dans 192.168.10.0/24) et le ping doit être bloqué.

Sur Kali Linux

```
ssh -p 2222 -i ~/.ssh/sentech_ed25519 debian@192.168.10.10
ping 192.168.10.10
```

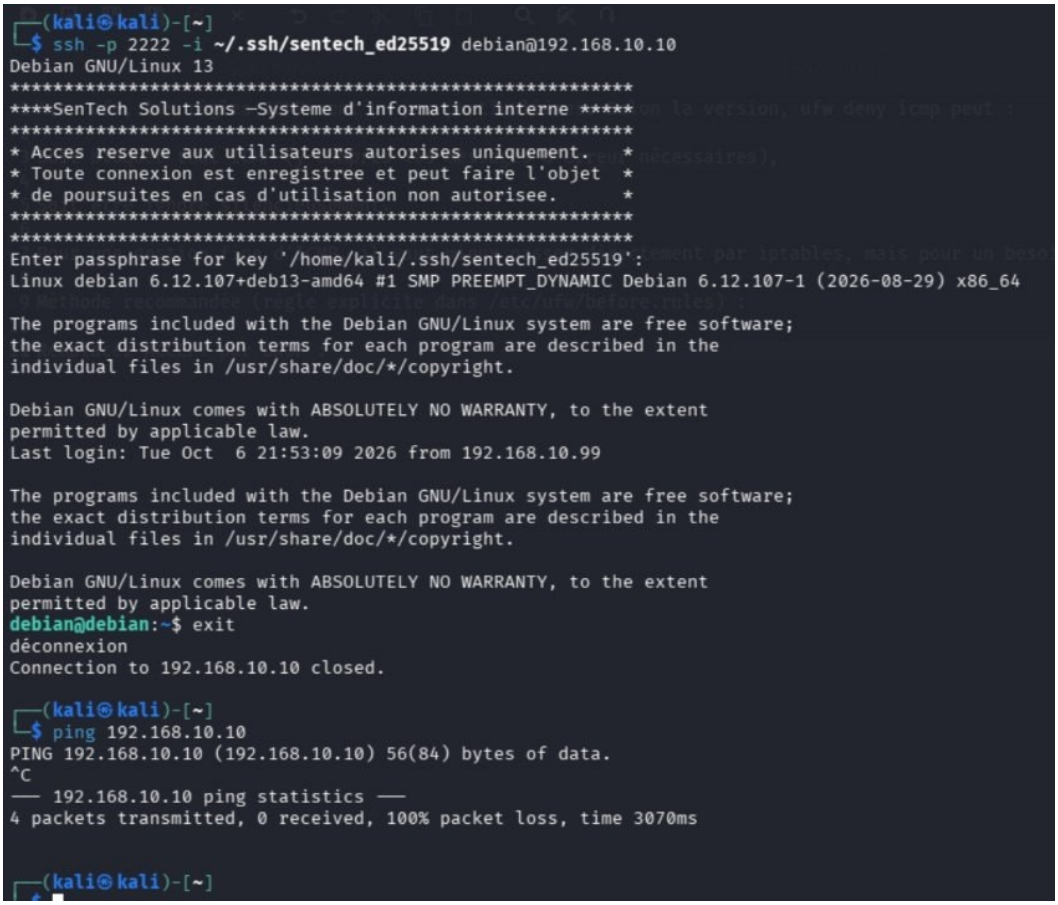


Figure 30 : *SSH accepté, ping sans réponse (100 % de perte)*

Règles avancées et journalisation : blocage explicite des protocoles non chiffrés ou dangereux.

Sur Debian Serveur

```
sudo ufw logging on
sudo ufw logging low
sudo ufw deny 23/tcp comment 'Bloquer Telnet - protocole non chiffre'
sudo ufw deny 21/tcp comment 'Bloquer FTP - protocole non chiffre'
sudo ufw deny 135/tcp comment 'Bloquer RPC Windows'
sudo ufw deny 445/tcp comment 'Bloquer SMB'
sudo ufw status numbered
```

```
Firewall reloaded
root@debian:~# ufw logging on
Logging enabled
root@debian:~# ufw logging low
Logging enabled
root@debian:~# ufw deny 23/tcp comment 'Bloquer Telnet - protocole non chiffre'
Rule added
Rule added (v6)
root@debian:~# ufw deny 21/tcp comment 'Bloquer FTP - protocole non chiffre'
ufw deny 135/tcp comment 'Bloquer RPC Windows'
ufw deny 445/tcp comment 'Bloquer SMB'
Rule added
Rule added (v6)
Rule added
Rule added (v6)
Rule added
Rule added (v6)
root@debian:~# ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 2222/tcp                   ALLOW IN    192.168.10.0/24            # SSH interne SenTech
[ 2] 23/tcp                     DENY IN     Anywhere                   # Bloquer Telnet - protocole non chiffre
[ 3] 21/tcp                     DENY IN     Anywhere                   # Bloquer FTP - protocole non chiffre
[ 4] 135/tcp                    DENY IN     Anywhere                   # Bloquer RPC Windows
[ 5] 445/tcp                    DENY IN     Anywhere                   # Bloquer SMB
[ 6] 23/tcp (v6)                DENY IN     Anywhere (v6)              # Bloquer Telnet - protocole non chiffre
[ 7] 21/tcp (v6)                DENY IN     Anywhere (v6)              # Bloquer FTP - protocole non chiffre
[ 8] 135/tcp (v6)               DENY IN     Anywhere (v6)              # Bloquer RPC Windows
[ 9] 445/tcp (v6)               DENY IN     Anywhere (v6)              # Bloquer SMB
```

Figure 31 : *Règles UFW numérotées*

3-2 Règles iptables directes

UFW n'est qu'une façade : nous pouvons observer et compléter les règles réelles avec iptables.

Sur Debian Serveur

```
sudo iptables -L -n -v --line-numbers
sudo iptables -L INPUT -n -v
```

```
root@debian:~# iptables -L -n -v --line-numbers
iptables -L INPUT -n -v
Chain INPUT (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1     7494  837K ufw-before-logging-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0
2     7494  837K ufw-before-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0
3      849  255K ufw-after-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0
4      107 11768 ufw-after-logging-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0
5      107 11768 ufw-reject-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0
6      107 11768 ufw-track-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ufw-before-logging-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0
2        0     0 ufw-before-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0
3        0     0 ufw-after-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0
4        0     0 ufw-after-logging-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0
5        0     0 ufw-reject-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0
6        0     0 ufw-track-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1     5119  477K ufw-before-logging-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0
2     5119  477K ufw-before-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0
3      136 13234 ufw-after-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0
4      136 13234 ufw-after-logging-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0
5      136 13234 ufw-reject-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0
6      136 13234 ufw-track-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain ufw-after-forward (1 references)
num   pkts bytes target     prot opt in     out     source               destination

Chain ufw-after-input (1 references)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ufw-skip-to-policy-input  udp  --  *      *       0.0.0.0/0            0.0.0.0/0           udp dpt:137
2        0     0 ufw-skip-to-policy-input  udp  --  *      *       0.0.0.0/0            0.0.0.0/0           udp dpt:138
3        0     0 ufw-skip-to-policy-input  tcp  --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:139
4        0     0 ufw-skip-to-policy-input  tcp  --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:445
5      239 78392 ufw-skip-to-policy-input  udp  --  *      *       0.0.0.0/0            0.0.0.0/0           udp dpt:67
6        0     0 ufw-skip-to-policy-input  udp  --  *      *       0.0.0.0/0            0.0.0.0/0           udp dpt:68
```

Figure 32 : *Chaîne INPUT vue par iptables*

Limiter à 5 nouvelles connexions SSH par minute depuis une même IP. Le module `recent` garde en mémoire les adresses qui se connectent ; la première règle enregistre chaque nouvelle connexion, la seconde jette celles d'une adresse qui en fait 5 en 60 secondes.

Sur Debian Serveur

```
sudo iptables -A INPUT -p tcp --dport 2222 \
  -m state --state NEW \
  -m recent --set --name SSH_SCAN

sudo iptables -A INPUT -p tcp --dport 2222 \
  -m state --state NEW \
  -m recent --update --seconds 60 --hitcount 5 \
  --name SSH_SCAN -j DROP

sudo iptables -L INPUT -n -v | grep -A2 "SSH_SCAN"
```

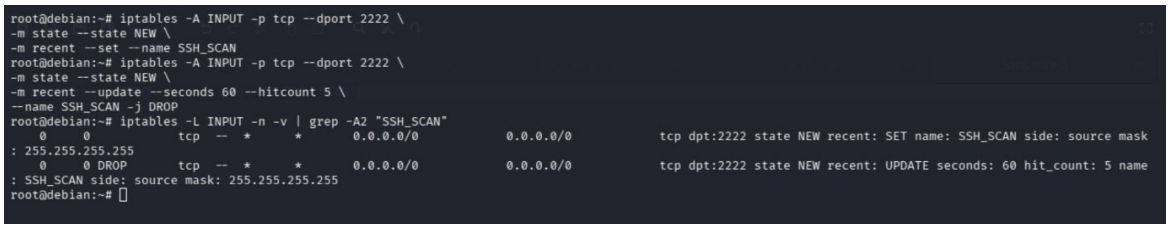


Figure 33 : *Les deux règles SSH_SCAN sont en place*

Rendre les règles persistantes. Les règles iptables sont perdues au redémarrage ; le paquet `iptables-persistent` les restaure au démarrage.

Sur Debian Serveur

```
sudo apt install iptables-persistent -y
# Répondre « Yes » pour sauvegarder les règles IPv4 et IPv6
sudo netfilter-persistent save
cat /etc/iptables/rules.v4
```



Figure 34 : *Installation d'iptables-persistent et sauvegarde des règles*

3-3 Test complet et découverte du conflit UFW / iptables-persistent

Sur Kali Linux

```
nmap -sV -p 22,21,23,80,443,2222,3306,445 192.168.10.10
```



Figure 35 : *Les ports sensibles répondent « closed » : aucun pare-feu global n'est actif*

Deux états de port apparaissent dans Nmap :

- **closed** : le port n'écoute pas, mais la machine répond activement pour dire « rien ici » (paquet RST) ;
- **filtered** : un pare-feu empêche la machine de répondre ; Nmap ne sait donc pas si le port est ouvert ou fermé.

Ici tous les ports sont `closed`, ce qui signifie que le pare-feu n'est plus actif. L'explication : lors de l'installation d'`iptables-persistent`, `apt` a **désinstallé UFW automatiquement** (les deux outils gèrent le même pare-feu et ne cohabitent pas facilement). Résultat :

- UFW est désinstallé ;
- `iptables-persistent` est installé, mais aucune règle générale n'a été sauvegardée ;
- il ne reste que les deux règles iptables ajoutées à la main (la protection anti-force-brute SSH via `recent`).

Aucun pare-feu global ne protégeait donc plus la machine : les ports fermés répondaient avec un RST, ce qui donne `closed` dans Nmap.

3-4 Correction : reconstruction du pare-feu avec iptables

Puisque nous avons choisi iptables-persistent, il faut construire les règles à la main. Voici une base saine, dans cet ordre :

1. **Autoriser le loopback** (la machine doit pouvoir se parler à elle-même) ;
2. **Autoriser les connexions déjà établies** (très important : sans cela, les réponses aux connexions que le serveur initie lui-même seraient bloquées) ;
3. **Autoriser SSH uniquement depuis le LAN** ;
4. **Bloquer les pings** (ICMP echo-request) ;
5. **Politique par défaut : tout bloquer en entrée** ;
6. **Sauvegarder** pour que les règles persistent au redémarrage ;
7. **Vérifier**.

Sur Debian Serveur (en root)

```
# 1. Autoriser le loopback
iptables -A INPUT -i lo -j ACCEPT
# 2. Autoriser les connexions établies
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
# 3. Autoriser SSH uniquement depuis le LAN
iptables -A INPUT -p tcp --dport 2222 -s 192.168.10.0/24 -j ACCEPT
# 4. Bloquer les pings (ICMP echo-request)
iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
# 5. Politique par défaut : tout bloquer en entrée
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT
# 6. Sauvegarder les règles pour qu'elles persistent au redémarrage
netfilter-persistent save
# 7. Vérifier
iptables -L -n -v
```

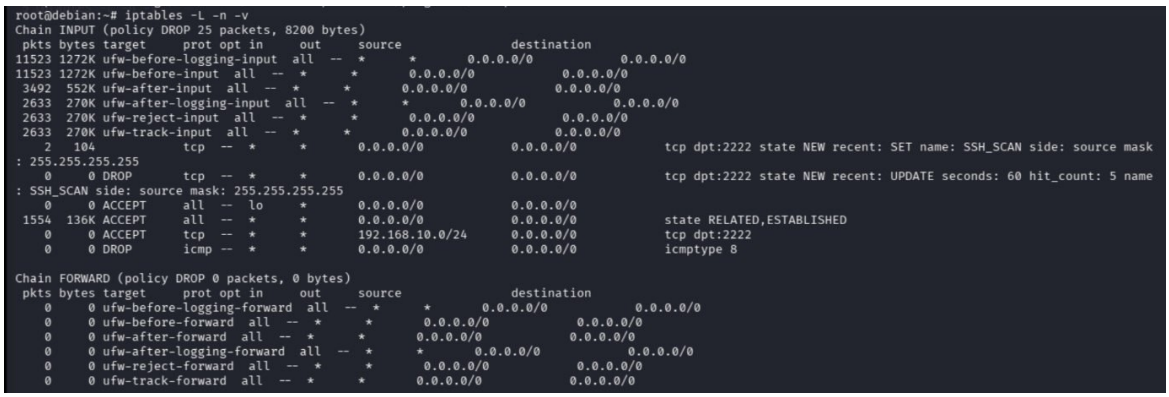


Figure 36 : *Règles finales : politique DROP en entrée, SSH autorisé depuis 192.168.10.0/24*

> **ATTENTION** **ne jamais passer la politique en DROP avant d'avoir autorisé SSH**
>
> Les étapes 1 à 3 doivent précéder l'étape 5. Si la politique `DROP` est appliquée avant la règle qui autorise SSH, la session en cours est coupée. En cas d'erreur, la console de virt-manager permet de remettre la politique à `ACCEPT` avec `iptables -P INPUT ACCEPT`.

> **REMARQUE** **chaînes « ufw-* »**
>
> Dans la capture, on voit encore des chaînes nommées `ufw-before-input`, `ufw-after-input`, etc. Ce sont des restes de l'ancienne installation d'UFW, conservés dans la configuration sauvegardée. Elles sont inoffensives mais inutiles : une configuration de production les supprimerait pour garder un jeu de règles lisible.

Nouveau scan Nmap :

Sur Kali Linux

```
nmap -sV -p 22,21,23,80,443,2222,3306,445 192.168.10.10
```

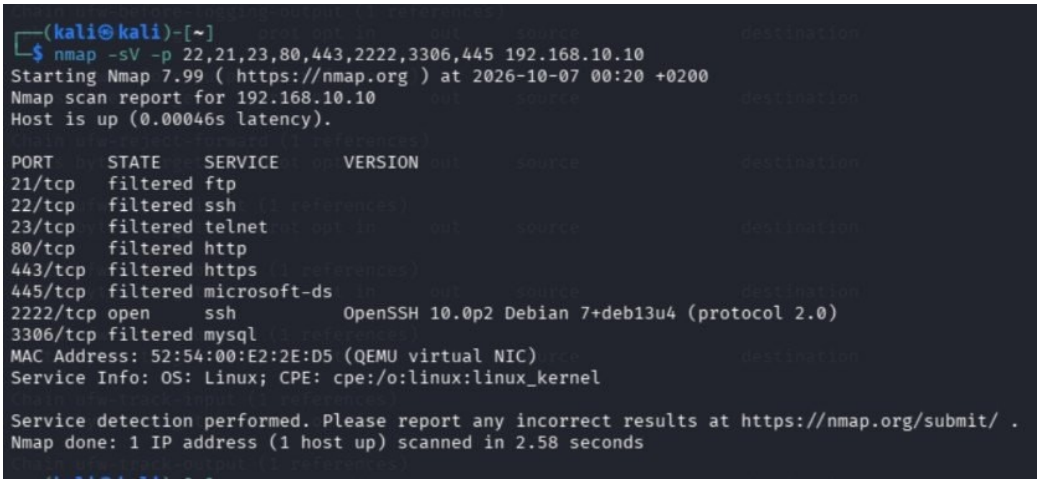


Figure 37 : *Tous les ports sont désormais « filtered » sauf le port SSH 2222, ouvert*

Partie 4 : Déploiement de Fail2Ban

Sur Debian Serveur

```
sudo apt install fail2ban -y
sudo systemctl status fail2ban
```

Fail2Ban utilise deux types de fichiers : `jail.conf` (la configuration d'origine, **à ne jamais modifier directement**, car une mise à jour du paquet l'écraserait) et `jail.local` (fichier de surcharge locale,

préservé lors des mises à jour). Il suffit que `jail.local` contienne **uniquement les paramètres que l'on change** : Fail2Ban lit d'abord `jail.conf`, puis applique `jail.local` par-dessus.

Sur Debian Serveur
```
sudo touch /etc/fail2ban/jail.local
ls -la /etc/fail2ban/jail.local
```

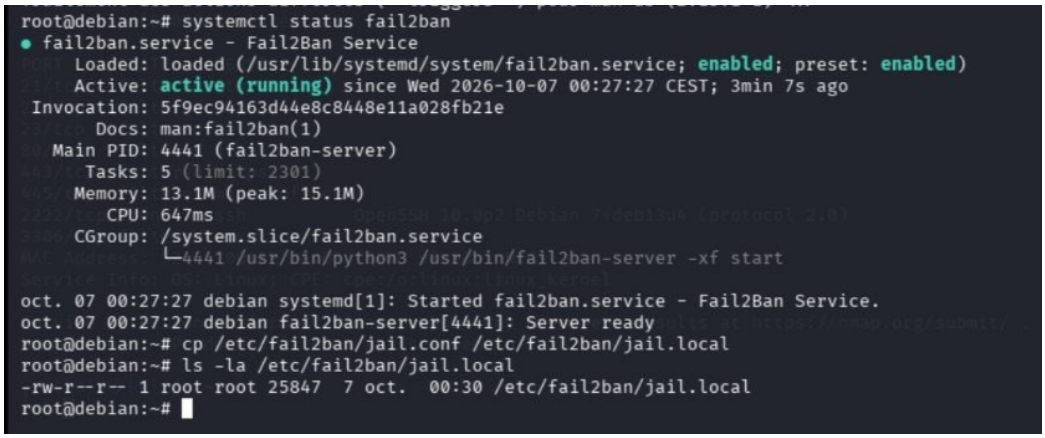


Figure 38 : *Fichier jail.local dans /etc/fail2ban/*

1. Édition du fichier.

Sur Debian Serveur
```
sudo nano /etc/fail2ban/jail.local
```

/etc/fail2ban/jail.local
```
[DEFAULT]
# IP jamais bannies (le réseau de gestion : OPNsense, passerelle du LAN)
ignoreip = 127.0.0.1/8 ::1 192.168.10.1
# Durée de bannissement : 1 heure (3600 secondes). -1 pour un bannissement permanent.
bantime = 3600
# Fenêtre de temps pour compter les échecs : 10 minutes
findtime = 600
# Nombre d'échecs avant bannissement
maxretry = 3
# Backend de détection des logs
backend = auto

[sshd]
enabled  = true
port     = 2222
filter   = sshd
logpath  = %(sshd_log)s
backend  = systemd
maxretry = 3
bantime  = 3600
findtime = 600
banaction = iptables-multiport
```

POUR COMPRENDRE **Règles impératives pour ce fichier**

- **Une seule section `[sshd]`** dans tout le fichier : une section en double provoque une erreur.
- **Une seule occurrence par option** (`port`, `logpath`, `enabled`, `backend`, etc.).
- **Aucun caractère invisible** : tapez les lignes à la main au lieu de les copier-coller depuis un document. Lors de notre test, un espace insécable collé avec le nom de la jail a produit l'erreur « Sorry but the jail 'sshd ' does not exist ».
- Le port doit correspondre au port SSH réel (ici 2222).
- Sur Debian 13, les journaux SSH sont dans le journal systemd (`journalctl`) et non dans un fichier `auth.log` : c'est pourquoi le backend est `systemd`.

2. Test de la configuration. Avant de redémarrer le service, testez toujours la syntaxe :

Sur Debian Serveur
```
sudo fail2ban-client -t
```

3. Activation et démarrage :

Sur Debian Serveur
```
sudo systemctl restart fail2ban
sudo systemctl enable fail2ban
```

4. Vérification que le service tourne, puis que la jail SSH est active :

Sur Debian Serveur
```
sudo systemctl status fail2ban
sudo fail2ban-client status sshd
```

Simulation d'une attaque par force brute

Depuis Kali, nous essayons de nous connecter avec plusieurs faux comptes. Le serveur n'accepte que les clés, donc chaque tentative échoue ; mais Fail2Ban compte chaque échec.

Sur Kali Linux
```
ssh -p 2222 -o PreferredAuthentications=password -o PubkeyAuthentication=no
fakeuser@192.168.10.10
ssh -p 2222 -o PreferredAuthentications=password -o PubkeyAuthentication=no
admin@192.168.10.10
ssh -p 2222 -o PreferredAuthentications=password -o PubkeyAuthentication=no
root@192.168.10.10
ssh -p 2222 -o PreferredAuthentications=password -o PubkeyAuthentication=no
ubuntu@192.168.10.10
```

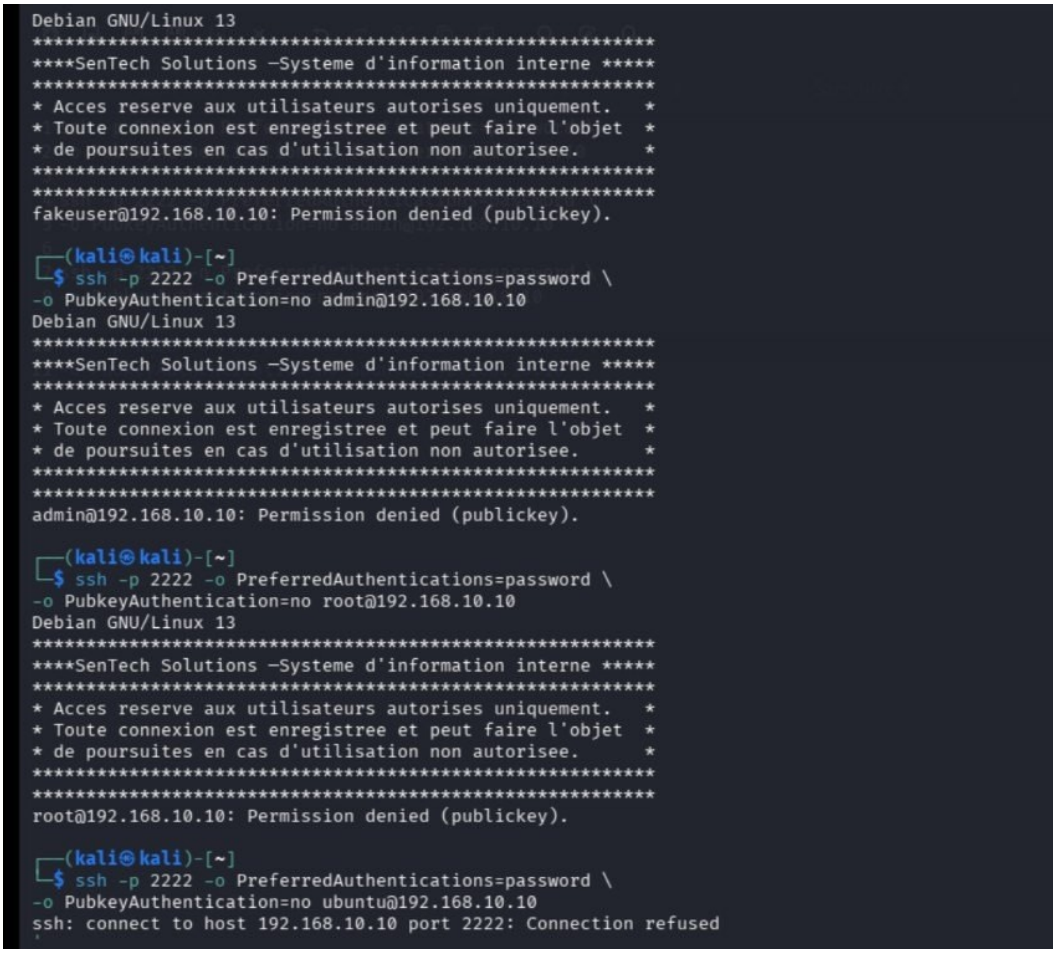


Figure 39 : *Trois refus « Permission denied (publickey) », puis « Connection refused » à la quatrième tentative*

Après **3 tentatives échouées**, Kali (192.168.10.99) est banni : la quatrième tentative n'obtient même plus de réponse d'authentification, la connexion est refusée.

Observation du bannissement :

Sur Debian Serveur

```
sudo fail2ban-client status sshd
sudo iptables -L f2b-sshd -n -v
sudo tail -30 /var/log/fail2ban.log
```

```
    `- Banned IP list:
root@debian:~# fail2ban-client status sshd
2026-10-07 10:56:54,461 fail2ban              [4978]: ERROR   NOK: ('sshd\xa0',)
Sorry but the jail 'sshd ' does not exist
root@debian:~# iptables -L f2b-sshd -n -v
Chain f2b-sshd (1 references)
 pkts bytes target     prot opt in     out     source               destination
    1    60 REJECT     all  --  *      *       192.168.10.99        0.0.0.0/0            reject-with icmp-port-unreachable
    0     0 RETURN     all  --  *      *       0.0.0.0/0            0.0.0.0/0
root@debian:~# tail -30 /var/log/fail2ban.log
2026-10-07 00:27:27,768 fail2ban.filtersystemd  [4441]: INFO    [sshd] Jail is in operation now (process new journal entries)
2026-10-07 00:27:27,773 fail2ban.jail           [4441]: INFO    Jail 'sshd' started
2026-10-07 00:47:15,039 fail2ban.server         [4441]: INFO    Shutdown in progress...
2026-10-07 00:47:15,040 fail2ban.observer       [4441]: INFO    Observer stop ... try to end queue 5 seconds
2026-10-07 00:47:15,060 fail2ban.observer       [4441]: INFO    Observer stopped, 0 events remaining.
2026-10-07 00:47:15,102 fail2ban.server         [4441]: INFO    Stopping all jails
2026-10-07 00:47:15,255 fail2ban.actions        [4441]: NOTICE  [sshd] Flush ticket(s) with nftables
2026-10-07 00:47:15,255 fail2ban.jail           [4441]: INFO    Jail 'sshd' stopped
2026-10-07 00:47:15,257 fail2ban.database       [4441]: INFO    Connection to database closed.
2026-10-07 00:47:15,258 fail2ban.server         [4441]: INFO    Exiting Fail2ban
2026-10-07 10:35:38,488 fail2ban.server         [4947]: INFO    ──────────────────────────────────────────────────
2026-10-07 10:35:38,488 fail2ban.server         [4947]: INFO    Starting Fail2ban v1.1.0
2026-10-07 10:35:38,489 fail2ban.observer       [4947]: INFO    Observer start...
2026-10-07 10:35:38,497 fail2ban.database       [4947]: INFO    Connected to fail2ban persistent database '/var/lib/fail2ban/fail2ban.sqlite3'
2026-10-07 10:35:38,498 fail2ban.jail           [4947]: INFO    Creating new jail 'sshd'
2026-10-07 10:35:38,505 fail2ban.jail           [4947]: INFO    Jail 'sshd' uses systemd {}
2026-10-07 10:35:38,506 fail2ban.jail           [4947]: INFO    Initiated 'systemd' backend
2026-10-07 10:35:38,507 fail2ban.filter         [4947]: INFO      maxLines: 1
2026-10-07 10:35:38,530 fail2ban.filtersystemd  [4947]: INFO    [sshd] Added journal match for: '_SYSTEMD_UNIT=ssh.service + _COMM=sshd'
2026-10-07 10:35:38,530 fail2ban.filter         [4947]: INFO      maxRetry: 3
2026-10-07 10:35:38,530 fail2ban.filter         [4947]: INFO      findtime: 600
2026-10-07 10:35:38,531 fail2ban.actions        [4947]: INFO      banTime: 3600
2026-10-07 10:35:38,531 fail2ban.filter         [4947]: INFO      encoding: UTF-8
2026-10-07 10:35:38,532 fail2ban.filtersystemd  [4947]: INFO    [sshd] Jail is in operation now (process new journal entries)
2026-10-07 10:35:38,533 fail2ban.jail           [4947]: INFO    Jail 'sshd' started
2026-10-07 10:52:39,759 fail2ban.filter         [4947]: INFO    [sshd] Found 192.168.10.99 - 2026-10-07 10:52:38
2026-10-07 10:52:58,574 fail2ban.filter         [4947]: INFO    [sshd] Found 192.168.10.99 - 2026-10-07 10:52:58
2026-10-07 10:53:08,574 fail2ban.filter         [4947]: INFO    [sshd] Found 192.168.10.99 - 2026-10-07 10:53:08
2026-10-07 10:53:08,628 fail2ban.actions        [4947]: NOTICE  [sshd] Ban 192.168.10.99
2026-10-07 10:56:54,460 fail2ban.transmitter    [4947]: ERROR   Command ['status', 'sshd\xa0'] has failed. Received UnknownJailException('sshd\xa0
')
```

Figure 40 : *Règle REJECT créée par Fail2Ban pour 192.168.10.99 et journal de l'événement (la première commande a échoué à cause d'un espace insécable, voir l'encadré ci-dessus)*

Le journal montre les trois tentatives détectées (« Found », à 10:52:38, 10:52:58 et 10:53:08) puis le bannissement (« Ban 192.168.10.99 »). La chaîne `f2b-sshd` contient bien la règle `REJECT` qui bloque cette adresse.

Vérification depuis Kali :

```
┌──(kali㉿kali)-[~]
└─$ ssh -p 2222 debian@192.168.10.10
ssh: connect to host 192.168.10.10 port 2222: Connection refused

┌──(kali㉿kali)-[~]
└─$
```

Figure 41 : *La connexion est refusée depuis l'adresse bannie*

Débannissement manuel (par exemple après un test) :

Sur Debian Serveur

```
sudo fail2ban-client set sshd unbanip 192.168.10.99
sudo fail2ban-client status sshd
```

```
root@debian:~# fail2ban-client set sshd unbanip 192.168.10.99
1
root@debian:~# fail2ban-client status sshd
Status for the jail: sshd
 ├ Filter
 |  ├ Currently failed: 0
 |  ├ Total failed:     3
 |  `- Journal matches:  _SYSTEMD_UNIT=ssh.service + _COMM=sshd
 `- Actions
    ├ Currently banned: 0
    ├ Total banned:     1
    `- Banned IP list:
root@debian:~#
```

Figure 42 : *Adresse débannie : la liste des IP bannies est de nouveau vide*

Jail personnalisée pour les échecs su et sudo

Au-delà de SSH, nous créons une seconde jail qui surveille les échecs d'authentification des commandes `su` et `sudo`. Elle a besoin d'un **filtre**, c'est-à-dire d'une expression régulière qui reconnaît la ligne de journal à surveiller.

Sur Debian Serveur
```
sudo nano /etc/fail2ban/jail.local
```

Ajouter à la fin du fichier :

/etc/fail2ban/jail.local
```
[sentech-custom]
enabled  = true
filter   = sentech-custom
backend  = systemd
maxretry = 5
bantime  = 1800
findtime = 300
port     = all
```

Créer le filtre :

Sur Debian Serveur
```
sudo nano /etc/fail2ban/filter.d/sentech-custom.conf
```

/etc/fail2ban/filter.d/sentech-custom.conf
```
[Definition]
failregex = pam_unix\(su.*\): authentication failure.*rhost=<HOST>
            pam_unix\(sudo.*\): authentication failure.*rhost=<HOST>
ignoreregex =
```

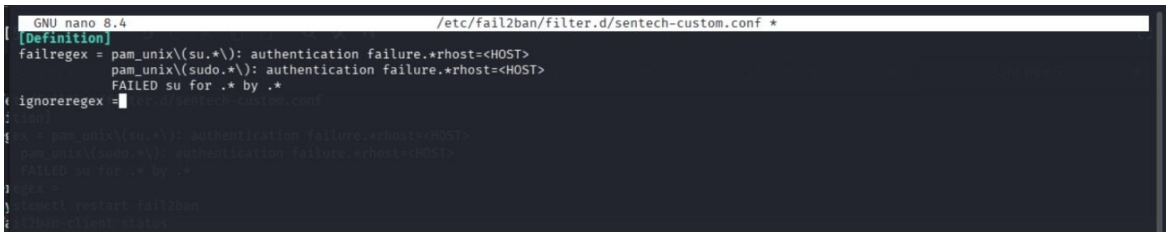


Figure 43 : *Filtre sentech-custom.conf*

POUR COMPRENDRE **le marqueur <HOST>**

Fail2Ban a besoin de savoir **quelle adresse bannir** : la balise `<HOST>` indique à quel endroit de la ligne de journal se trouve l'adresse IP. Une règle sans `<HOST>` détecte l'événement mais ne permet pas de bannir quelqu'un ; nous l'avons donc retirée du filtre. Les barres obliques inverses devant les parenthèses sont nécessaires car, dans une expression régulière, la parenthèse a un sens spécial.

Limite à connaître : ces deux règles ne se déclenchent que lorsque la ligne de journal contient une adresse distante (`rhost=`). Un échec de `sudo` tapé directement sur la console n'en contient pas et n'est donc pas banni ; il reste en revanche enregistré par Auditd (partie suivante).

Sur Debian Serveur
```
sudo systemctl restart fail2ban
sleep 3
sudo fail2ban-client status
sudo fail2ban-client status sentech-custom
```

Partie 5 : Configuration d'Auditd

Auditd observe le noyau et écrit un événement chaque fois qu'une règle est déclenchée. Nous l'installons puis lui donnons nos règles.

Sur Debian Serveur
```
sudo apt install auditd audispd-plugins -y
sudo systemctl enable --now auditd
sudo systemctl status auditd
```

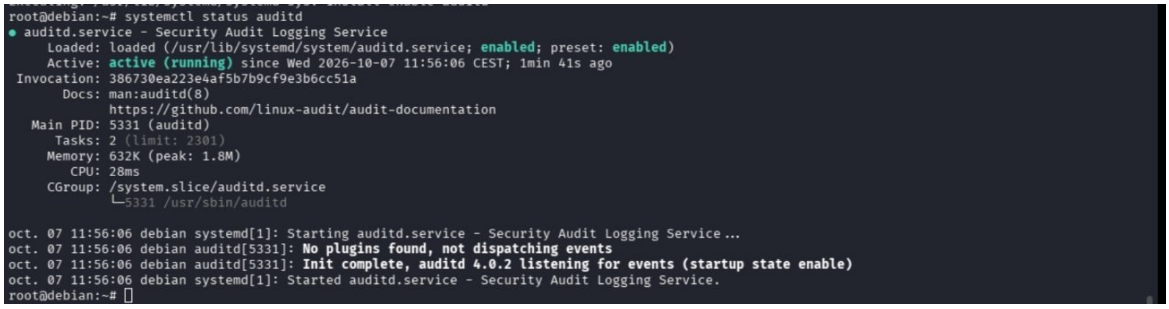


Figure 44 : *Service auditd actif*

Une règle Auditd a l'une de ces deux formes :

- `-w /chemin -p permissions -k nom_de_cle` : surveillance d'un fichier ou d'un dossier (`w` écriture, `r` lecture, `x` exécution, `a` changement d'attributs) ;
- `-a always,exit -F arch=b64 -S syscall -F filtre -k cle` : surveillance d'un appel système (une demande faite par un programme au noyau).

Le dernier élément, `-k`, est une **clé** (une étiquette) qui permet de retrouver facilement les événements avec `ausearch -k`.

Sur Debian Serveur
```
sudo nano /etc/audit/rules.d/sentech.rules
```

/etc/audit/rules.d/sentech.rules
```
## ===== Surveillance des fichiers d'identité =====
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/group -p wa -k identity
-w /etc/gshadow -p wa -k identity

## ===== Surveillance de la configuration sudo =====
-w /etc/sudoers -p wa -k sudoers
-w /etc/sudoers.d/ -p wa -k sudoers

## ===== Surveillance de la configuration SSH =====
-w /etc/ssh/sshd_config -p wa -k sshd_config

## ===== Surveillance des commandes privilégiées =====
-a always,exit -F arch=b64 -S execve -F euid=0 -k root_commands
-a always,exit -F arch=b32 -S execve -F euid=0 -k root_commands

## ===== Surveillance des changements de permissions =====
-a always,exit -F arch=b64 -S chmod -S fchmod -S fchmodat -S chown -S fchown -S fchownat -k
perm_change
-a always,exit -F arch=b32 -S chmod -S fchmod -S fchmodat -S chown -S fchown -S fchownat -k
perm_change

## ===== Surveillance des accès refusés =====
-a always,exit -F arch=b64 -S open -S openat -F exit=-EACCES -k access_denied
-a always,exit -F arch=b64 -S open -S openat -F exit=-EPERM -k access_denied

## ===== Surveillance des montages =====
-a always,exit -F arch=b64 -S mount -k mount_ops

## ===== Surveillance des répertoires sensibles =====
```

```
-w /srv/sentech/finance/ -p rwxa -k finance_access
-w /srv/sentech/backup/ -p rwxa -k backup_access

## ===== Surveillance des modifications du pare-feu =====
-w /etc/iptables/ -p wa -k firewall_config
```

POUR COMPRENDRE **Ce qui a été ajouté ou corrigé dans ces règles**

- Les lignes de commentaire commencent par ## : sans le #, Auditd refuse le fichier.
- **`openat`** a été ajouté à côté de `open`. Les programmes modernes ouvrent leurs fichiers avec `openat` ; sans lui, la clé `access_denied` aurait manqué la plupart des refus d'accès.
- Le dossier `/etc/ufw/` n'est plus surveillé, car UFW a été désinstallé (voir la Partie 3) ; seul `/etc/iptables/` contient désormais les règles du pare-feu.

REMARQUE **Point d'attention : règles arch=b32 sur un système 64 bits pur**

Les règles en `b64` et `b32` couvrent les appels système 64 bits et 32 bits, au cas où un programme 32 bits serait exécuté sur ce serveur. Sur certains noyaux compilés sans support de la compatibilité 32 bits, `augenrules --load` peut afficher un avertissement du type « Error sending add rule » pour les lignes `b32`. Ce n'est pas bloquant : les règles `b64` (les plus importantes, car Debian 13 amd64 est un système 64 bits) sont chargées normalement. Si vous voyez cet avertissement, vous pouvez supprimer sans risque les lignes `-F arch=b32` du fichier et relancer `augenrules --load`.

Sur Debian Serveur

```
sudo augenrules --load
sudo auditctl -l
sudo auditctl -l | wc -l
```

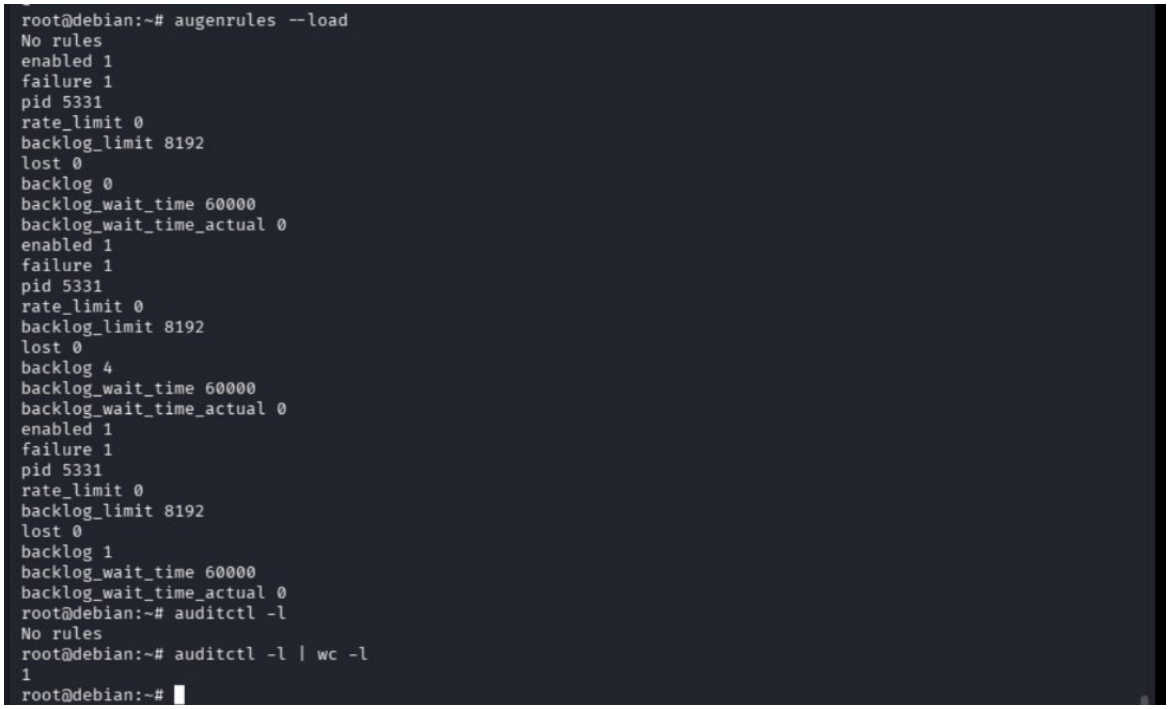


Figure 45 : *Règles Auditd chargées*

Test des règles Auditd

Test 1 : surveillance de /etc/passwd. `touch` modifie la date du fichier : c'est un « changement d'attribut » qui doit être enregistré.

Sur Debian Serveur

```
sudo touch /etc/passwd
sleep 2
```

```
sudo ausearch -k identity --start today
```

Test 2 : surveillance des commandes root.

```
Sur Debian Serveur
sudo ls /root
sudo cat /etc/shadow | head -3
sudo ausearch -k root_commands --start today | head -40
```

Test 3 : surveillance des accès au dossier finance, puis des accès refusés. Alice reçoit un droit de lecture sur le dossier finance, puis tente aussi d'entrer dans le dossier backup, qui lui est interdit : cette tentative doit produire un événement `access_denied`.

```
Sur Debian Serveur
sudo setfacl -m u:alice:rx /srv/sentech/finance
su - alice
ls /srv/sentech/finance/
ls /srv/sentech/backup/
exit
sudo ausearch -k finance_access --start today
sudo ausearch -k access_denied --start today
```

Utilisez les outils d'analyse intégrés :

```
Sur Debian Serveur
sudo aureport --auth --start today
sudo aureport -x --start today --summary
sudo aureport -f --start today
sudo ausearch -ua 1001 --start today
```

POUR COMPRENDRE **ausearch et aureport**

`ausearch` retrouve des événements précis (par clé, par utilisateur…). `aureport` produit des **synthèses** : authentifications, exécutables les plus lancés, fichiers les plus touchés. Dans la dernière commande, `1001` est l'identifiant (UID) d'alice, le premier compte créé après `debian` (1000).

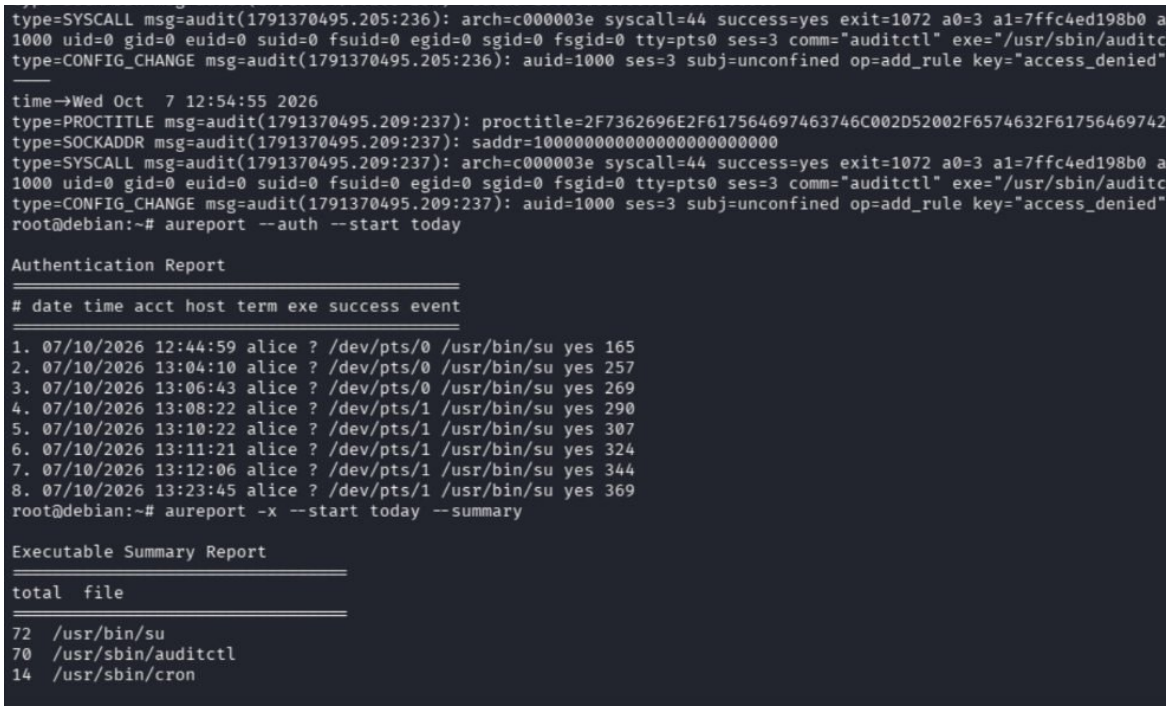


Figure 46 : *Résultat des recherches ausearch / aureport*

Partie 6 : Durcissement de PAM et sudo

PAM (Pluggable Authentication Modules) est le mécanisme que Linux utilise pour vérifier un mot de passe, quel que soit le programme (connexion console, `su`, `sudo`…). En le configurant, on impose les mêmes règles partout.

Politique de mots de passe

Sur Debian Serveur
```
sudo apt install libpam-pwquality -y
sudo nano /etc/security/pwquality.conf
```

Remplacez le contenu par :

/etc/security/pwquality.conf
```
# Configuration de la qualité des mots de passe - SenTech Solutions
minlen = 12
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = -1
maxrepeat = 3
maxclassrepeat = 4
difok = 5
gecoscheck = 1
```

Paramètre	Effet
minlen = 12	Longueur minimale de 12 caractères
dcredit, ucredit, lcredit, ocredit = -1	Au moins un chiffre, une majuscule, une minuscule et un caractère spécial (la valeur négative signifie « obligatoire »)
maxrepeat = 3	Pas plus de 3 fois le même caractère de suite
maxclassrepeat = 4	Pas plus de 4 caractères consécutifs de la même catégorie
difok = 5	Au moins 5 caractères différents de l'ancien mot de passe
gecoscheck = 1	Interdit d'utiliser le nom complet de l'utilisateur dans son mot de passe

Vérifions que PAM appelle bien ce module :

Sur Debian Serveur
```
grep pwquality /etc/pam.d/common-password
```

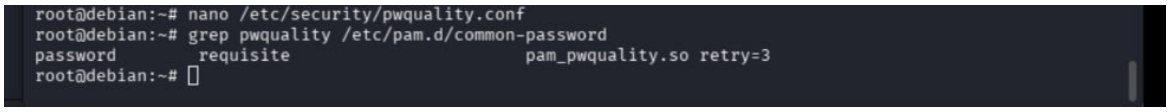


Figure 47 : *La ligne pam_pwquality.so est présente dans common-password*

Pour le tester, on change un mot de passe en tant qu'utilisateur ordinaire (`su - alice` puis `passwd`) avec une valeur trop simple comme `azerty` : le système doit la refuser en indiquant la règle violée.

Verrouillage après échecs répétés (pam_faillock)

Objectif : après 5 mots de passe faux, le compte est bloqué pendant 10 minutes (600 secondes), ce qui ruine toute attaque par force brute sur les mots de passe. On sauvegarde d'abord le fichier, puis on le modifie.

Sur Debian Serveur (en root)
```
cp /etc/pam.d/common-auth /etc/pam.d/common-auth.bak
nano /etc/pam.d/common-auth
```

Remplacez son contenu par (**une règle par ligne**) :

/etc/pam.d/common-auth

```
# Verrouillage après 5 tentatives échouées pendant 10 minutes - SenTech Solutions
auth    required                        pam_faillock.so preauth silent deny=5 unlock_time=600
auth    [success=1 default=ignore]      pam_unix.so nullok
auth    [default=die]                   pam_faillock.so authfail deny=5 unlock_time=600
auth    sufficient                      pam_faillock.so authsucc
auth    requisite                       pam_deny.so
auth    required                        pam_permit.so
```

Pour que le compteur soit aussi pris en compte à l'ouverture de session, on ajoute dans `/etc/pam.d/common-account` la ligne :

/etc/pam.d/common-account

```
account required pam_faillock.so
```

ATTENTION **ne pas couper la branche sur laquelle on est assis**

Sauvegardez le fichier, puis **vérifiez qu'une session administrateur reste ouverte** pendant tout le test. Une erreur dans les fichiers PAM peut empêcher toute connexion : la copie `common-auth.bak` et la console de virt-manager permettent de revenir en arrière.

Premier test : l'accès SSH par mot de passe reste impossible. Depuis un nouveau terminal de Kali (sans fermer la session principale) :

Sur Kali Linux

```
for i in 1 2 3 4 5 6; do
  ssh -p 2222 -o PreferredAuthentications=password -o PubkeyAuthentication=no \
      debian@192.168.10.10 exit
done
```

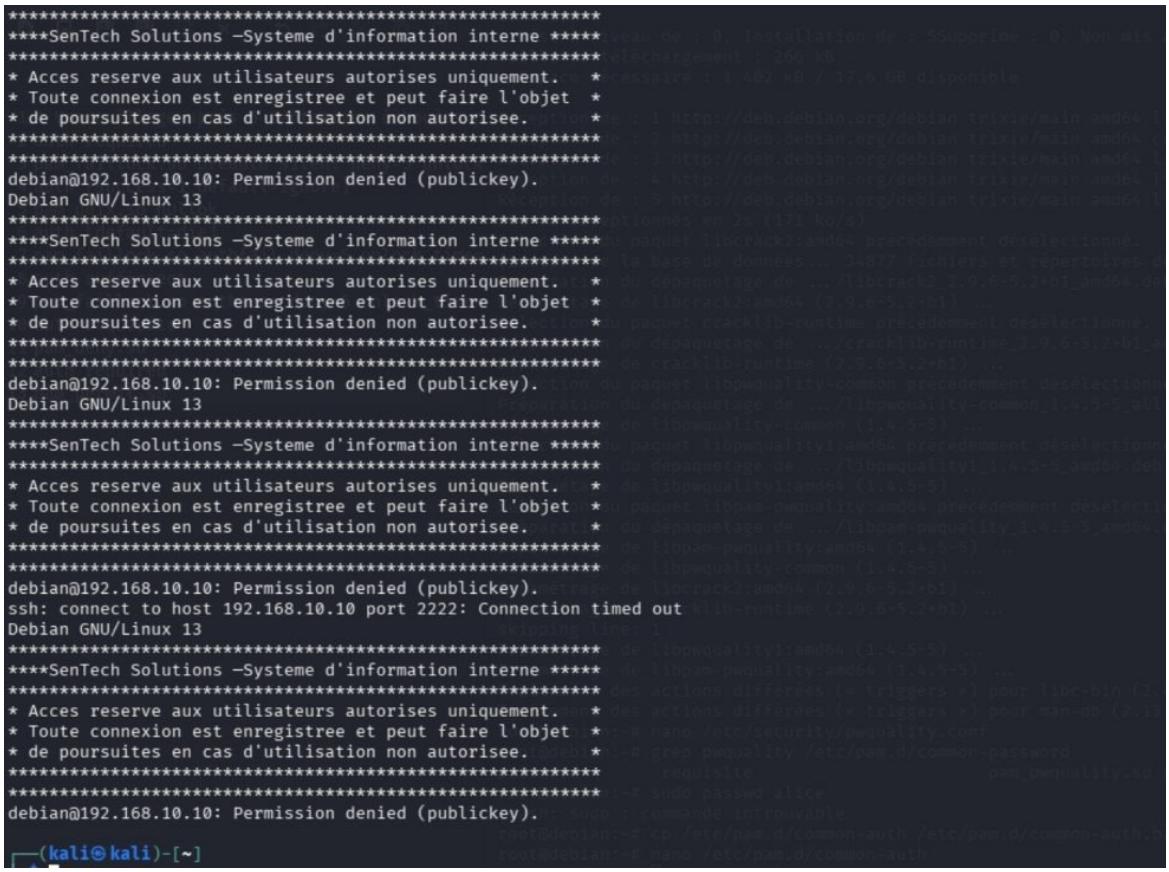


Figure 48 : *Les tentatives par mot de passe sont refusées (« Permission denied (publickey) ») ; l'une d'elles expire (« Connection timed out »), effet de la limitation de connexions du pare-feu ou de Fail2Ban*

Ce résultat est **normal** : comme `PasswordAuthentication no` est actif dans SSH, le serveur refuse l'authentification par mot de passe avant même d'interroger PAM. Aucun échec n'est donc compté par `pam_faillock`, et la commande de consultation affiche un tableau vide :

Sur Debian Serveur
```
sudo faillock --user debian
```



Figure 49 : *Compteur faillock vide : aucun mot de passe n'a atteint PAM par SSH*

Deuxième test : le verrouillage, depuis la console. Les commandes `su` et `login` passent, elles, par PAM. Depuis la console de virt-manager, connecté en `debian`, on saisit volontairement un mauvais mot de passe 5 fois :

Sur Debian Serveur (console virt-manager)
```
su - alice        # saisir un mauvais mot de passe, 5 fois de suite
sudo faillock --user alice
```

Après 5 échecs, le compte d'alice est verrouillé pendant 600 secondes et `faillock` liste les tentatives. Pour déverrouiller manuellement avant l'expiration du délai :

Sur Debian Serveur
```
sudo faillock --user alice --reset
```

> ATTENTION **Risque de verrouillage de votre propre compte**
> Testez cette fonctionnalité avec un compte que vous pouvez vous permettre de bloquer temporairement (comme `alice`), ou utilisez `sudo faillock --user <compte> --reset` immédiatement après le test. N'expérimentez jamais cette commande sur votre seule session active sans avoir un second accès de secours (par exemple la console graphique de la VM dans virt-manager, qui reste accessible même si SSH est verrouillé).

Durcissement de sudo

Nous créons d'abord le fichier de journal avec des droits stricts (lecture et écriture pour root seulement) :

Sur Debian Serveur
```
sudo touch /var/log/sudo.log
sudo chmod 600 /var/log/sudo.log
sudo chown root:root /var/log/sudo.log
```

Le fichier de configuration de sudo ne se modifie **jamais** avec un éditeur ordinaire, mais avec `visudo`, qui vérifie la syntaxe avant d'enregistrer (une erreur dans ce fichier peut retirer tout accès administrateur) :

Sur Debian Serveur
```
sudo visudo
```

/etc/sudoers (ajouts avec visudo)
```
# === Configuration globale sudo - SenTech Solutions ===
Defaults timestamp_timeout=5
Defaults logfile="/var/log/sudo.log"
Defaults log_input, log_output
Defaults badpass_message="Mot de passe incorrect - Cet evenement est enregistre"
Defaults env_reset
```

```
Defaults secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
Defaults !visiblepw

# Permettre à charlie (administrateur système) d'utiliser sudo
charlie ALL=(ALL:ALL) ALL
```

Directive	Rôle
timestamp_timeout=5	Le mot de passe est redemandé après 5 minutes sans utiliser sudo
logfile="/var/log/sudo.log"	Écrit les journaux sudo dans ce fichier
log_input, log_output	Enregistre les commandes tapées et leur sortie (très verbeux)
badpass_message	Message affiché en cas de mauvais mot de passe
env_reset	Nettoie les variables d'environnement avant d'exécuter la commande
secure_path	Impose une liste de dossiers sûrs pour trouver les programmes
!visiblepw	Refuse de demander un mot de passe si celui-ci serait visible à l'écran

ATTENTION **log_input et log_output**

Ces deux options enregistrent tout ce qui est tapé et affiché dans les commandes lancées par sudo, **y compris d'éventuels mots de passe saisis dans ces programmes**. Le fichier de journal doit donc rester en `600` et ne servir qu'à l'investigation.

Vérification de la syntaxe, puis test de journalisation :

Sur Debian Serveur
```
sudo visudo -c
sudo ls /root
sudo cat /var/log/sudo.log
```



Figure 50 : *Syntaxe valide ; la commande sudo ls /root est journalisée*

Verrouillage du compte root

Sur Debian Serveur
```
sudo passwd -S root
sudo passwd -l root
sudo passwd -S root
```

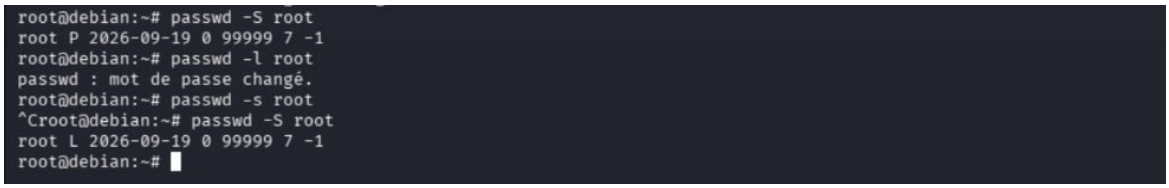


Figure 51 : *État du compte root avant et après verrouillage (L = verrouillé)*

Verrouiller root n'empêche pas d'utiliser sudo : on peut toujours faire `sudo su -` si nécessaire (sous réserve d'avoir les droits sudo). On supprime ainsi la possibilité de se connecter directement en root avec un mot de passe, tout en gardant l'administration par sudo, qui est tracée.

Partie 7 : Hardening système complémentaire (sysctl)

`sysctl` règle des paramètres du noyau Linux. Nous les regroupons dans un fichier dédié, chargé à chaque démarrage.

Sur Debian Serveur
```
sudo nano /etc/sysctl.d/99-sentech-security.conf
```

/etc/sysctl.d/99-sentech-security.conf
```
# === Durcissement noyau - SenTech Solutions ===

# === PROTECTION RÉSEAU ===
net.ipv4.ip_forward = 0
net.ipv6.conf.all.forwarding = 0
net.ipv4.conf.default.rp_filter = 1
net.ipv4.conf.all.rp_filter = 1
net.ipv4.tcp_syncookies = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.default.accept_redirects = 0
net.ipv6.conf.all.accept_redirects = 0
net.ipv4.conf.all.send_redirects = 0
net.ipv4.icmp_echo_ignore_broadcasts = 1

# === PROTECTION MÉMOIRE ===
kernel.randomize_va_space = 2
fs.suid_dumpable = 0

# === PROTECTION SYSTÈME ===
kernel.dmesg_restrict = 1
kernel.yama.ptrace_scope = 1
```

Paramètre	Effet
ip_forward = 0	La machine ne fait pas office de routeur
rp_filter = 1	Rejette les paquets dont l'adresse source est impossible (usurpation)
tcp_syncookies = 1	Protège contre l'inondation de demandes de connexion (SYN flood)
accept_redirects = 0, send_redirects = 0	Ignore et n'envoie pas de messages qui changeraient les routes
icmp_echo_ignore_broadcasts = 1	Ne répond pas aux pings envoyés à tout le réseau (attaque « smurf »)
randomize_va_space = 2	ASLR : positions mémoire aléatoires, ce qui complique les exploits
suid_dumpable = 0	Pas de fichier d'image mémoire pour les programmes privilégiés
dmesg_restrict = 1	Les messages du noyau sont réservés à root
yama.ptrace_scope = 1	Un processus ne peut espionner que ses propres enfants

Les lignes qui commencent par `#` sont des commentaires : **sans ce `#`, `sysctl` refuse le fichier**.
Chargement et vérification :

Sur Debian Serveur
```
sudo sysctl --system
sudo sysctl net.ipv4.ip_forward
sudo sysctl kernel.randomize_va_space
sudo sysctl net.ipv4.tcp_syncookies
```

Bannière de sécurité complète

La bannière affichée avant la connexion est complétée par un avertissement juridique plus formel (elle remplace le contenu précédent de `/etc/issue.net`).

Sur Debian Serveur
```
sudo nano /etc/issue.net
```

/etc/issue.net

```
**********************************************************************
*********  SenTech Solutions - Infrastructure Systeme  ************
**********************************************************************
 * ACCES AUTORISE AUX UTILISATEURS HABILITES UNIQUEMENT
 * Toute connexion est enregistree et tracee.
 * L'utilisation non autorisee de ce systeme est passible de
   poursuites judiciaires conformement a la legislation en vigueur.
 * En cas de probleme : admin@sentech.sn
**********************************************************************
```

Partie 8 : Vérification finale et comparaison

Pour mesurer l'effet du durcissement, nous refaisons exactement le même scan Nmap qu'au début, puis comparons les deux résultats avec `diff`.

Sur Kali Linux

```
nmap -sV 192.168.10.10 -oN /tmp/scan_apres_hardening.txt
diff /tmp/scan_avant_hardening.txt /tmp/scan_apres_hardening.txt
```



Figure 52 : *Différences entre le scan avant et après durcissement*

Interprétation :

1. Passage de « closed » à « filtered ». Avant le durcissement, les ports non utilisés répondaient par un paquet RST, ce qui les faisait apparaître comme `closed` (fermés). La machine révélait ainsi son état et permettait à un attaquant de distinguer les ports fermés des ports ouverts. Après l'application des règles iptables avec politique par défaut DROP, les mêmes ports ne renvoient plus aucune réponse. Ils sont désormais classés `filtered` (filtrés), c'est-à-dire invisibles. Un attaquant ne peut plus déterminer si un port est fermé ou simplement bloqué par un pare-feu.

2. Augmentation du temps de scan. Le scan initial durait 1,29 seconde. Après durcissement, il passe à 19,51 secondes, soit environ 15 fois plus long. Cette augmentation s'explique par le fait que Nmap doit attendre un délai d'expiration (timeout) pour chaque port qui ne répond pas, au lieu de recevoir un RST immédiat. Ce comportement ralentit considérablement les tentatives de reconnaissance et décourage les scans automatisés.

En somme, le `diff` confirme objectivement que le pare-feu bloque silencieusement tous les ports non autorisés. Le serveur n'expose plus que le port SSH personnalisé (2222), et son comportement réseau ne permet plus à un observateur extérieur de cartographier facilement les services disponibles. Le durcissement atteint donc son objectif : réduire la surface d'attaque et rendre la machine quasi invisible aux scans réseau basiques.

Tableau de vérification finale

Ce tableau regroupe les commandes à exécuter pour contrôler, point par point, l'état de la machine. Il peut servir de liste de contrôle lors de chaque revue de sécurité.

Thème	Commande	Explication simple
SSH	systemctl is-active ssh	Vérifie que le service SSH est démarré
SSH	sshd -T | grep port	Affiche le port d'écoute SSH
SSH	sshd -T | grep permitrootlogin	Vérifie si root peut se connecter directement
SSH	sshd -T | grep passwordauthentication	Vérifie si l'authentification par mot de passe est autorisée
SSH	sshd -T | grep pubkeyauthentication	Vérifie si l'authentification par clé est activée
SSH	sshd -T | grep maxauthtries	Nombre maximal de tentatives d'authentification
SSH	sshd -T | grep allowusers	Liste les utilisateurs autorisés en SSH
SSH	ss -tlnp | grep 2222	Vérifie que le port 2222 est en écoute
SSH	ls -la /home/debian/.ssh/	Vérifie les permissions du dossier .ssh
SSH	cat /home/debian/.ssh/authorized_keys	Affiche la clé publique autorisée
SSH	ls -ld /home/debian /home/debian/.ssh	Vérifie les permissions des dossiers parents
Pare-feu	iptables -L -n -v	Affiche toutes les règles du pare-feu
Pare-feu	iptables -S | grep "^-P"	Affiche les politiques par défaut
Pare-feu	cat /etc/iptables/rules.v4	Affiche les règles sauvegardées pour le redémarrage
Pare-feu	systemctl is-active netfilter-persistent	Vérifie que le service de restauration des règles tourne
Pare-feu	iptables -L -n | grep f2b	Vérifie que Fail2Ban a créé ses chaînes
Fail2Ban	systemctl is-active fail2ban	Vérifie que Fail2Ban tourne
Fail2Ban	systemctl is-enabled fail2ban	Vérifie que Fail2Ban démarrera au démarrage
Fail2Ban	fail2ban-client status	Liste les jails actives
Fail2Ban	fail2ban-client status sshd	Statistiques de la jail SSH
Fail2Ban	fail2ban-client status sentech-custom	Statistiques de la jail personnalisée
Fail2Ban	fail2ban-client -t	Teste la validité de la configuration
Fail2Ban	tail -50 /var/log/fail2ban.log	Derniers événements de Fail2Ban
Audit	systemctl is-active auditd	Vérifie que le service d'audit tourne
Audit	systemctl is-enabled auditd	Vérifie qu'il démarrera au démarrage
Audit	auditctl -l | wc -l	Compte le nombre de règles d'audit chargées
Audit	auditctl -l	Affiche toutes les règles d'audit
Audit	ausearch -k identity --start today -i	Recherche les événements liés à « identity »
Audit	ausearch -k access_denied --start today -i	Recherche les accès refusés
Noyau	sysctl net.ipv4.ip_forward	Vérifie si le routage IP est activé
Noyau	sysctl net.ipv4.tcp_syncookies	Vérifie la protection SYN flood
Noyau	sysctl kernel.randomize_va_space	Vérifie l'ASLR
Noyau	sysctl -a 2>/dev/null | grep -E "accept_redirects| accept_source_route"	Vérifie que les redirections ICMP et le routage par la source sont désactivés
Comptes	passwd -S root | cut -d' ' -f2	Vérifie si le compte root est verrouillé
Comptes	grep -E "/(bash|sh|zsh)$" /etc/passwd	Liste les utilisateurs avec un shell interactif

Thème	Commande	Explication simple
Comptes	awk -F: '($2==""){print $1}' /etc/shadow	Liste les comptes sans mot de passe
Comptes	awk -F: '($3=="0"){print $1}' /etc/passwd	Liste les comptes avec UID 0
Sudo	visudo -c	Vérifie la syntaxe du fichier sudoers
Sudo	grep -E "^Defaults.*logfile" /etc/sudoers	Vérifie la journalisation sudo
Sudo	ls -la /var/log/sudo.log	Vérifie le fichier de journal sudo
Sudo	getent group sudo	Liste les membres du groupe sudo
Services inutiles	systemctl list-unit-files --state=enabled	Liste tous les services activés
Services inutiles	systemctl list-unit-files --state=enabled | grep -E "cups|avahi|bluetooth|rpc|nfs"	Vérifie les services indésirables
Services inutiles	systemctl is-enabled cups 2>/dev/null	Vérifie si CUPS est activé
Services inutiles	systemctl is-enabled avahi-daemon 2>/dev/null	Vérifie si Avahi est activé
Scan externe	nmap -sV -p- 192.168.10.10	Scanne tous les ports
Scan externe	nmap -sV -p 22,2222,80,443,3306,21,23,445 192.168.10.10	Scanne les ports sensibles
Scan externe	ping -c 3 192.168.10.10	Teste la réponse au ping
Scan externe	ssh -p 2222 -i ~/.ssh/sentech_ed25519 debian@192.168.10.10	Teste la connexion SSH par clé
Persistance	netfilter-persistent save	Sauvegarde les règles iptables
Persistance	systemctl is-enabled fail2ban	Fail2Ban au démarrage
Persistance	systemctl is-enabled auditd	Auditd au démarrage
Persistance	systemctl is-enabled ssh	SSH au démarrage
Persistance	systemctl is-enabled netfilter-persistent	Règles iptables au démarrage

Remarque : les commandes `ufw` ne figurent pas dans ce tableau, car UFW a été désinstallé sur notre machine au profit d'`iptables-persistent` (voir la Partie 3).

Simulation d'attaque complète

Pour finir, nous rejouons une attaque en trois temps : un scan de ports, une tentative de force brute, puis une connexion légitime qui doit toujours fonctionner.

Sur Kali Linux

```
echo "=== Test 1 : Scan de ports ==="
nmap -sV 192.168.10.10 -p 22,21,23,80,2222

echo "=== Test 2 : Tentatives de force brute SSH ==="
for i in 1 2 3 4; do
  echo "Tentative $i..."
  ssh -p 2222 -o ConnectTimeout=5 -o PreferredAuthentications=password \
      -o PubkeyAuthentication=no fakeuser@192.168.10.10 2>&1 | tail -1
  sleep 1
done

echo "=== Test 3 : Connexion légitime (doit fonctionner) ==="
ssh -p 2222 -i ~/.ssh/sentech_ed25519 debian@192.168.10.10 "echo 'Connexion légitime OK'"
```

REMARQUE

Si l'adresse de Kali a été bannie par le test 2, le test 3 échouera jusqu'au débannissement : `sudo fail2ban-client set sshd unbanip 192.168.10.99`. Cela prouve d'ailleurs que Fail2Ban fonctionne.

Vérification des traces sur Debian Serveur :

Sur Debian Serveur

```
sudo fail2ban-client status sshd
sudo journalctl -u ssh --since today | tail -20
sudo ausearch -k root_commands --start today | tail -20
```



Figure 53 : *Événements Auditd (clé root_commands) enregistrés pendant la simulation*

Debian Serveur dispose maintenant d'un pare-feu local actif, de Fail2Ban, d'Auditd, d'une politique de mots de passe stricte et d'un sudo journalisé, en complément du pare-feu périmétrique OPNsense.

PARTIE VI

Partie VI : Audit, analyse et réponse locale

Contexte fictif. SenTech Solutions a finalisé le déploiement de son infrastructure sécurisée. Avant la mise en production officielle, la direction exige un audit de sécurité complet et documenté. Notre mission, en tant qu'administrateur sécurité senior, est de conduire cet audit en trois phases : évaluation de la posture de sécurité avec **Lynis**, analyse approfondie des journaux système, scan de vulnérabilités avec **OpenVAS**. Nous terminons par la mise en place d'une procédure de **sauvegarde et restauration** fiable, et par la rédaction du **rapport de remédiation** final destiné à la direction.

Machines impliquées : Debian Serveur (192.168.10.10, machine auditée), Debian Client (192.168.10.11, serveur de sauvegarde), Kali Linux (192.168.10.99, scanner OpenVAS/Greenbone).

Rappel théorique

L'audit de sécurité est une évaluation systématique et documentée de la posture de sécurité d'un système : « Ce système est-il configuré conformément aux bonnes pratiques et aux exigences de sécurité ? ». Il suit le cycle **PDCA** (Plan-Do-Check-Act) et se situe dans la phase *Check* : il mesure l'écart entre l'état réel du système et l'état souhaité (référentiels CIS Benchmarks, recommandations de l'ANSSI).

Lynis est un outil d'audit de sécurité open source pour Unix/Linux. Il s'exécute localement sur la machine auditée et vérifie des centaines de points de configuration, chacun classé `[OK]`, `[WARNING]`, `[SUGGESTION]` ou `[FOUND]`. Il calcule un **Hardening Index** de 0 à 100 :

Score	Niveau	Interprétation
0 à 39	Faible	Système par défaut, non durci
40 à 64	Moyen	Quelques mesures appliquées
65 à 79	Bon	Hardening sérieux effectué
80 à 100	Excellent	Niveau production sécurisé

Les **journaux système** servent à la surveillance proactive, à l'investigation après un incident (forensique) et à la conformité. Voici où les trouver :

Fichier / commande	Contenu
/var/log/auth.log	Connexions SSH, sudo, su, PAM (présent seulement si rsyslog est installé)
/var/log/syslog	Messages système généraux (idem)
/var/log/kern.log	Messages du noyau (idem)
/var/log/fail2ban.log	Bannissements Fail2Ban
/var/log/audit/	Événements Auditd
/var/log/clamav/	Rapports ClamAV
journalctl	Journal systemd centralisé : c'est la source principale sur Debian 13

OpenVAS (Greenbone Community Edition) teste activement une cible depuis le réseau, à la différence de Lynis qui s'exécute localement :

	Lynis	OpenVAS
Perspective	Interne (sur la machine elle-même)	Externe (depuis le réseau)

	Lynis	OpenVAS
Teste	La configuration du système	Les services exposés et leurs vulnérabilités
Analogie	Un médecin qui vous examine	Un test de résistance aux effractions
Résultat	Score et recommandations	Liste de CVE avec score CVSS

Le **score CVSS** note les vulnérabilités de 0 à 10 : de 9,0 à 10,0 *critique* (à corriger immédiatement), de 7,0 à 8,9 *élevé* (24 à 72 h), de 4,0 à 6,9 *moyen* (30 jours), de 0,1 à 3,9 *faible* (prochaine maintenance).

La règle 3-2-1 pour les sauvegardes : 3 copies des données (l'original, une copie sur le serveur de sauvegarde et une copie hors site), sur 2 supports différents, dont 1 copie hors site. Une sauvegarde qui n'a jamais été vérifiée (par exemple par une empreinte `sha256sum`) est aussi inutile qu'une absence de sauvegarde.

Partie 1 : Audit Lynis, mesure de la posture de sécurité

Installation de Lynis sur le serveur

Sur Debian Serveur
```
sudo apt install lynis -y
lynis --version
```

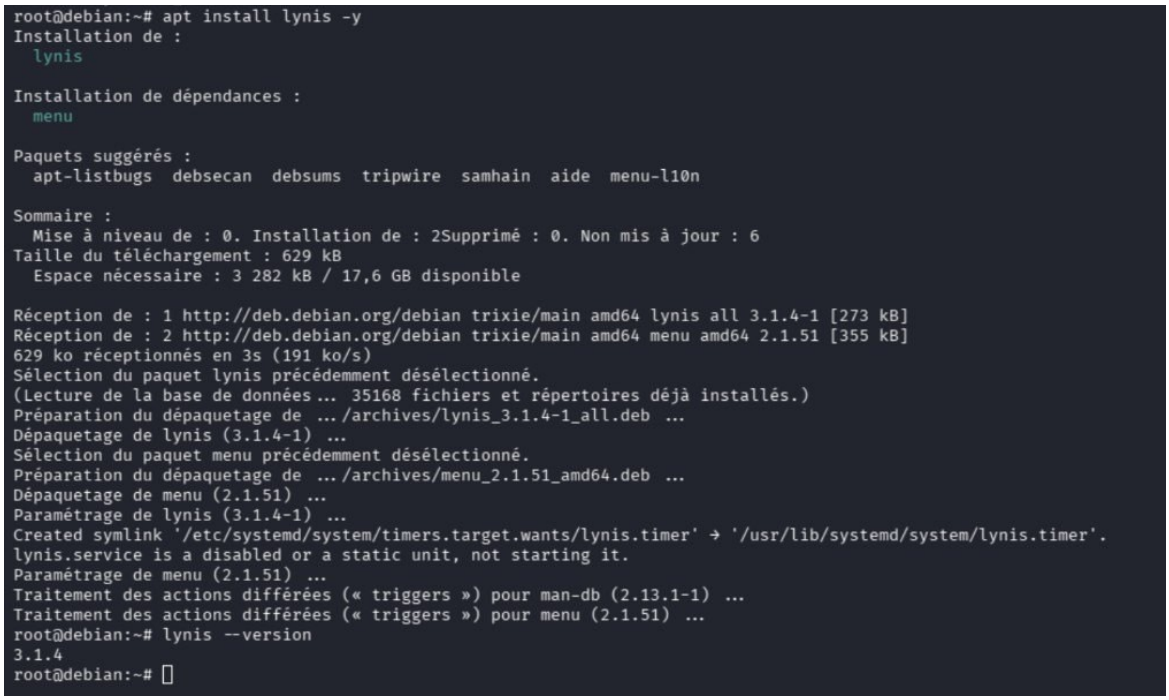


Figure 54 : *Installation de Lynis (version 3.1.4)*

Premier audit : mesure de l'état actuel

Sur Debian Serveur
```
sudo mkdir -p /var/log/sentech/audits
sudo lynis audit system --verbose 2>&1 | \
  sudo tee /var/log/sentech/audits/lynis_$(date +%Y%m%d_%H%M%S).log
```

La commande lance l'audit et enregistre à la fois l'affichage à l'écran et un fichier daté (`tee`), ce qui permettra la comparaison ultérieure.

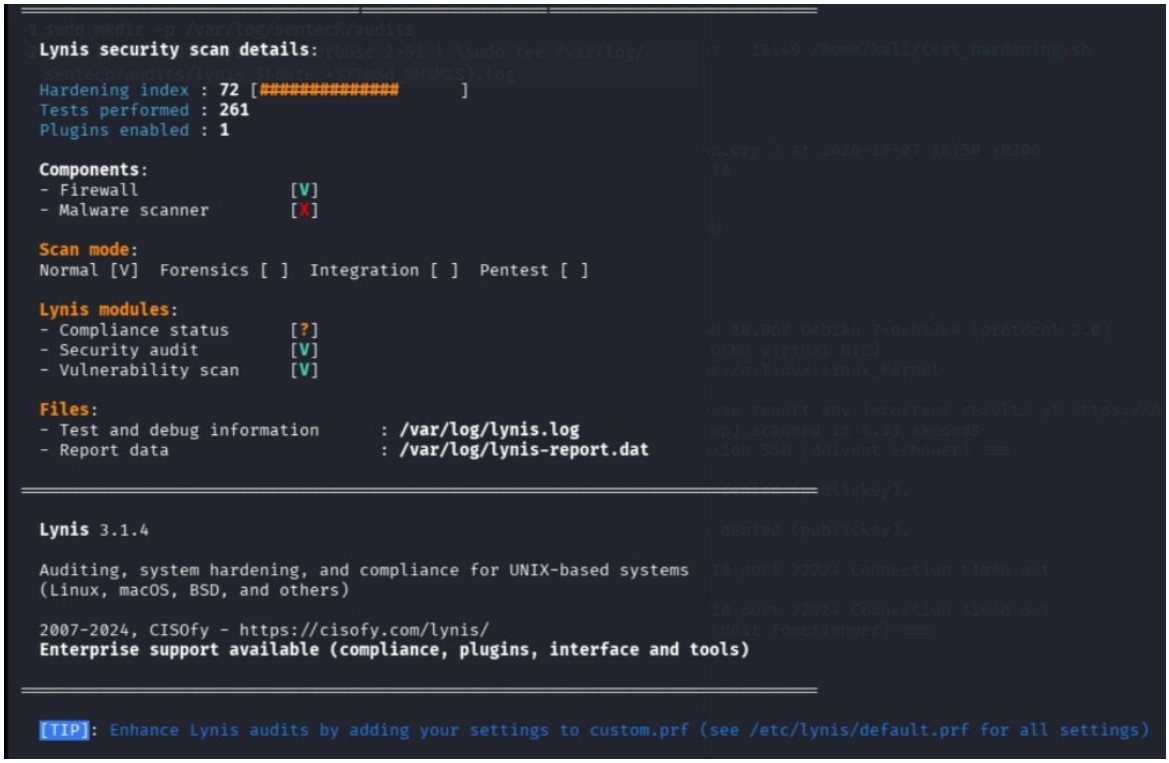


Figure 55 : *Premier audit : Hardening index de 72, 261 tests effectués*

Le serveur obtient **72/100**, soit le niveau « Bon » de notre tableau. La capture montre aussi que le pare-feu est détecté (`Firewall [V]`) mais qu'aucun logiciel antimalware n'est installé (`Malware scanner [X]`) : ce point sera traité dans la Partie VII avec ClamAV.

Lecture des avertissements :

Sur Debian Serveur

```
grep "Hardening index" /var/log/sentech/audits/lynis_*.log | tail -1
grep -c "\[WARNING\]" /var/log/sentech/audits/lynis_*.log | tail -1
grep "\[WARNING\]" /var/log/sentech/audits/lynis_*.log | tail -20
```

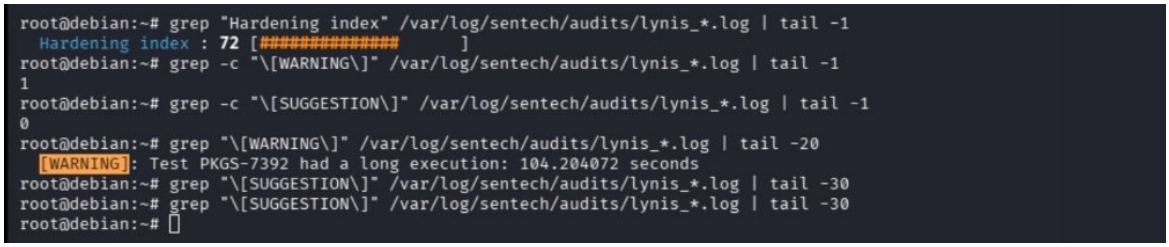


Figure 56 : *Un seul avertissement : un test (PKGS-7392) a duré 104 secondes, ce qui n'est pas une faille de sécurité*

POUR COMPRENDRE **où sont les suggestions ?**

L'affichage de Lynis contient des suggestions, mais elles se retrouvent plus facilement dans le fichier de données `/var/log/lynis-report.dat`, sur les lignes qui commencent par `suggestion[]=` : `sudo grep "^suggestion" /var/log/lynis-report.dat`. C'est de cette liste que nous tirons les sept corrections suivantes.

Application des recommandations de Lynis

Correction 1 : paramètres noyau complémentaires. On ajoute à la fin du fichier sysctl trois paramètres de protection réseau :

Sur Debian Serveur

```
sudo nano /etc/sysctl.d/99-sentech-security.conf
```

/etc/sysctl.d/99-sentech-security.conf (lignes à ajouter)

```
# === Compléments recommandés par Lynis ===
# Protège contre l'attaque « TIME-WAIT assassination »
net.ipv4.tcp_rfc1337 = 1
# Journalise les paquets dont l'adresse source est impossible (« martiens »)
net.ipv4.conf.all.log_martians = 1
net.ipv4.conf.default.log_martians = 1
```

Sur Debian Serveur

```
sudo sysctl --system
sudo sysctl fs.suid_dumpable net.ipv4.tcp_rfc1337 net.ipv4.conf.all.log_martians
```



Figure 57 : *Valeurs lues avec sysctl (les paramètres tcp_rfc1337 et log_martians valent 0 avant la correction ; ils doivent valoir 1 une fois les lignes ajoutées)*

Correction 2 : bannière SSH (déjà appliquée, vérifions simplement).

Sur Debian Serveur

```
sudo sshd -T | grep banner
grep -i "^Banner" /etc/ssh/sshd_config || echo "Banner /etc/issue.net" | sudo tee -a
/etc/ssh/sshd_config
sudo systemctl restart ssh
```



Figure 58 : *La directive Banner pointe sur /etc/issue.net*

Correction 3 : synchronisation horaire (NTP). Des horloges justes sont indispensables pour comparer des journaux entre machines et pour dater une preuve en cas d'incident.

Sur Debian Serveur

```
timedatectl status
sudo apt install systemd-timesyncd -y
sudo systemctl enable --now systemd-timesyncd
sudo nano /etc/systemd/timesyncd.conf
```

/etc/systemd/timesyncd.conf

```
[Time]
NTP=pool.ntp.org 0.africa.pool.ntp.org 1.africa.pool.ntp.org
FallbackNTP=time.google.com time.cloudflare.com
```

Sur Debian Serveur

```
sudo systemctl restart systemd-timesyncd
timedatectl status
```

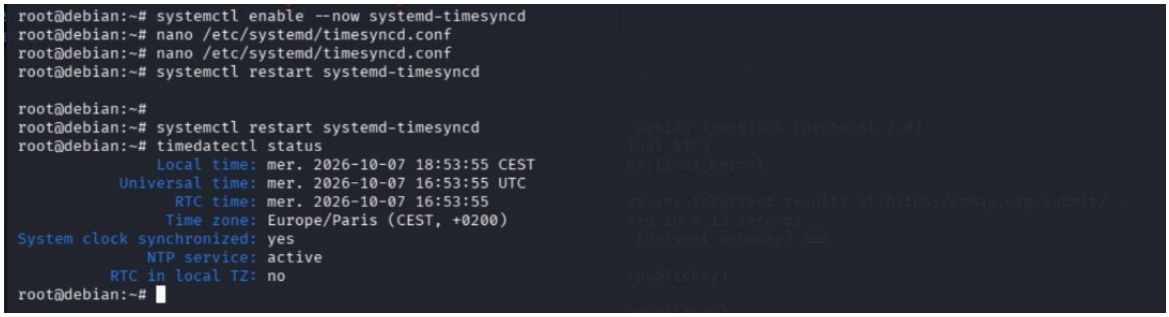


Figure 59 : *« System clock synchronized: yes » : l'horloge est synchronisée*

Correction 4 : intégrité des fichiers (AIDE). AIDE photographie l'empreinte (hash) des fichiers système, puis signale tout fichier modifié par la suite : c'est un détecteur de falsification.

Sur Debian Serveur

```
sudo apt install aide aide-common -y
sudo aideinit
sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db
sudo aide --check 2>/dev/null | tail -20
```

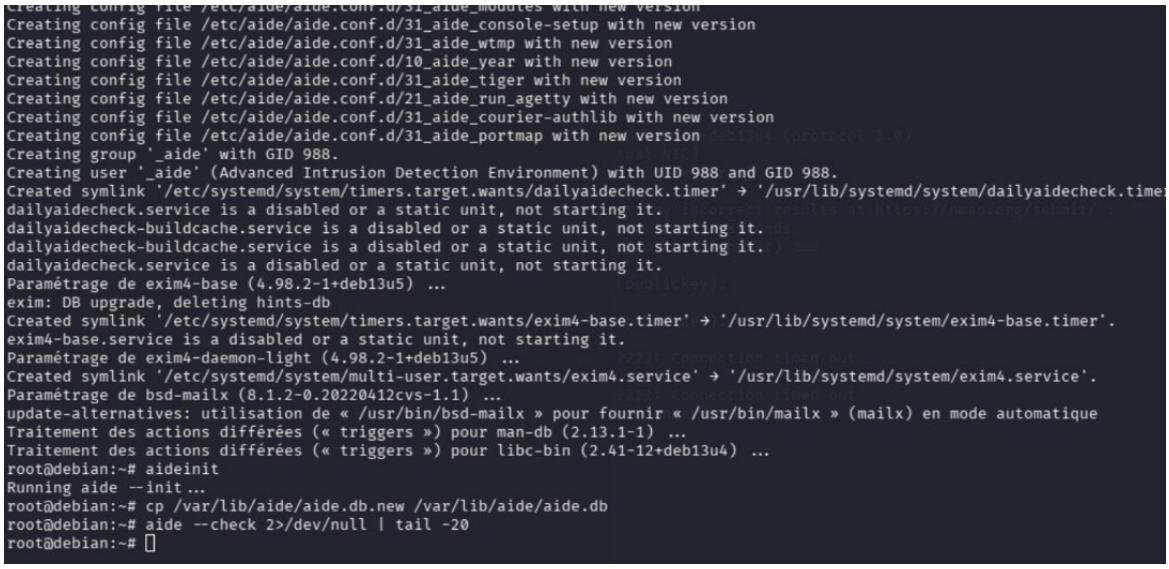


Figure 60 : *Installation d'AIDE, initialisation de la base de référence et première vérification*

> **REMARQUE** **AIDE installe aussi un serveur de messagerie**
> L'installation d'AIDE a tiré en dépendance le serveur de messagerie `exim4`, destiné à envoyer des rapports par e-mail. Il ne doit pas être accessible depuis le réseau : le contrôle `ss -tlnp` réalisé plus loin (Partie 3) montre que seul `sshd` écoute.

Correction 5 : politique umask. Le `umask` fixe les permissions par défaut des nouveaux fichiers. La valeur `027` donne le droit de lecture au groupe mais **aucun droit aux autres** (un fichier créé en 640 au lieu de 644).

Sur Debian Serveur

```
umask
sudo nano /etc/profile
```

Ajouter **uniquement** la ligne suivante en fin de fichier `/etc/profile` :

/etc/profile (fin de fichier)

```
umask 027
```

Puis, **dans le terminal** (et non dans le fichier), appliquer la même valeur à `bash.bashrc` :

Sur Debian Serveur

```
echo "umask 027" | sudo tee -a /etc/bash.bashrc
```

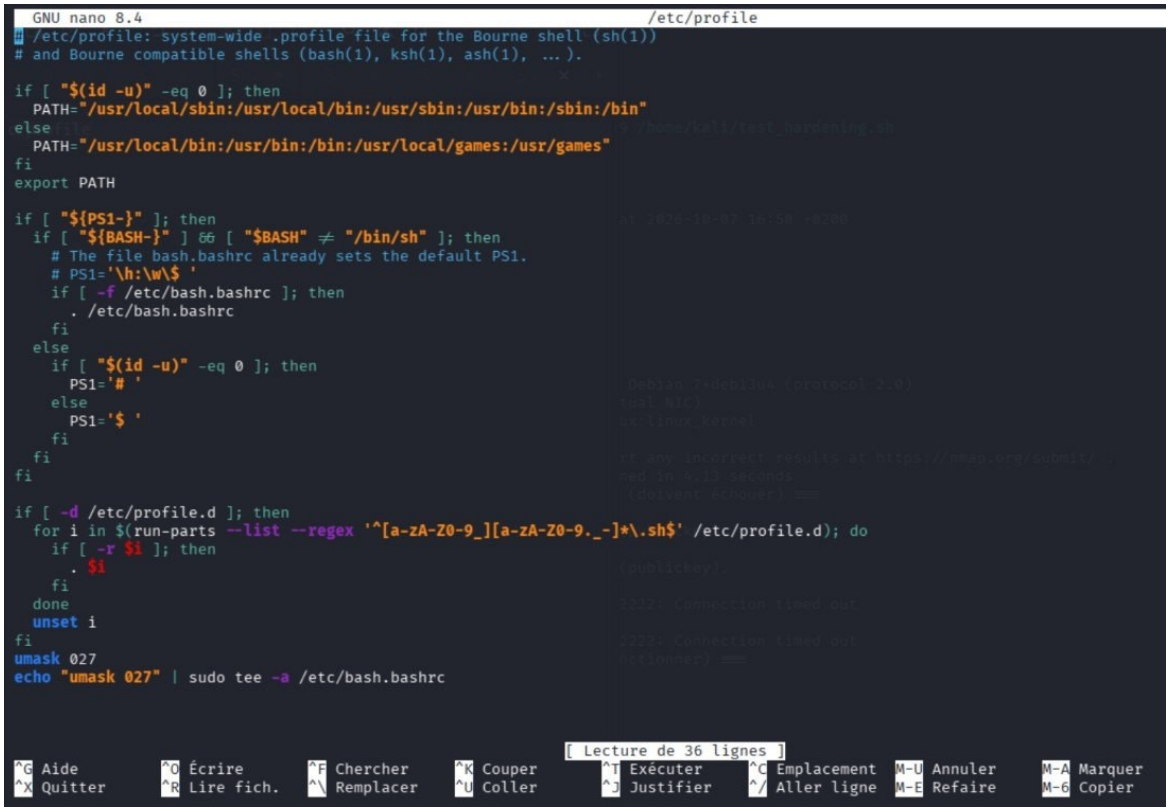


Figure 61 : *Fin du fichier /etc/profile : la seconde ligne visible sur la capture (echo ...) a été ajoutée par erreur dans le fichier*

ATTENTION **Erreur à éviter : une commande collée dans un fichier**

Sur la capture, la ligne `echo "umask 027" | sudo tee -a /etc/bash.bashrc` se trouve **dans** `/etc/profile`. Une commande doit être tapée dans le terminal, pas écrite dans le fichier : à chaque connexion, `/etc/profile` la rejouerait et ajouterait une ligne de plus à `bash.bashrc`. On la supprime avec : `sudo sed -i '/^echo "umask 027"/d' /etc/profile`

Correction 6 : sécurité des répertoires personnels. Par défaut un utilisateur peut lire le dossier personnel des autres. On passe les dossiers `/home/*` en `750`.

Sur Debian Serveur

```
ls -la /home/
for user_home in /home/*/; do
  sudo chmod 750 "$user_home"
  echo "Permissions 750 appliquées sur $user_home"
done
ls -la /home/
```

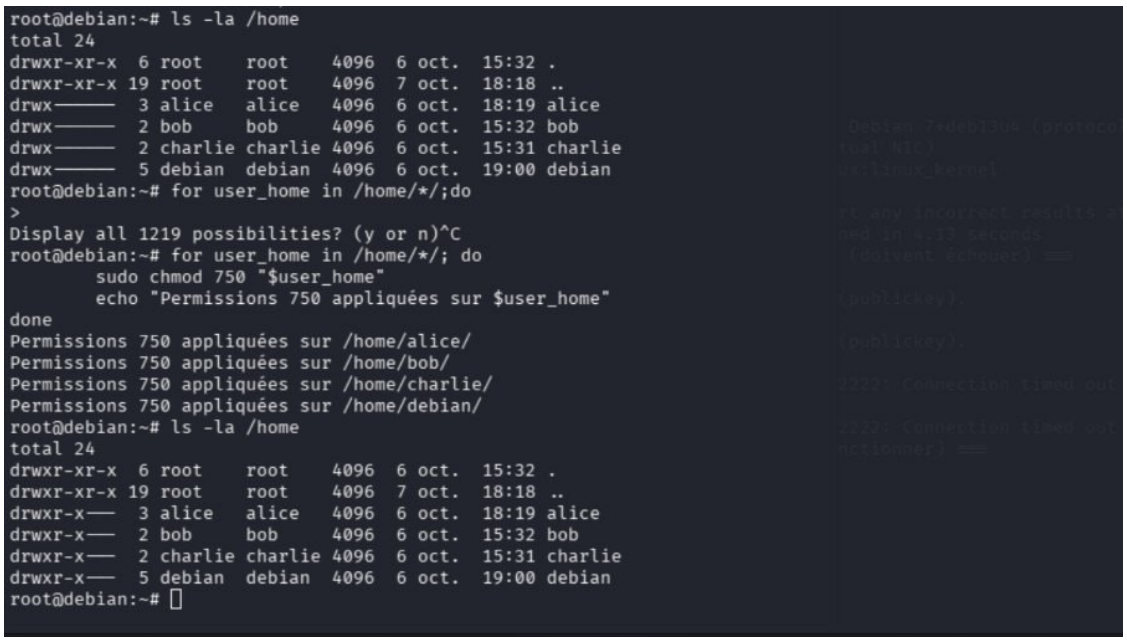


Figure 62 : *Dossiers personnels avant et après chmod 750*

Correction 7 : comptes système et comptes sans mot de passe. On recherche d'abord les comptes sans mot de passe et les comptes système qui ont un shell interactif, puis on verrouille les comptes système standard.

Sur Debian Serveur (en root)

```
awk -F: '($2 == "") { print $1 }' /etc/shadow
awk -F: '($3 < 1000) && ($7 != "/usr/sbin/nologin") && ($7 != "/bin/false") { print $1 " :
shell=" $7 }' /etc/passwd

for account in daemon bin sys games man lp mail news uucp proxy backup list irc; do
  usermod --shell /usr/sbin/nologin "$account" 2>/dev/null
  passwd -l "$account" 2>/dev/null
done
echo "Comptes système verrouillés."
```



Figure 63 : *Aucun compte sans mot de passe ; seuls root et sync gardent un shell ; comptes système verrouillés*

REMARQUE **Ne pas verrouiller le compte « backup » de Debian Client**

Le nom `backup` figure dans la liste ci-dessus parce que c'est un compte système Debian standard,

généralement inutilisé. Sur **Debian Serveur**, le verrouiller est normal et sans danger. Il ne faut pas le confondre avec le compte applicatif `backup` que nous créerons plus tard sur **Debian Client** pour recevoir les sauvegardes : ce sont deux machines différentes. Ne créez jamais de compte applicatif `backup` sur Debian Serveur lui-même, pour éviter toute confusion.

Second audit et comparaison

1. Lancer l'audit et sauvegarder le journal.

Sur Debian Serveur
```
sudo lynis audit system 2>&1 | sudo tee /var/log/sentech/audits/lynis_$(date +
%Y%m%d_%H%M%S)_postcorrections.log
```

ATTENTION à la syntaxe de date

Dans notre essai, la commande a été saisie avec `$(date%Y%m%d_%H%M%S)` (le `+` oublié) ; le fichier s'est alors appelé `lynis__postcorrections.log`, sans date. La forme correcte est `$(date +%Y%m%d_%H%M%S)`.

2. Afficher le Hardening index du rapport final :

Sur Debian Serveur
```
grep "Hardening index" /var/log/sentech/audits/lynis_*postcorrections*.log
```

3. Script de comparaison. On l'écrit une fois, puis on le relance à chaque audit :

Sur Debian Serveur
```
sudo nano /root/compare_lynis.sh
sudo chmod +x /root/compare_lynis.sh
sudo /root/compare_lynis.sh
```

/root/compare_lynis.sh
```
#!/bin/bash
# Compare l'audit Lynis initial et l'audit après corrections
DIR=/var/log/sentech/audits

# Rapport initial : le plus ancien fichier sans "postcorrections"
INITIAL_LOG=$(ls -tr $DIR/lynis_*.log | grep -v postcorrections | head -1)
# Rapport final : le plus récent fichier "postcorrections"
FINAL_LOG=$(ls -t $DIR/lynis_*postcorrections.log | head -1)

echo "=== AUDIT INITIAL ==="
echo "Fichier : $INITIAL_LOG"
grep "Hardening index" "$INITIAL_LOG"
grep -c "\[WARNING\]" "$INITIAL_LOG"

echo ""
echo "=== AUDIT POST-CORRECTIONS ==="
echo "Fichier : $FINAL_LOG"
grep "Hardening index" "$FINAL_LOG"
grep -c "\[WARNING\]" "$FINAL_LOG"

echo ""
echo "=== WARNINGS RESTANTS ==="
grep "\[WARNING\]" "$FINAL_LOG"
```



Figure 64 : *Résultat : index 72 avant, 73 après corrections ; 1 avertissement avant, 0 après*

POUR COMPRENDRE **Interpréter le résultat**

Le gain est faible (+1 point) car le serveur avait déjà été fortement durci dans la Partie V avant ce premier audit. Un score de 72 à 73 correspond au niveau « Bon » ; l'avertissement unique du premier audit (un test lent) a disparu. Une mesure ultérieure, utilisée dans le rapport final, affiche 71 : l'indice varie d'un ou deux points d'une exécution à l'autre selon les tests réalisés. Ce qui compte ici est l'ordre de grandeur et l'absence d'avertissement, pas la valeur exacte. Pour dépasser 80, il faudrait appliquer les suggestions plus avancées (partitions séparées, durcissement du noyau et des modules, etc.), au prix d'un travail plus lourd.

Partie 2 : Analyse approfondie des journaux système

Exploration avec journalctl

`journalctl` interroge le journal centralisé de systemd. Les options les plus utiles sont `-n` (nombre de lignes), `-f` (suivi en direct), `-u` (un service), `-b` (depuis le dernier démarrage), `--since`/`--until` (période) et `-p err` (erreurs seulement).

Sur Debian Serveur

```
sudo journalctl -n 50
sudo journalctl -f
sudo journalctl -u ssh -n 20
sudo journalctl -u fail2ban -n 20
sudo journalctl -u auditd -n 20
sudo journalctl -b
sudo journalctl --since today
sudo journalctl --since "2026-10-07 08:00:00" --until "2026-10-07 18:00:00"
sudo journalctl -p err --since today
```

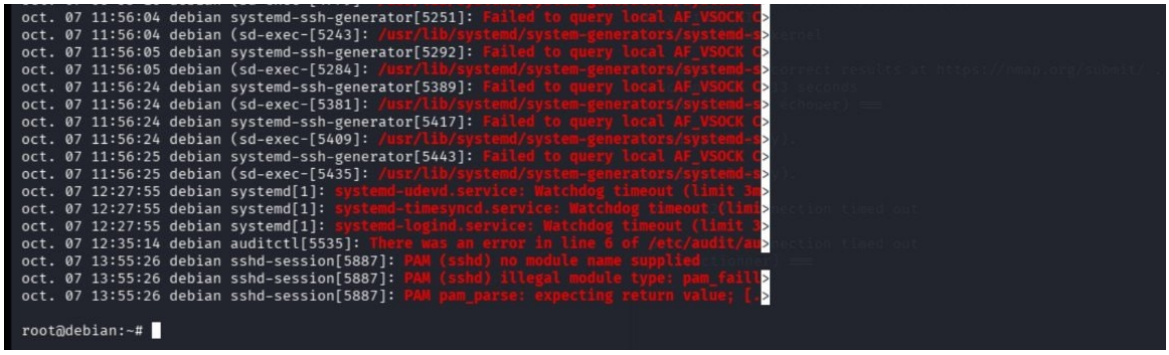


Figure 65 : *Exemple de sortie journalctl*

Analyse des connexions SSH

Sur Debian Serveur

```
# Échecs de connexion
journalctl -u ssh --since today | grep -i "Failed"
# Connexions réussies
journalctl -u ssh --since today | grep -i "Accepted" | tail -20
# Suivi en direct
journalctl -u ssh -f
```

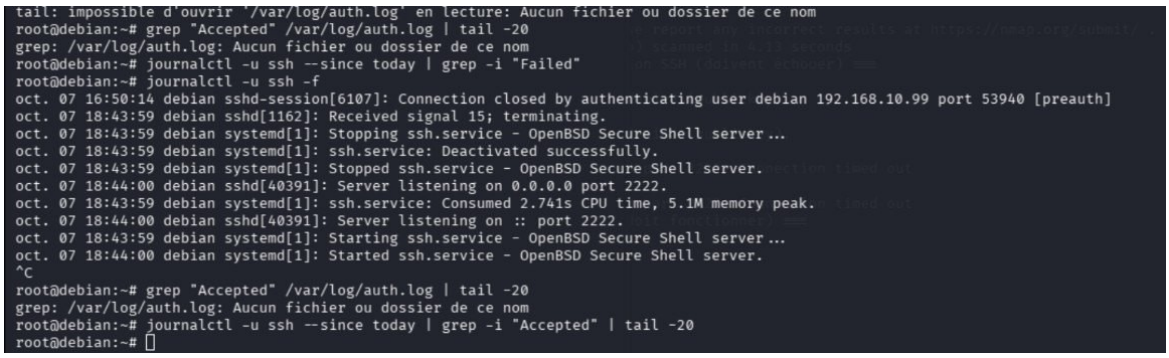


Figure 66 : *Recherche des connexions SSH dans le journal*

Recherches plus ciblées :

Sur Debian Serveur

```
# Échecs, utilisateurs invalides et connexions fermées
journalctl -u ssh --since today | grep -E "Failed password|Invalid user|Connection closed" |
tail -20

# Les 10 adresses IP qui échouent le plus souvent
journalctl -u ssh --since today | grep -E "Failed password|Invalid user" | grep -oE "from [0-
9.]+" | sort | uniq -c | sort -rn | head -10

# Commandes sudo et ouvertures de session
journalctl -t sudo --since today | tail -20
journalctl --since today | grep -E "session opened|session closed" | tail -20
```

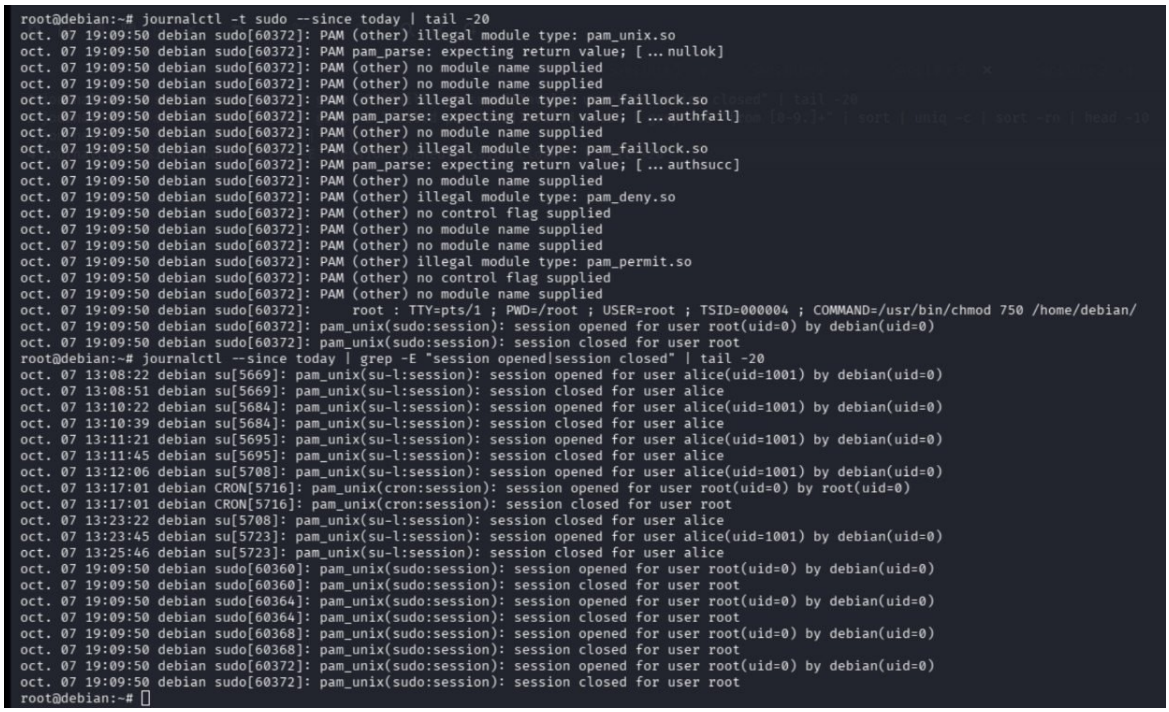


Figure 67 : *Résultats de ces recherches*

POUR COMPRENDRE **la chaîne de commandes « sort | uniq -c | sort -rn »**

Cette suite sert à **compter** : `grep -oE` extrait l'adresse IP de chaque ligne, le premier `sort` regroupe les lignes identiques, `uniq -c` compte les doublons et le second `sort -rn` classe du plus fréquent au moins fréquent. Résultat : la liste des adresses qui attaquent le plus.

Générer des événements à analyser

Un petit script produit des événements dans chacune des sources, puis les relit.

Sur Debian Serveur
```
sudo nano test_audit_ssh.sh
```

test_audit_ssh.sh
```
#!/bin/bash
echo "=== Génération d'événements ==="
ls /root
cat /etc/shadow | head -3
systemctl status ssh

echo ""
echo "=== Connexions réussies aujourd'hui ==="
journalctl -u ssh --since today | grep "Accepted"

echo ""
echo "=== Échecs d'authentification aujourd'hui ==="
journalctl -u ssh --since today | grep -E "Failed|Invalid"

echo ""
echo "=== Commandes sudo aujourd'hui ==="
journalctl -t sudo --since today | tail -10
```

Sur Debian Serveur (en root)
```
chmod +x test_audit_ssh.sh
./test_audit_ssh.sh
```

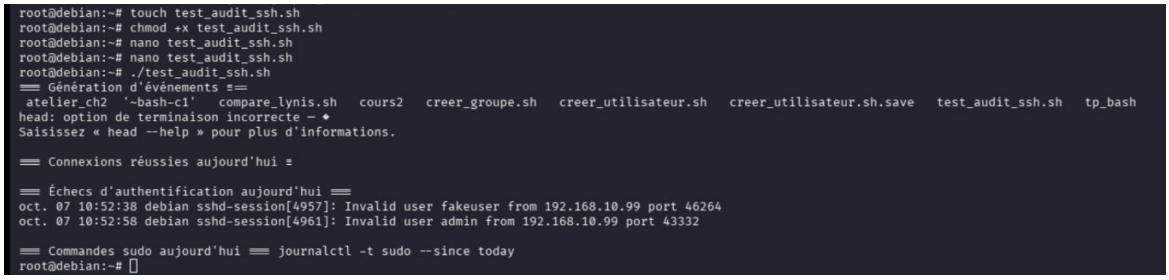


Figure 68 : *Exécution du script de génération d'événements*

Analyse avancée avec Auditd

Ce second script regroupe tous les événements d'audit de la journée dans un rapport lisible. Il doit être lancé en root.

Sur Debian Serveur
```
sudo nano rapport_audit.sh
```

rapport_audit.sh
```
#!/bin/bash
# ============================================================
# Script : rapport_audit.sh
# Objet  : Générer un rapport complet des événements d'audit
# Auteur : SenTech Solutions
# Date   : 2026-10-07
# ============================================================
echo "========================================="
echo " RAPPORT D'AUDIT - $(date '+%Y-%m-%d %H:%M')"
echo "========================================="

echo ""
echo "=== 1. Résumé général des événements ==="
aureport --start today

echo ""
echo "=== 2. Événements d'authentification ==="
aureport --auth --start today

echo ""
```

```
echo "=== 3. Résumé exécutif ==="
aureport -x --start today --summary

echo ""
echo "=== 4. Fichiers accédés (30 derniers) ==="
aureport -f --start today | head -30

echo ""
echo "=== 5. Événements échoués ==="
aureport --failed --start today

echo ""
echo "=== 6. Modifications d'identité (/etc/passwd, etc.) ==="
ausearch -k identity --start today -i

echo ""
echo "=== 7. Commandes root (30 dernières) ==="
ausearch -k root_commands --start today -i | tail -30

echo ""
echo "=== 8. Changements de permissions (20 derniers) ==="
ausearch -k perm_change --start today -i | tail -20

echo ""
echo "=== 9. Accès refusés (20 derniers) ==="
ausearch -k access_denied --start today -i | tail -20

echo ""
echo "=== Fin du rapport ==="
```

Sur Debian Serveur (en root)

```
chmod +x rapport_audit.sh
./rapport_audit.sh > /var/log/sentech/rapport_audit_$(date +%Y%m%d).txt
```

Partie 3 : Scan de vulnérabilités avec OpenVAS/Greenbone (sur Kali)

L'audit Lynis a évalué la configuration interne du serveur. OpenVAS va maintenant tester les services exposés depuis le réseau, exactement comme le ferait un attaquant externe.

Sur Kali Linux

```
sudo apt update && sudo apt dist-upgrade -y
sudo apt install gvm -y
sudo gvm-setup
sudo gvm-start
sudo gvm-check-setup
```

POUR COMPRENDRE **ce que fait chaque commande**

`gvm-setup` installe et initialise Greenbone, télécharge les bases de vulnérabilités (c'est long) et affiche le mot de passe de l'utilisateur `admin` : **notez-le**. `gvm-start` démarre les services, et `gvm-check-setup` vérifie que tout est en ordre.

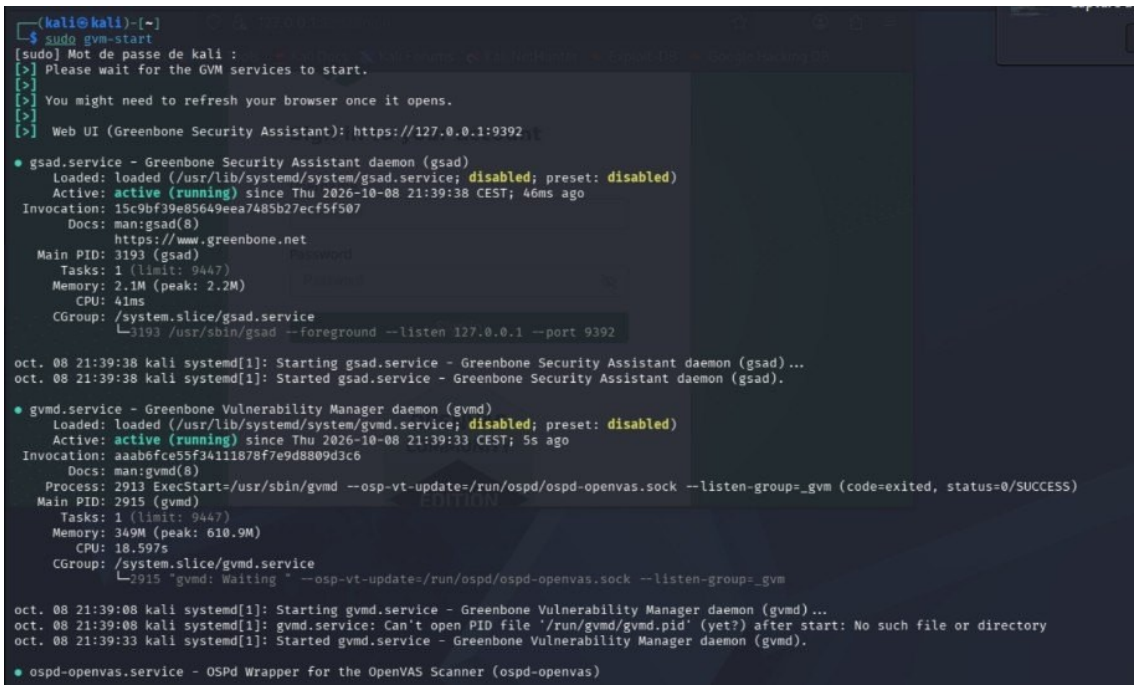


Figure 69 : *Démarrage des services Greenbone : l'interface web est sur https://127.0.0.1:9392*

Accès à l'interface web Greenbone

Sur Kali Linux, ouvrir Firefox (installé par défaut) à l'adresse `https://127.0.0.1:9392` et se connecter avec l'utilisateur `admin`. Le navigateur signale un certificat non reconnu : c'est normal, il s'agit d'un certificat auto-signé local.

Créer une cible de scan

Dans l'interface Greenbone : **Configuration > Targets > New Target** (icône étoile).

Champ	Valeur
Name	Debian-Serveur-SenTech
Hosts	192.168.10.10
Port List	All IANA assigned TCP and UDP
Alive Test	Consider Alive

Créer et lancer un scan

Scans > Tasks > New Task (icône étoile).

Champ	Valeur
Name	Audit SenTech Debian Serveur
Scan Config	Full and fast
Target	Debian-Serveur-SenTech
Scanner	OpenVAS Default

Cliquer sur **Save**, puis sur le bouton **Start** (triangle vert). Un scan complet prend 10 à 30 minutes selon les services exposés.

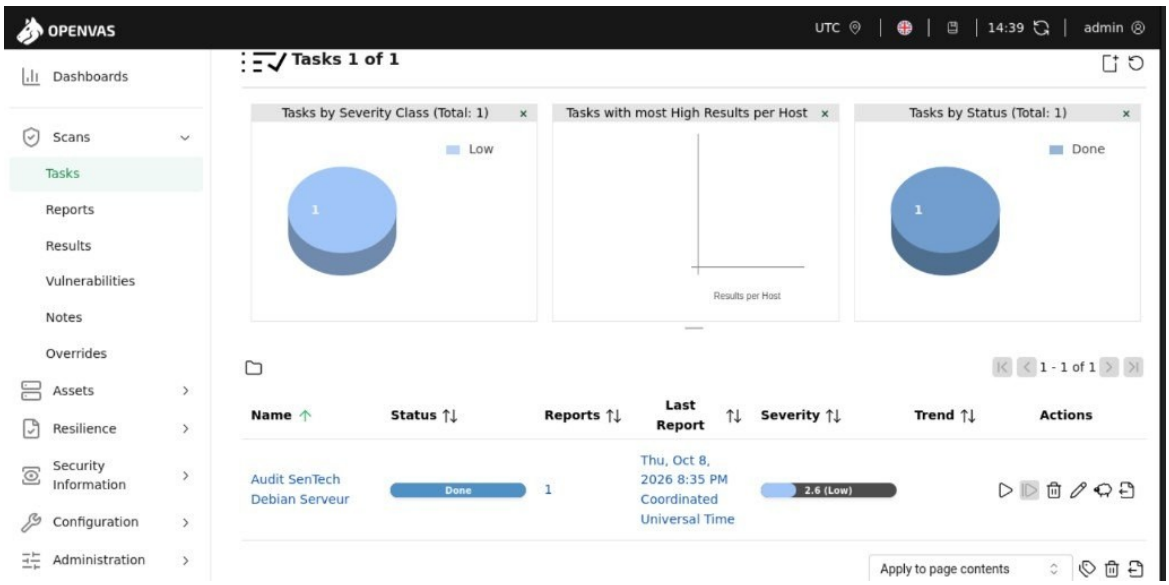


Figure 70 : *La tâche « Audit SenTech Debian Serveur » est terminée (Done) avec une sévérité maximale de 2,6 (Low)*

Analyser les résultats

Une fois le statut **Done** atteint : **Scans > Results**, filtrer par hôte 192.168.10.10 et trier par *Severity* décroissante.

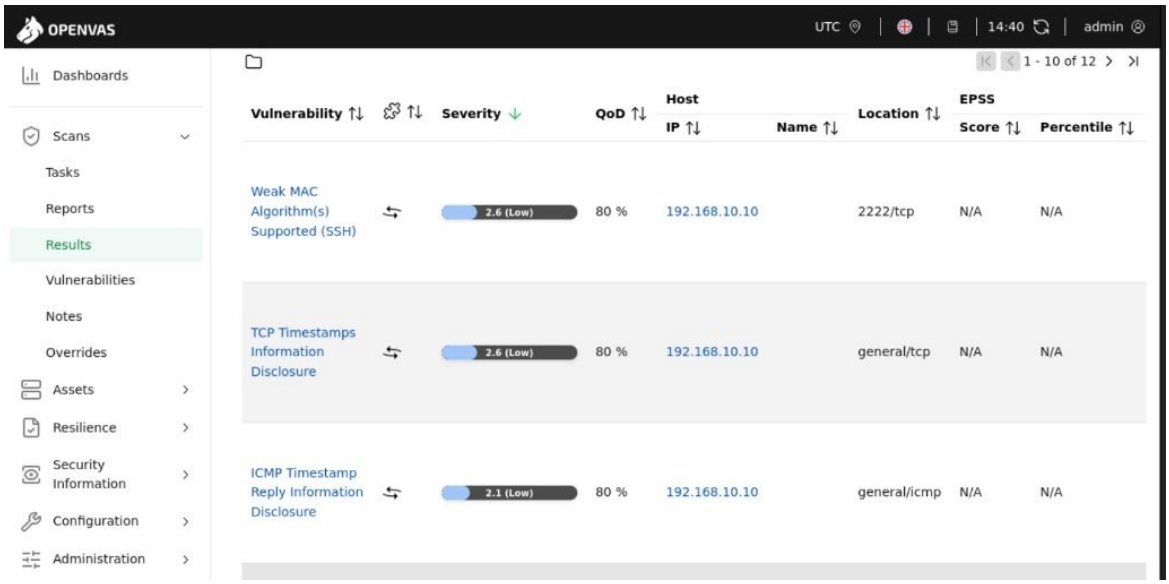


Figure 71 : *Résultats classés par sévérité : 12 résultats, tous de niveau faible*

Le scan trouve 12 résultats. Les plus sévères ont une note de 2,6/10 : **aucune vulnérabilité critique, élevée ou moyenne**. Les trois résultats visibles sur la capture et leur correction :

Résultat (note CVSS)	Signification	Correction proposée
Weak MAC Algorithm(s) Supported (SSH), 2,6, port 2222	Le serveur SSH accepte encore des algorithmes d'intégrité anciens (par exemple hmac-sha1)	Dans sshd_config : MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com
TCP Timestamps Information Disclosure, 2,6	Les horodatages TCP permettent d'estimer depuis combien de temps la machine est allumée	sysctl : net.ipv4.tcp_timestamps = 0 (risque très faible, correction facultative)
ICMP Timestamp Reply Information Disclosure, 2,1	La machine répond aux requêtes ICMP « timestamp », et révèle ainsi son heure	iptables -A INPUT -p icmp --icmp-type timestamp-request -j DROP

Sur Debian Serveur : vérifier les services encore exposés.

Sur Debian Serveur

```
ss -tlnp
```

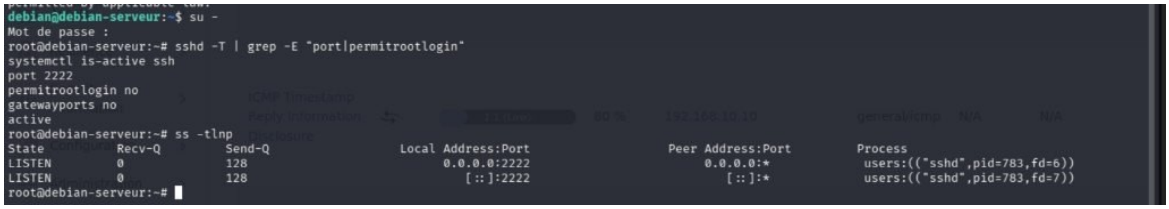


Figure 72 : *Seul sshd écoute, sur le port 2222 (IPv4 et IPv6)*

Depuis Kali : scan Nmap final de confirmation.

Sur Kali Linux

```
nmap -sV --open 192.168.10.10 -oN /tmp/nmap_final_tp5.txt
cat /tmp/nmap_final_tp5.txt
```

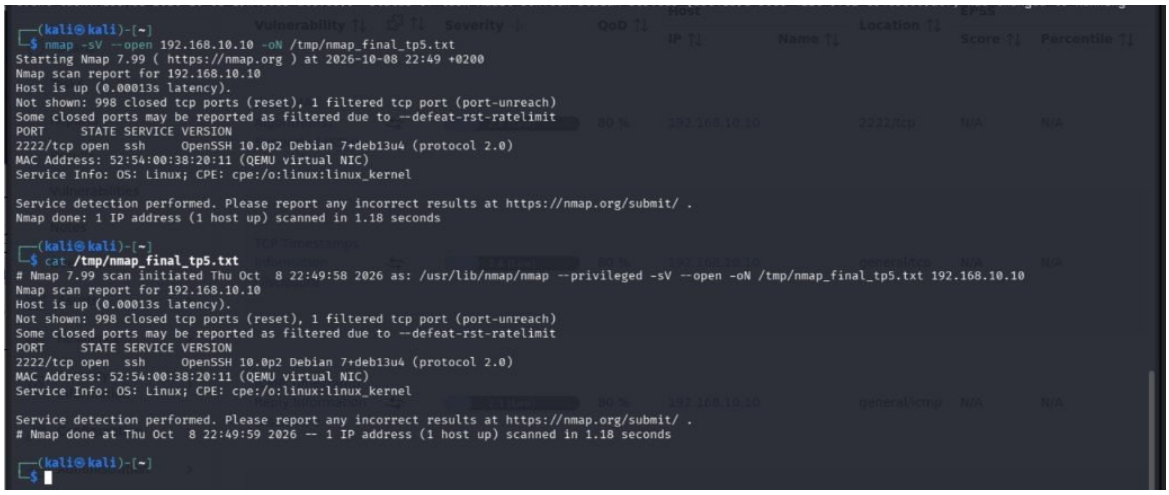


Figure 73 : *Scan final : un seul port ouvert, 2222/tcp (OpenSSH 10.0p2 sur Debian)*

ATTENTION **Point d'attention : « closed » au lieu de « filtered »**

Le scan final affiche « 998 closed tcp ports (reset) », alors que celui du 7 octobre (Partie 8) affichait « filtered ». Les ports fermés répondent donc à nouveau par un RST, ce qui indique que la politique DROP n'était plus active sur le serveur lors de ce scan (règles non rechargées après un redémarrage, par exemple). À contrôler : `sudo iptables -S | grep "^-P"` (doit afficher `-P INPUT DROP`), `systemctl is-active netfilter-persistent`, puis `sudo netfilter-persistent save`, redémarrage et nouveau scan. Le résultat reste favorable (un seul port ouvert) mais la protection par le pare-feu doit être rétablie.

Exporter le rapport. Scans > Reports, ouvrir le rapport, choisir le format PDF ou XML, puis *Download*.

Partie 4 : Sauvegarde et restauration système

Un système bien sécurisé sans plan de sauvegarde éprouvé n'est pas un système fiable. Notre schéma est simple : **Debian Serveur** envoie ses sauvegardes à **Debian Client**, chaque nuit et sans intervention humaine. Debian Client (192.168.10.11) a déjà son adressage réseau statique configuré (Partie III) ; nous vérifions la connectivité, puis nous le préparons à recevoir des sauvegardes.

Préparation de Debian Client

Sur Debian Client

```
ping -c 3 192.168.10.10

# Installer rsync (outil de copie incrémentale) et créer l'utilisateur de sauvegarde
```

```
sudo apt install rsync -y
sudo useradd -m -s /bin/bash backup
sudo passwd backup
```

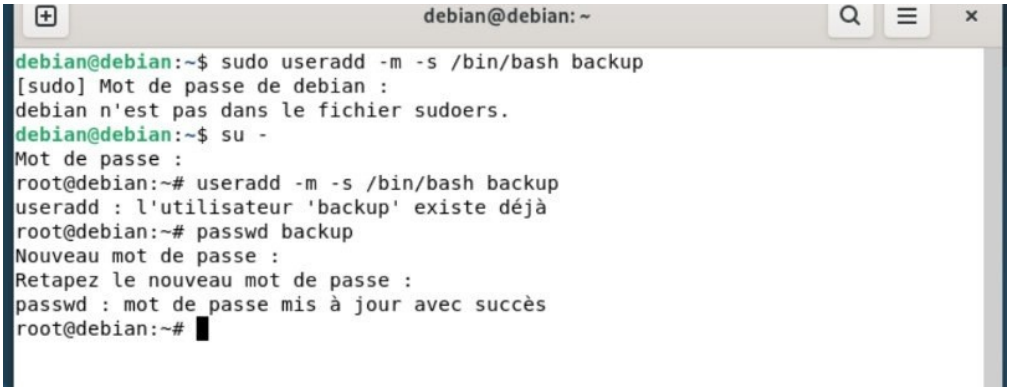


Figure 74 : *Création de l'utilisateur backup sur Debian Client*

Créer l'arborescence de stockage des sauvegardes :

Sur Debian Client
```
sudo mkdir -p /srv/backups/{debian_serveur,configs,archives}
sudo chown -R backup:backup /srv/backups/
sudo chmod 750 /srv/backups/
ls -la /srv/backups/
```

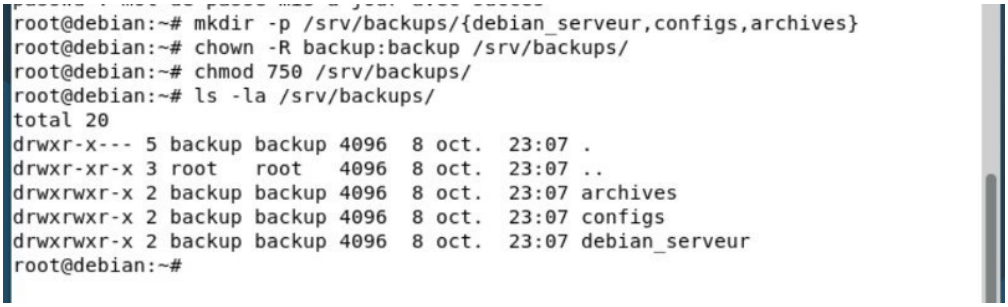


Figure 75 : *Arborescence /srv/backups appartenant à l'utilisateur backup*

Les trois sous-dossiers ont un rôle précis : `configs` reçoit les archives des fichiers de configuration, `debian_serveur` reçoit la copie des données métier (`/srv/sentech/`), et `archives` est réservé aux sauvegardes complètes futures. Le dossier est en `750` : lisible par le propriétaire et son groupe, fermé aux autres.

Authentification SSH entre Debian Serveur et Debian Client

Pour que les sauvegardes automatiques fonctionnent sans intervention humaine, Debian Serveur doit pouvoir se connecter à Debian Client sans mot de passe. On crée donc une clé dédiée à cet usage, sur Debian Serveur, **en root** (car c'est root que cron exécutera les scripts).

Sur Debian Serveur
```
sudo ssh-keygen -t ed25519 -C "backup@sentech" -f /root/.ssh/sentech_backup_key -N ""
sudo ls -la /root/.ssh/sentech_backup_key*
```

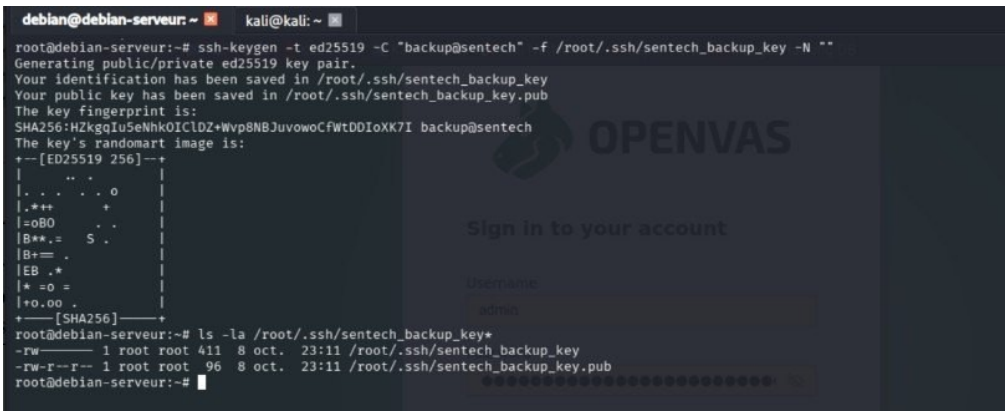


Figure 76 : *Clé sentech_backup_key et sentech_backup_key.pub*

POUR COMPRENDRE **Pourquoi -N "" ici, alors que l'on imposait une passphrase pour la clé de Kali ?**

La clé `sentech_ed25519` sert à un **humain** qui se connecte manuellement ; elle porte donc une passphrase comme couche de défense supplémentaire. La clé `sentech_backup_key` est utilisée par un **script automatique** exécuté par cron, sans personne derrière le clavier pour saisir une passphrase : elle doit donc en être dépourvue. En contrepartie, cette clé est strictement cantonnée à un usage précis (l'utilisateur `backup` sur Debian Client uniquement) : c'est un compromis de sécurité standard et documenté pour l'automatisation. Pour aller plus loin, on peut restreindre la clé côté Debian Client en préfixant sa ligne dans `authorized_keys` par `from="192.168.10.10"`, de sorte qu'elle ne fonctionne que depuis le serveur.

Sur Debian Serveur

```
sudo ssh-copy-id -i /root/.ssh/sentech_backup_key.pub -p 22 backup@192.168.10.11
# Entrer le mot de passe de « backup » une dernière fois

sudo ssh -i /root/.ssh/sentech_backup_key backup@192.168.10.11 "echo 'Connexion SSH backup
OK'"
```



Figure 77 : *La connexion par clé vers Debian Client fonctionne*

Debian Client n'a pas de SSH durci sur un port non standard : contrairement à Debian Serveur (port 2222), il écoute SSH sur le port 22 standard, d'où l'option `-p 22`.

Sauvegarde des fichiers de configuration critiques

Ce script, qui n'apparaissait pas dans la première version du dossier, archive les fichiers de configuration que nous avons modifiés, calcule leur empreinte SHA256, envoie le tout à Debian Client puis **vérifie** que la copie reçue est identique à l'original.

Sur Debian Serveur

```
sudo mkdir -p /var/backup/sentech /var/log/sentech
sudo nano /usr/local/bin/sentech-backup-configs.sh
```

/usr/local/bin/sentech-backup-configs.sh

```
#!/bin/bash
# Sauvegarde des fichiers de configuration critiques - SenTech Solutions
set -euo pipefail

DATE=$(date +%Y%m%d_%H%M%S)
LOCAL_DIR="/var/backup/sentech"
ARCHIVE="configs_${DATE}.tar.gz"
REMOTE_USER="backup"
REMOTE_HOST="192.168.10.11"
REMOTE_DIR="/srv/backups/configs"
SSH_KEY="/root/.ssh/sentech_backup_key"
SSH="ssh -i ${SSH_KEY}"

mkdir -p "$LOCAL_DIR"

echo "[$(date)] Demarrage de la sauvegarde des configurations..."

# 1. Archive des fichiers de configuration
tar czf "${LOCAL_DIR}/${ARCHIVE}" \
    /etc/ssh/sshd_config \
    /etc/fail2ban/jail.local \
    /etc/fail2ban/filter.d/sentech-custom.conf \
    /etc/audit/rules.d/sentech.rules \
    /etc/iptables/rules.v4 /etc/iptables/rules.v6 \
    /etc/sysctl.d/99-sentech-security.conf \
    /etc/security/pwquality.conf \
    /etc/pam.d/common-auth /etc/pam.d/common-account \
    /etc/sudoers /etc/sudoers.d \
    /etc/issue.net /etc/network/interfaces
chmod 600 "${LOCAL_DIR}/${ARCHIVE}"
echo "[$(date)] Archive creee : ${ARCHIVE}"

# 2. Empreinte SHA256 de l'archive
LOCAL_SUM=$(sha256sum "${LOCAL_DIR}/${ARCHIVE}" | cut -d' ' -f1)
echo "${LOCAL_SUM}  ${ARCHIVE}" > "${LOCAL_DIR}/${ARCHIVE%.tar.gz}.sha256"
echo "[$(date)] Empreinte SHA256 : ${LOCAL_SUM}"

# 3. Transfert vers Debian Client (archive + empreinte)
scp -i "${SSH_KEY}" "${LOCAL_DIR}/${ARCHIVE}" "${LOCAL_DIR}/${ARCHIVE%.tar.gz}.sha256" \
    "${REMOTE_USER}@${REMOTE_HOST}:${REMOTE_DIR}/"
echo "[$(date)] Transfert vers ${REMOTE_HOST} termine."

# 4. Vérification d'intégrité : l'empreinte de la copie distante doit être identique
REMOTE_SUM=$($SSH "${REMOTE_USER}@${REMOTE_HOST}" "sha256sum ${REMOTE_DIR}/${ARCHIVE}" | cut
-d' ' -f1)
if [ "${LOCAL_SUM}" = "${REMOTE_SUM}" ]; then
    echo "[$(date)] Verification d'integrite : OK (SHA256 identique)"
else
    echo "[$(date)] ERREUR : empreintes differentes, sauvegarde corrompue !"
    exit 1
fi

# 5. Nettoyage : ne garder que 7 jours d'archives en local
find "${LOCAL_DIR}" -name 'configs_*' -mtime +7 -delete
echo "[$(date)] Sauvegarde terminee avec succes."
```

Le rendre exécutable et le lancer :

Sur Debian Serveur

```
sudo chmod +x /usr/local/bin/sentech-backup-configs.sh
sudo /usr/local/bin/sentech-backup-configs.sh
```



Figure 78 : *Exécution du script : archive créée, empreinte calculée, transfert, vérification d'intégrité « OK »*

Sauvegarde des données métier avec rsync

`rsync` ne copie que ce qui a changé depuis la dernière fois, ce qui le rend rapide et économe en réseau. Le script ci-dessous synchronise `/srv/sentech/` vers Debian Client.

Sur Debian Serveur (en root)

```
sudo apt install rsync -y
sudo nano /usr/local/bin/sentech-backup-data.sh
```

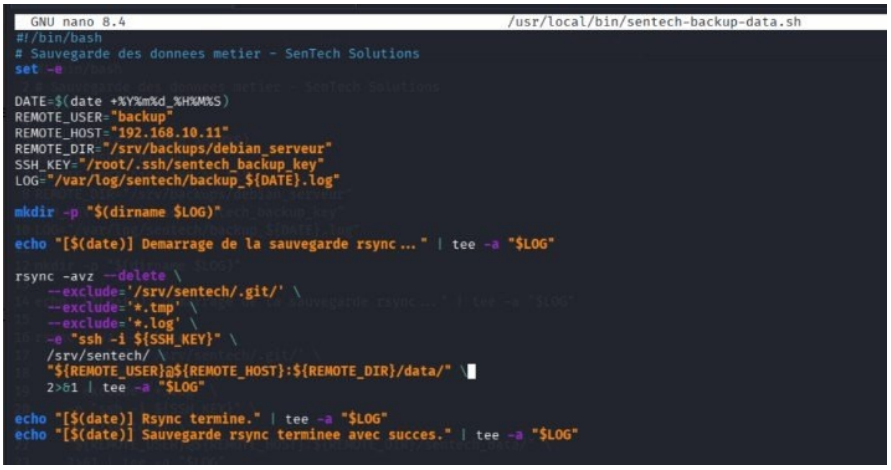


Figure 79 : *Le script dans l'éditeur nano*

/usr/local/bin/sentech-backup-data.sh

```
#!/bin/bash
# Sauvegarde des donnees metier - SenTech Solutions
set -euo pipefail

DATE=$(date +%Y%m%d_%H%M%S)
REMOTE_USER="backup"
REMOTE_HOST="192.168.10.11"
REMOTE_DIR="/srv/backups/debian_serveur"
SSH_KEY="/root/.ssh/sentech_backup_key"
LOG="/var/log/sentech/backup_${DATE}.log"

mkdir -p "$(dirname "$LOG")"

echo "[$(date)] Demarrage de la sauvegarde rsync..." | tee -a "$LOG"

rsync -avz --delete \
    --exclude='/srv/sentech/.git/' \
    --exclude='*.tmp' \
    --exclude='*.log' \
    -e "ssh -i ${SSH_KEY}" \
    /srv/sentech/ \
    "${REMOTE_USER}@${REMOTE_HOST}:${REMOTE_DIR}/data/" \
    2>&1 | tee -a "$LOG"

echo "[$(date)] Rsync termine." | tee -a "$LOG"
echo "[$(date)] Sauvegarde rsync terminee avec succes." | tee -a "$LOG"
```

POUR COMPRENDRE **Ce que fait rsync -avz --delete**

`-a` copie en conservant permissions, dates et propriétaires ; `-v` affiche ce qui est copié ; `-z` compresse pendant le transfert ; `--delete` supprime à destination les fichiers qui ont été supprimés à la source, de sorte que la copie reste un reflet exact. **Attention :** `--delete` signifie aussi qu'une suppression accidentelle à la source est répercutée à la prochaine sauvegarde ; c'est pourquoi on garde en plus des archives datées (script précédent).

Rendre exécutable, préparer le dossier distant, puis lancer :

Sur Debian Client

```
mkdir -p /srv/backups/debian_serveur/data
```

```
sudo chown -R backup:backup /srv/backups/
sudo chmod 750 /srv/backups/
ls -la /srv/backups/debian_serveur/
```

```
Sur Debian Serveur
# Vérifier qu'il y a des données à sauvegarder
ls -la /srv/sentech/

sudo chmod +x /usr/local/bin/sentech-backup-data.sh
sudo /usr/local/bin/sentech-backup-data.sh
```

ATTENTION **Erreur détectée : un script qui annonce « succès » alors qu'il a échoué**

Lors du premier essai, `rsync` n'était pas installé et le terminal affichait « rsync : commande introuvable ». Pourtant le script a écrit « Sauvegarde rsync terminee avec succes » : en effet, dans un enchaînement `commande | tee`, c'est le code de retour de `tee` (toujours « réussi ») qui est vu par le script. Deux corrections ont été apportées : l'installation de `rsync` sur **les deux machines**, et la ligne `set -euo pipefail` en tête de script, qui arrête le script dès qu'une commande de l'enchaînement échoue. **Une sauvegarde qui échoue en silence est plus dangereuse qu'une sauvegarde absente**, car on croit être protégé.

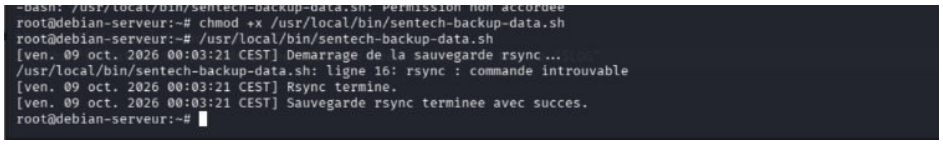


Figure 80 : *Premier essai : « rsync : commande introuvable » alors que le message final annonçait un succès*

Automatisation des sauvegardes

Étape 1 : éditer la crontab de root.

```
Sur Debian Serveur (en root)
crontab -e
```

Si c'est la première fois, il demande de choisir un éditeur : taper `2` (nano).

Étape 2 : ajouter ces lignes.

```
crontab de root
# Sauvegardes automatiques SenTech Solutions

# Sauvegarde des configurations : tous les jours a 01h00
0 1 * * * /usr/local/bin/sentech-backup-configs.sh >> /var/log/sentech/cron_backup.log 2>&1

# Sauvegarde des donnees (rsync) : tous les jours a 02h00
0 2 * * * /usr/local/bin/sentech-backup-data.sh >> /var/log/sentech/cron_backup.log 2>&1

# Rapport hebdomadaire : tous les lundis a 08h00
0 8 * * 1 ls -lah /var/backup/sentech/ >> /var/log/sentech/cron_backup.log 2>&1
```

POUR COMPRENDRE **lire une ligne cron**

Les cinq premiers champs sont : **minute, heure, jour du mois, mois, jour de la semaine**. `0 1 * * *` signifie « à 01h00, tous les jours ». `0 8 * * 1` signifie « à 08h00, le lundi ». La fin de la ligne (`>> fichier 2>&1`) ajoute à un journal à la fois les messages normaux et les erreurs.

Étape 3 : vérifier.

```
Sur Debian Serveur
crontab -l
```

Étape 4 : créer le dossier de journal (si ce n'est pas déjà fait).

Sur Debian Serveur
```
mkdir -p /var/log/sentech
ls -la /var/log/sentech/
```

Étape 5 : tester manuellement les deux scripts.

Sur Debian Serveur
```
/usr/local/bin/sentech-backup-configs.sh
/usr/local/bin/sentech-backup-data.sh
```

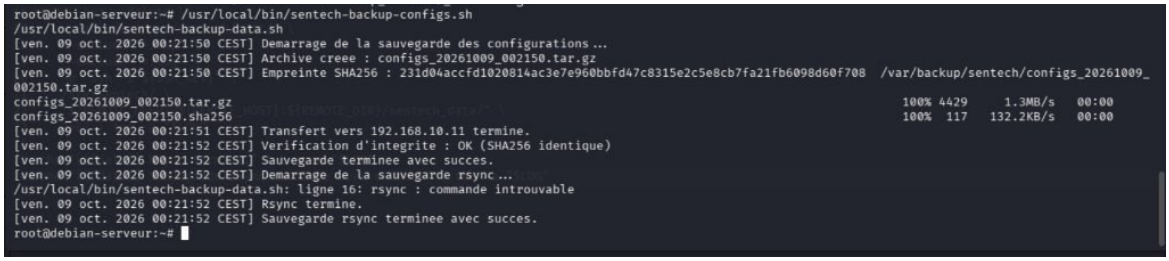


Figure 81 : *Test manuel des deux scripts*

Simulation d'incident et restauration

L'incident consiste à casser SSH sur Debian Serveur lui-même. Nous nous mettons à la place d'un administrateur qui a fait une erreur et cassé la configuration SSH. Puis nous testons si la sauvegarde permet de récupérer. C'est Debian Serveur qui va :

1. déplacer son propre fichier `/etc/ssh/sshd_config` ;
2. redémarrer son propre service SSH (qui va échouer) ;
3. se retrouver inaccessible en SSH depuis l'extérieur.

La restauration se fait donc **depuis la console de virt-manager**, qui ne dépend pas du réseau.

Phase 1 : vérifier la sauvegarde la plus récente.

Sur Debian Serveur (en root)
```
/usr/local/bin/sentech-backup-configs.sh
ssh -i /root/.ssh/sentech_backup_key backup@192.168.10.11 "ls -lah /srv/backups/configs/"
```

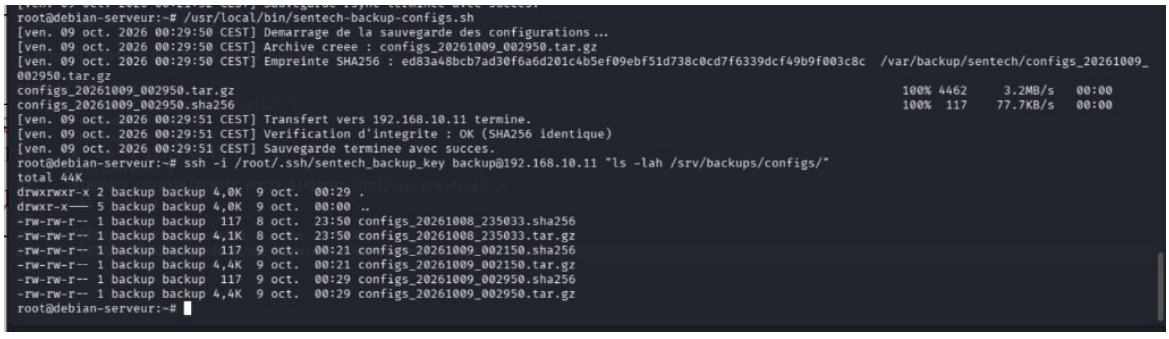


Figure 82 : *Les archives de configuration sont présentes sur Debian Client, chacune avec son fichier .sha256*

On note le nom du dernier fichier dans une variable pour l'utiliser ensuite automatiquement :

Sur Debian Serveur
```
LATEST_BACKUP=$(ssh -i /root/.ssh/sentech_backup_key backup@192.168.10.11 "ls -t
/srv/backups/configs/configs_*.tar.gz | head -1")
echo "Derniere sauvegarde : $LATEST_BACKUP"
```

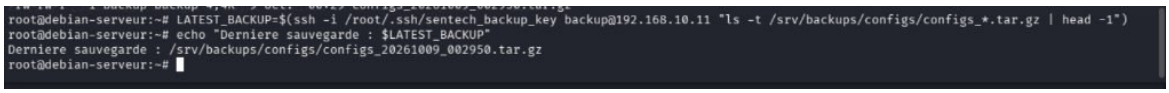


Figure 83 : *Nom de la dernière sauvegarde*

Phase 2 : simuler l'incident.

> **ATTENTION** **Avant de continuer**
> Gardez la console virt-manager ouverte : c'est votre filet de sécurité si SSH plante.

Sur Debian Serveur

```
# Copie de sécurité de la bonne configuration
cp /etc/ssh/sshd_config /tmp/sshd_config_fonctionnel
# Sabotage : on déplace la configuration
mv /etc/ssh/sshd_config /etc/ssh/sshd_config.CORRUPT
systemctl restart ssh 2>&1 || echo "INCIDENT : SSH ne redemarre plus !"
```



Figure 84 : *Le service ssh ne redémarre plus : l'incident est bien simulé*

Les sessions SSH déjà ouvertes restent actives, mais plus aucune nouvelle connexion n'est possible.

Phase 3 : restaurer depuis la sauvegarde. Si la variable `LATEST_BACKUP` n'est plus définie (terminal refermé entre-temps), on la recalcule avec la commande de la phase 1, puis on rapatrie l'archive :

Sur Debian Serveur (console virt-manager)

```
scp -i /root/.ssh/sentech_backup_key "backup@192.168.10.11:${LATEST_BACKUP}"
/tmp/restauration.tar.gz
ls -la /tmp/restauration.tar.gz
```

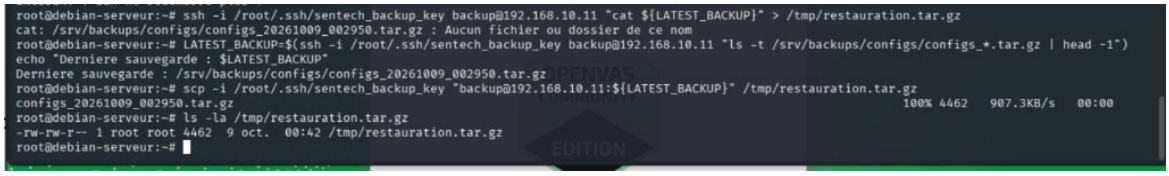


Figure 85 : *Archive rapatriée dans /tmp*

Phase 4 : extraire et restaurer. Par sécurité, `tar` retire le « / » initial des chemins en créant l'archive : le fichier est donc enregistré sous `etc/ssh/sshd_config`. Extraire depuis `/tmp` le déposerait dans `/tmp/etc/...`. Il faut extraire **à la racine**, avec l'option `-C /` :

Sur Debian Serveur

```
tar xzf /tmp/restauration.tar.gz -C /
# Vérifier que sshd_config est restauré
ls -la /etc/ssh/sshd_config
grep -E "Port|PermitRoot|PasswordAuth" /etc/ssh/sshd_config
# Supprimer le fichier corrompu
rm -f /etc/ssh/sshd_config.CORRUPT
```

> **REMARQUE** **Restauration ciblée**
> Cette extraction remet **tous** les fichiers de l'archive à l'état de la sauvegarde. Pour ne restaurer qu'un seul fichier : `tar xzf /tmp/restauration.tar.gz -C / etc/ssh/sshd_config`.



Figure 86 : *sshd_config est restauré avec Port 2222, PermitRootLogin no et PasswordAuthentication no*

Phase 5 : redémarrer SSH.

Sur Debian Serveur

```
systemctl restart ssh
systemctl is-active ssh
ss -tlnp | grep ssh
```

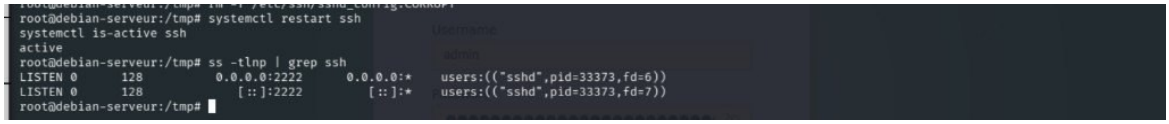


Figure 87 : *Le service est de nouveau « active » et écoute sur le port 2222*

Phase 6 : tester depuis Kali.

Sur Kali Linux

```
ssh -p 2222 -i ~/.ssh/sentech_ed25519 debian@192.168.10.10 "echo 'Connexion post-restauration
OK'"
```



Figure 88 : *Connexion post-restauration réussie*

La restauration a rétabli le service en quelques minutes, ce qui valide la procédure.

Test d'intégrité avec AIDE après restauration

AIDE a mémorisé l'état des fichiers (Correction 4 de la Partie 1). Après une restauration, des fichiers ont changé (par exemple `sshd_config`) : AIDE doit donc les signaler. Si ces modifications sont légitimes, on met à jour la base de référence.

Sur Debian Serveur (en root)

```
# 1. Vérifier l'intégrité : AIDE liste les fichiers modifiés
aide --check --config=/etc/aide/aide.conf

# 2. Les changements étant légitimes (restauration), mettre à jour la base
aide --update --config=/etc/aide/aide.conf
cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db

# 3. Vérifier la présence des fichiers
ls -la /var/lib/aide/
ls -la /etc/aide/
```

REMARQUE

Il n'est pas nécessaire de relancer `aideinit` : cette commande recrée la base de zéro et ferait disparaître l'historique de comparaison. On l'utilise uniquement à l'installation initiale.



Figure 89 : *Base AIDE mise à jour (empreintes SHA256, SHA512 et SHA3 de aide.db)*

Partie 5 : Rédaction du rapport de remédiation

Le rapport de remédiation est le livrable final de notre mission chez SenTech Solutions. Il doit être compréhensible par des lecteurs non techniques (direction, service juridique) tout en restant assez précis pour guider les équipes techniques. La première étape est de collecter les mesures dans un fichier unique.

Collecte des données pour le rapport

Sur Debian Serveur (en root)

```
mkdir -p /var/log/sentech/rapport_final
M=/var/log/sentech/rapport_final/metriques.txt
{
  echo "=== SCORES LYNIS ==="
  grep "Hardening index" /var/log/sentech/audits/lynis_*.log
  echo ""
  echo "=== SERVICES ACTIFS ==="
  systemctl list-units --type=service --state=active --no-pager
  echo ""
  echo "=== PORTS EN ECOUTE ==="
  ss -tlnp
  echo ""
  echo "=== REGLES IPTABLES ==="
  iptables -L -n -v
  echo ""
  echo "=== SAUVEGARDES ==="
  ls -lah /var/backup/sentech/
} > "$M"
cat "$M"
```

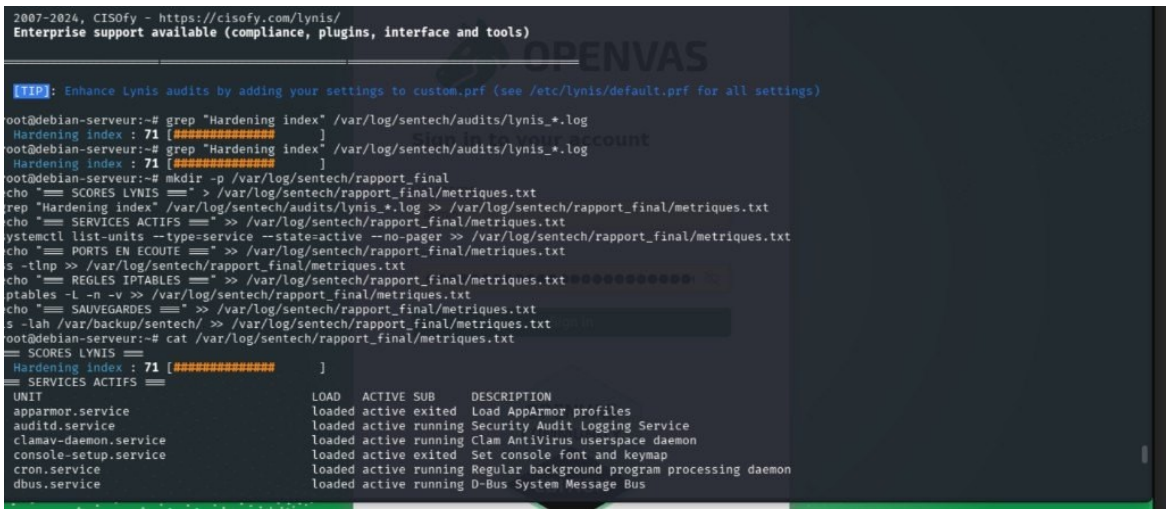


Figure 90 : *Fichier metriques.txt : scores Lynis, services actifs, ports, règles et sauvegardes*

Synthèse destinée à la direction

Constat de départ. Un serveur Debian installé avec ses réglages par défaut : toutes les connexions entrantes acceptées, accès SSH par mot de passe sur le port 22, aucune surveillance des actions sensibles, aucune sauvegarde. Les mesures « avant » du tableau ci-dessous sont celles relevées au début de la Partie V (le SSH par clés était alors déjà en place).

Résultats mesurés :

Indicateur	Avant	Après
Ports TCP vus par Nmap (scans des 6 et 7 octobre)	999 ports « closed » : la machine répond par un RST	999 ports « filtered » : aucune réponse ; seul le port SSH 2222 est ouvert
Durée d'un scan Nmap	1,29 s	19,51 s
Authentification SSH	Mot de passe, port 22	Clé uniquement, port 2222, root interdit
Attaque par force brute SSH	Aucune limite	Bannissement automatique après 3 échecs
Hardening index Lynis	Non mesuré avant durcissement	72 puis 73 sur 100 (« Bon »)
Vulnérabilités OpenVAS	Non mesuré avant durcissement	12 résultats, tous de niveau faible (2,6 au maximum)
Sauvegarde	Aucune	Quotidienne, vérifiée par SHA256, restauration testée

Actions réalisées : segmentation du réseau derrière OPNsense ; comptes, groupes, permissions et ACL selon le moindre privilège ; SSH par clés ; pare-feu iptables à politique DROP ; Fail2Ban ; Auditd (règles de surveillance) ; politique de mots de passe et verrouillage PAM ; sudo journalisé ; durcissement du noyau ; synchronisation horaire ; AIDE ; sauvegardes automatiques.

Risques résiduels et recommandations, par ordre de priorité :

N°	Risque ou point à traiter	Action recommandée
1	Pare-feu non actif lors du scan final (ports « closed »)	Vérifier la politique DROP et la persistance des règles (voir l'encadré de la Partie 3)
2	Algorithmes MAC faibles proposés par SSH	Restreindre la directive MACs dans sshd_config
3	Réponses ICMP timestamp et horodatages TCP	Ajouter la règle iptables et le paramètre sysctl indiqués
4	Une seule copie distante des sauvegardes (règle 3-2-1 incomplète)	Ajouter une copie hors site ou sur un second support
5	Clé de sauvegarde sans passphrase	Restreindre la clé à l'adresse du serveur (from=) et surveiller son usage

N°	Risque ou point à traiter	Action recommandée
6	Audit non automatisé	Planifier Lynis et le script rapport_audit.sh chaque semaine

PARTIE VII

Partie VII : Pilotage de ClamAV via l'interface graphique ClamUI

Présentation de ClamUI

ClamAV ne dispose pas d'interface graphique officielle. **ClamUI** est un projet open source construit avec GTK4 et libadwaita, qui s'intègre nativement aux environnements de bureau modernes.

Principe fondamental : ClamUI n'est pas un nouveau moteur antivirus. C'est une surcouche qui pilote les outils ClamAV existants (`clamscan`, `clamdscan`, `freshclam`, `clamonacc`) à votre place et présente les résultats dans une interface lisible. Tout ce que l'on ferait en ligne de commande, ClamUI le fait avec des boutons, des cases à cocher et des journaux affichés à l'écran.

Outil	Rôle
clamscan	Analyse un fichier ou un dossier à la demande (charge toute la base à chaque lancement)
clamd / clamdscan	Moteur résident en mémoire (démon) et son client, plus rapides pour des analyses répétées
freshclam	Met à jour la base de signatures de virus
clamonacc	Analyse « à l'accès » : vérifie un fichier au moment où il est ouvert (protection en temps réel)

Avant ClamUI, l'unique alternative graphique était ClamTk, une interface historique en Perl, simple mais vieillissante et limitée en fonctionnalités. ClamUI couvre un périmètre beaucoup plus large.

Étape 0 : Installation de ClamAV (sur Debian Client)

REMARQUE **Droits administrateur sur Debian Client**

Sur Debian Client, le compte `debian` n'est pas déclaré dans le fichier sudoers (la commande `sudo` répond « debian n'est pas dans le fichier sudoers », visible sur certaines captures). Les commandes marquées `sudo` ci-dessous s'exécutent donc **en root**, après `su -`, en omettant le mot `sudo`. Il ne faut pas confondre avec Debian Serveur, où sudo est configuré (Partie IV et V).

Sur Debian Client

```
sudo apt update
sudo apt install clamav clamav-daemon clamav-freshclam -y
sudo systemctl stop clamav-freshclam
sudo freshclam
sudo systemctl start clamav-freshclam
sudo systemctl enable clamav-freshclam
sudo systemctl status clamav-daemon
```

POUR COMPRENDRE **Pourquoi arrêter freshclam avant de l'exécuter à la main ?**

Le service `clamav-freshclam` met déjà à jour la base en arrière-plan. Deux mises à jour simultanées se gêneraient (verrou sur la base) : on arrête donc le service, on force une mise à jour manuelle pour partir d'une base complète, puis on redémarre le service et on l'active au démarrage.

Étape 1 : Installation de ClamUI

ClamUI existe sous deux formats : Flatpak et paquet `.deb`.

Choix du format. La version Flatpak présente des limitations connues sur les opérations nécessitant une élévation de privilèges (configuration du *On-Access Scanning*, la protection en temps réel). Pour ce

TP, qui couvre la protection en temps réel, le paquet `.deb` est donc recommandé ; c'est lui que nous utilisons.

Méthode recommandée : paquet .deb. Vérifier les prérequis système (déjà présents sur un Debian 13 à jour) :

Sur Debian Client
```
sudo apt install -y libgtk-4-1 libadwaita-1-0
python3 --version
```

Télécharger la dernière version des **deux** paquets `.deb` (l'application et son assistant de privilèges) depuis la page officielle des versions du projet, `https://github.com/linx-systems/clamui/releases`, puis les installer ensemble. Les numéros de version ci-dessous sont à adapter à la version publiée.

Sur Debian Client
```
cd ~/Téléchargements
wget https://github.com/linx-systems/clamui/releases/download/v0.4.0/clamui_0.4.0_all.deb
wget https://github.com/linx-systems/clamui/releases/download/v0.4.0/clamui-privileged-
helper_0.4.0_all.deb
sudo apt install -y ./clamui_0.4.0_all.deb ./clamui-privileged-helper_0.4.0_all.deb
ls -la /usr/bin/clamui /usr/bin/clamui-apply-preferences
clamui
```

Étape 2 : Découverte de l'interface et premier scan

1. Observer les profils d'analyse proposés : Dossier personnel (Home), Téléchargements, Système complet.
2. Lancer une analyse du dossier personnel (Home).
3. Observer que l'analyse est asynchrone : l'interface reste utilisable pendant le scan.
4. Consulter le rapport une fois terminé.

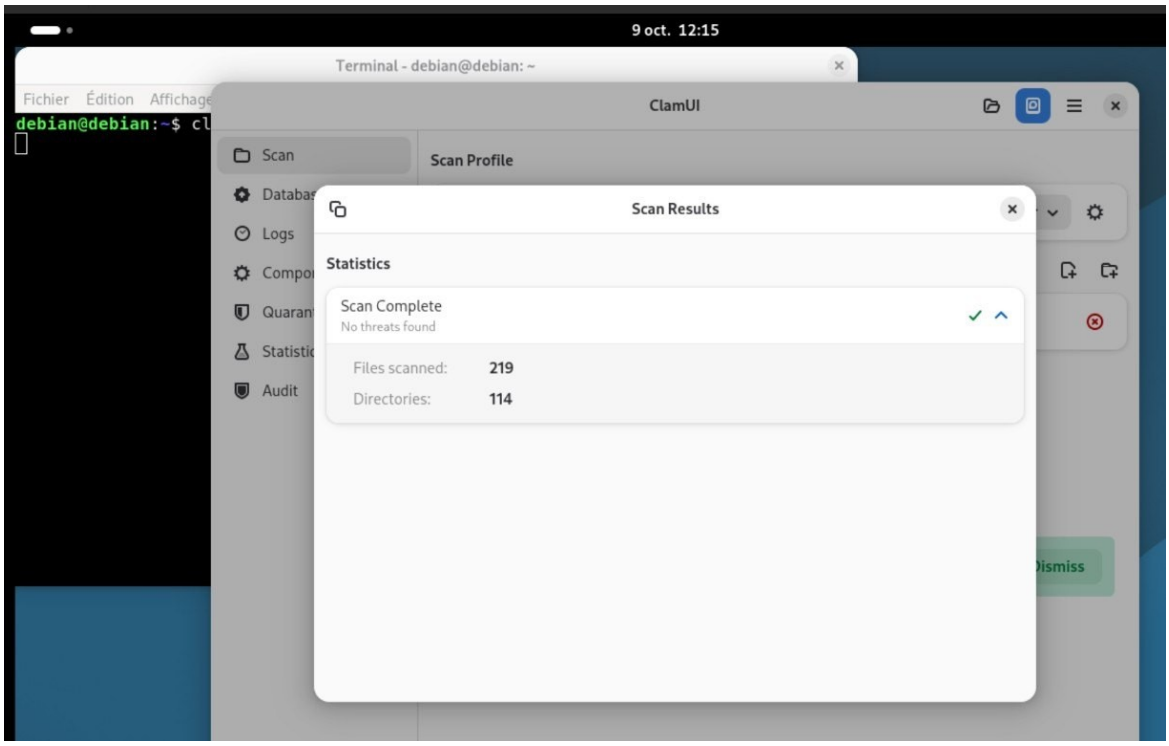


Figure 91 : *Premier scan terminé : 219 fichiers et 114 dossiers analysés, aucune menace*

Étape 3 : Test avec un fichier EICAR

Le fichier **EICAR** est une courte chaîne de texte standardisée, **totalement inoffensive**, que tous les antivirus reconnaissent volontairement comme un virus. Elle permet de vérifier qu'un antivirus fonctionne sans utiliser un vrai programme malveillant.

Sur Debian Client

```
EICAR='X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*'
echo "$EICAR" > ~/eicar.txt
ls -la ~/eicar.txt
cat ~/eicar.txt
```

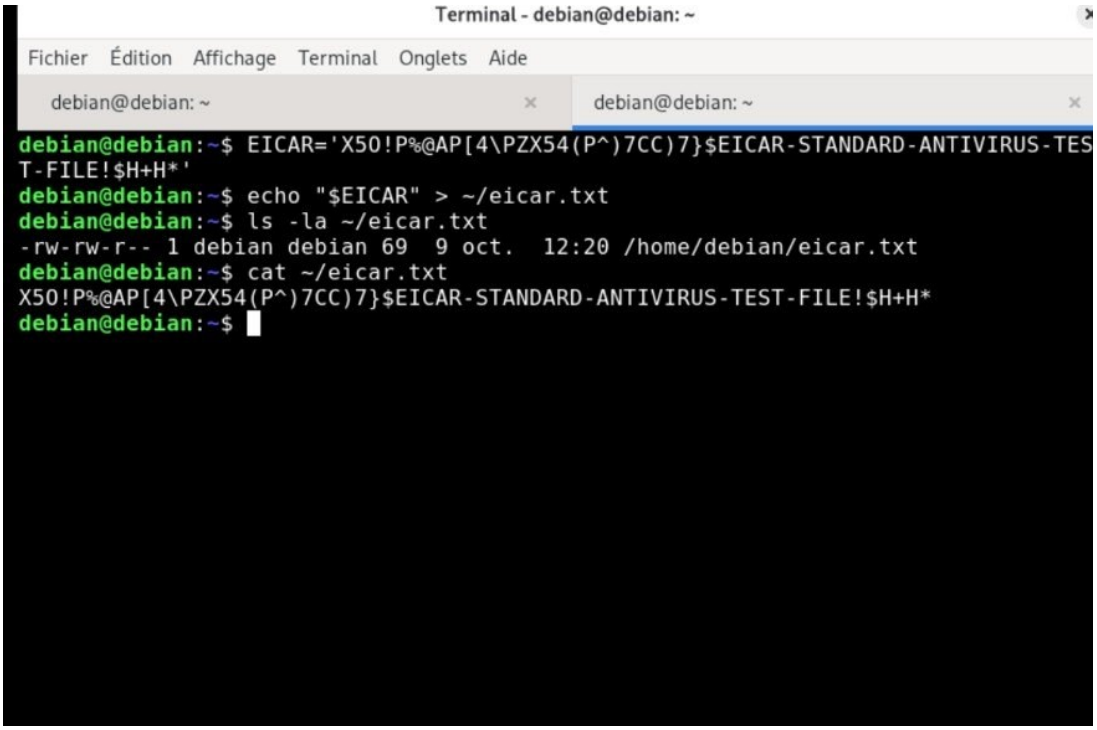


Figure 92 : *Création du fichier de test ~/eicar.txt (69 octets)*

Étape 4 : Relancer le scan

1. Cliquer sur **Dismiss** (fermer le message vert du scan précédent).
2. Cliquer sur **Start Scan**.
3. Cette fois, ClamUI analyse `/home/debian/` et trouve `eicar.txt`.
4. ClamUI affiche la détection `Eicar-Signature` pour `/home/debian/eicar.txt`.
5. ClamUI propose trois actions : **Quarantine** (mettre en quarantaine), **Exclude** (exclure du scan) ou **Copy Path** (copier le chemin).
6. Choisir **Quarantine**.

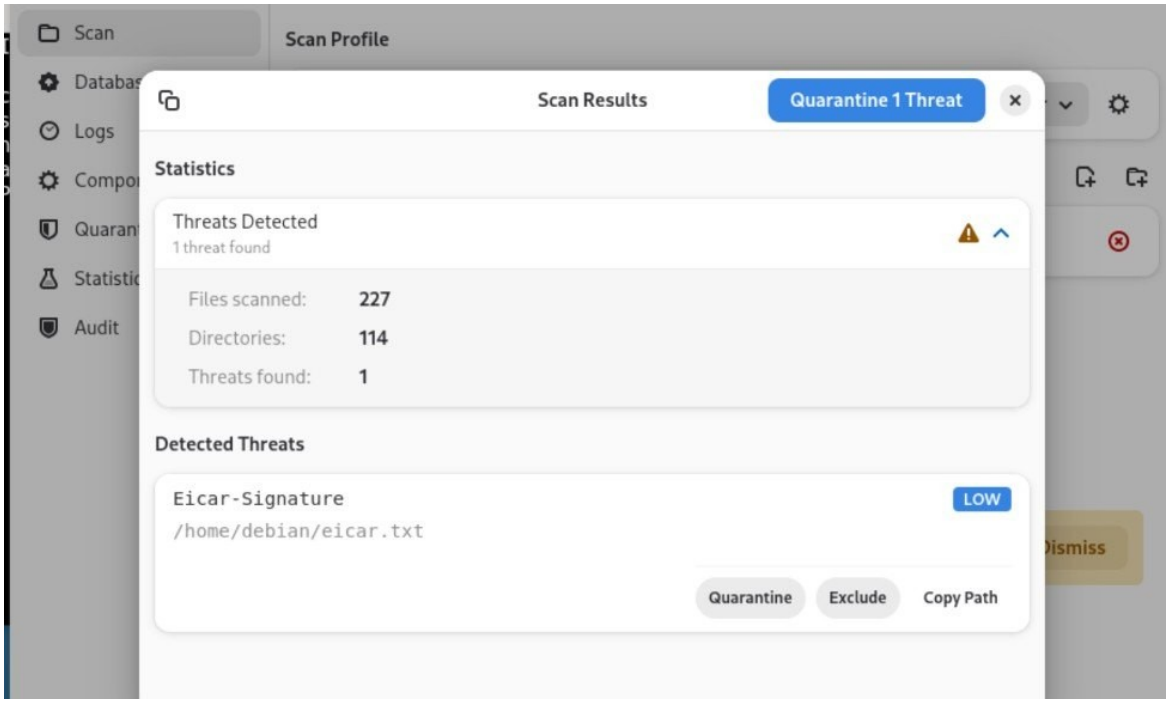


Figure 93 : *Menace détectée : Eicar-Signature dans /home/debian/eicar.txt, avec le bouton Quarantine*

REMARQUE **le bouton « EICAR Test »**

ClamUI possède son propre bouton **EICAR Test**, qui crée automatiquement un fichier de test (nommé `test_eicar.txt`) pour vérifier la chaîne de détection en un clic.

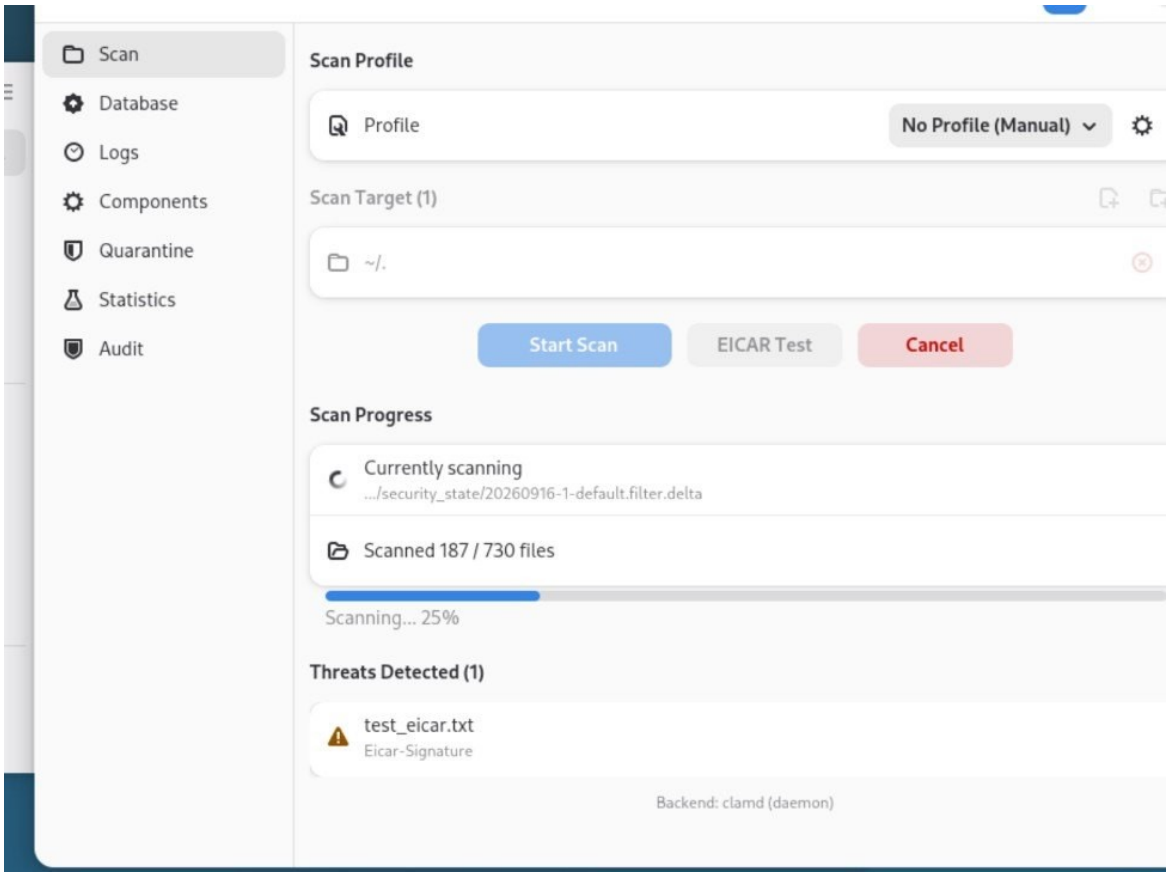


Figure 94 : *Analyse du dossier personnel en cours (25 %) : test_eicar.txt est déjà signalé comme menace*

Étape 5 : Vérifier la quarantaine

1. Dans le menu de gauche de ClamUI, cliquer sur **Quarantine**.
2. La ligne `eicar.txt` apparaît avec son état « Quarantined ».
3. Cliquer dessus pour voir les détails.
4. Deux actions sont possibles : **Restore** (restaurer le fichier) ou **Delete** (supprimer définitivement).

La **quarantaine** isole le fichier suspect sans le détruire : on peut ainsi le restaurer en cas de faux positif (alerte à tort) ou l'étudier avant suppression.

Étape 6 : Protection en temps réel (On-Access)

Jusqu'ici, il fallait lancer une analyse. La protection en temps réel, assurée par `clamonacc`, vérifie un fichier **au moment où il est ouvert** et réagit tout de suite. Dans ClamUI, elle se règle dans **Preferences > On-Access** ; c'est l'assistant de privilèges installé avec le paquet `.deb` qui écrit pour nous les directives `OnAccess…` dans le fichier de configuration de `clamd` (`/etc/clamav/clamd.conf`).

Pour tester, on ouvre (par exemple avec `cat`) le fichier EICAR situé dans un dossier surveillé : la détection doit être immédiate, sans lancer d'analyse. On vérifie que le service tourne avec :

Sur Debian Client
```
ps aux | grep clamonacc | grep -v grep
```

REMARQUE
`clamonacc` ne surveille que les dossiers déclarés dans la configuration : c'est la raison pour laquelle le disque monté sous `/media` (étape suivante) n'est pas couvert, et pourquoi nous utilisons un script de montage dédié.

Étape 7 : Analyse automatique des périphériques USB

Objectif. Simuler le branchement d'une clé USB et observer l'analyse automatique de ClamUI.

Méthode recommandée : second disque virtuel. Sur l'hôte Ubuntu, dans virt-manager :

1. Éteindre Debian Client.
2. Ouvrir virt-manager, puis Debian Client > Détails > Ajouter du matériel > Stockage, avec : taille 1 Go, format qcow2, bus USB (ou VirtIO si USB est indisponible).
3. Démarrer Debian Client.

Formater le disque USB (sur Debian Client)

1. Identifier le disque (dans un terminal) :

Sur Debian Client
```
lsblk
```

Un nouveau disque apparaît, par exemple `sda` (1 Go, sans partition).

2. Formater (vérifiez bien le nom du disque : `mkfs` efface tout ce que contient la cible).

Sur Debian Client
```
sudo mkfs.ext4 /dev/sda
```

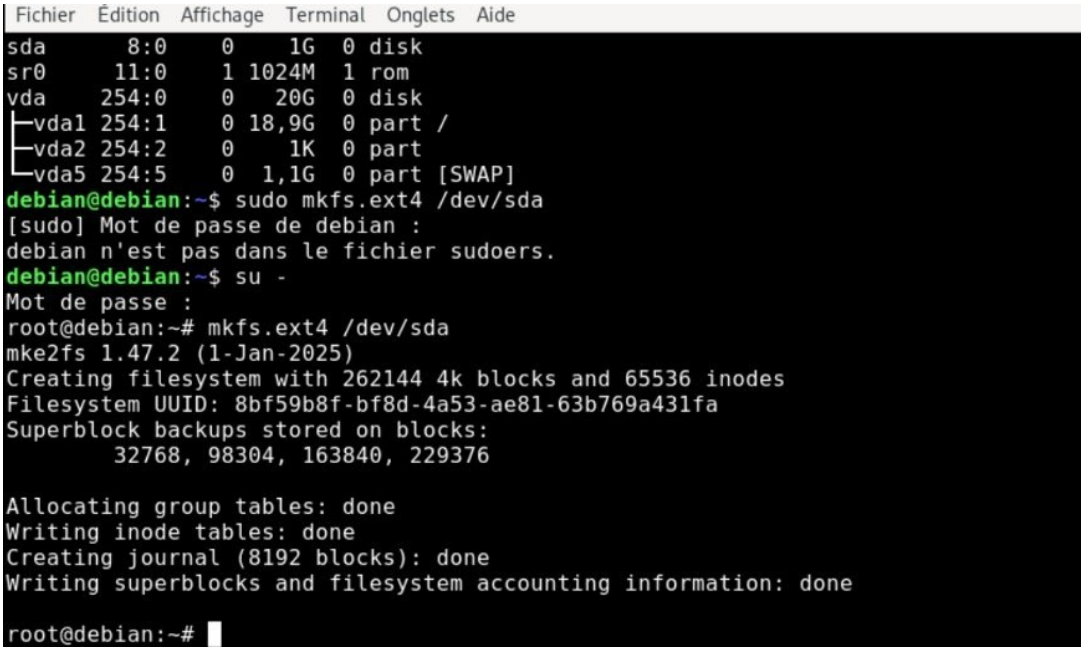


Figure 95 : *Disque sda de 1 Go ; la commande sudo est refusée pour l'utilisateur debian, le formatage est donc fait en root après « su - »*

3. Créer un point de montage et monter le disque :

Sur Debian Client

```
sudo mkdir -p /media/usb
sudo mount /dev/sda /media/usb
```

4. Copier un nouveau fichier EICAR sur le disque :

Sur Debian Client

```
EICAR='X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*'
echo "$EICAR" | sudo tee /media/usb/eicar_usb.txt
ls -la /media/usb/
```

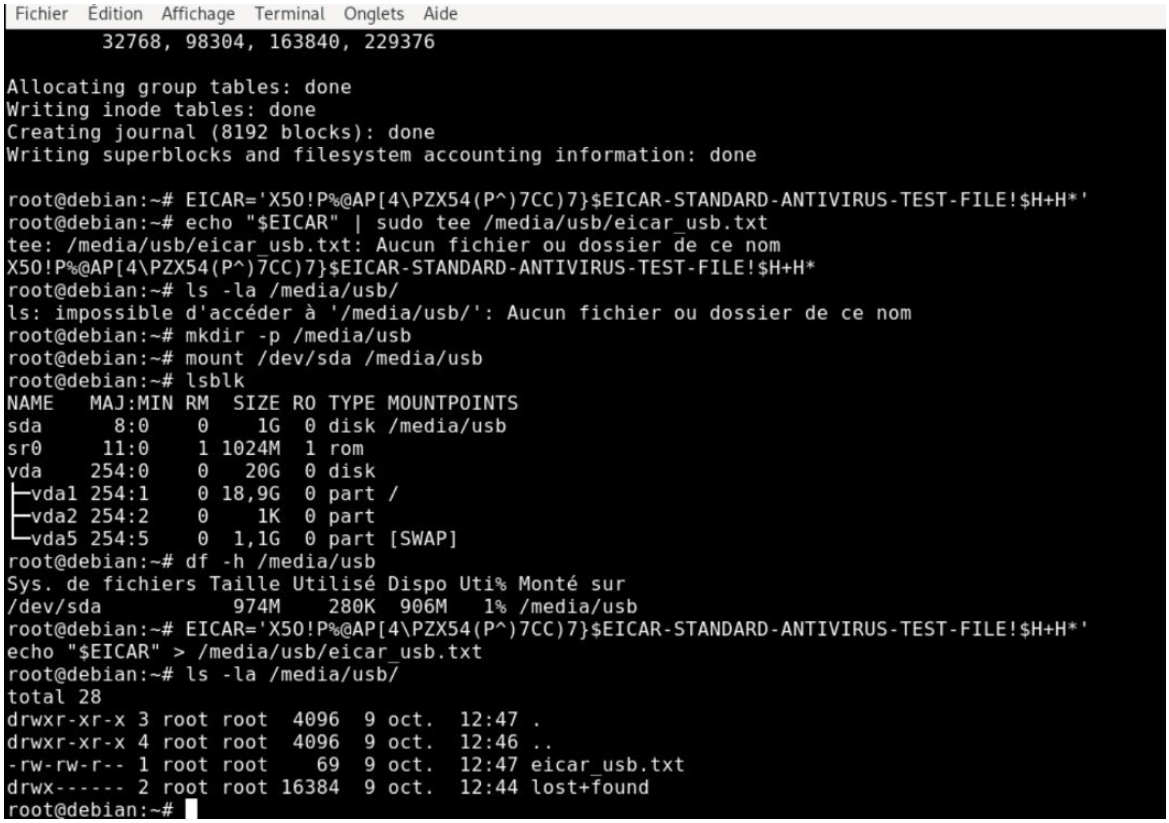


Figure 96 : *Disque monté sur /media/usb et fichier eicar_usb.txt copié (la première copie avait échoué car /media/usb n'existait pas encore)*

Activer l'analyse automatique des périphériques

1. Dans ClamUI, ouvrir **Preferences** (icône en haut à droite), onglet **Device Scan**.
2. Activer **Enable Device Auto-Scan** (« Scan newly mounted devices in the background »).
3. Choisir les types de périphériques à analyser (Removable Devices, External Drives), la taille maximale (32 Go) et activer Auto-Quarantine Threats.
4. Valider avec **Save**.

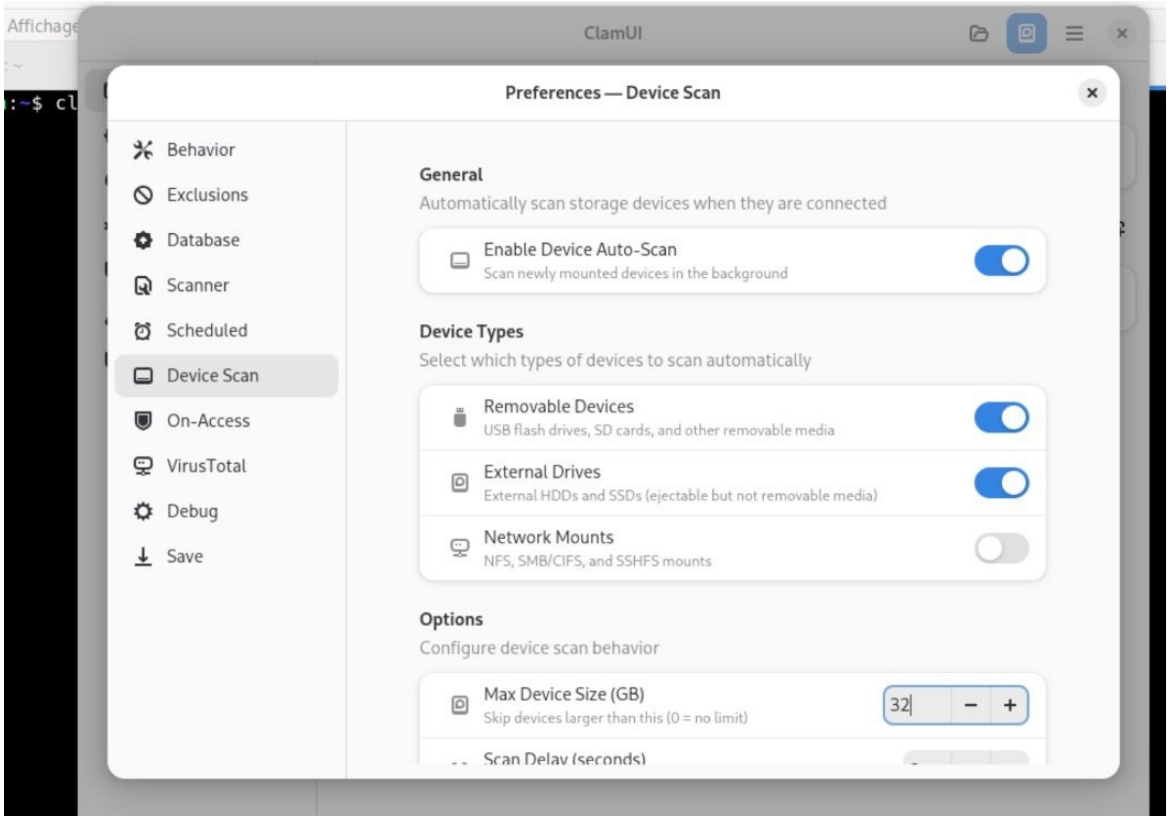


Figure 97 : *Preferences > Device Scan : l'analyse automatique des périphériques est activée*

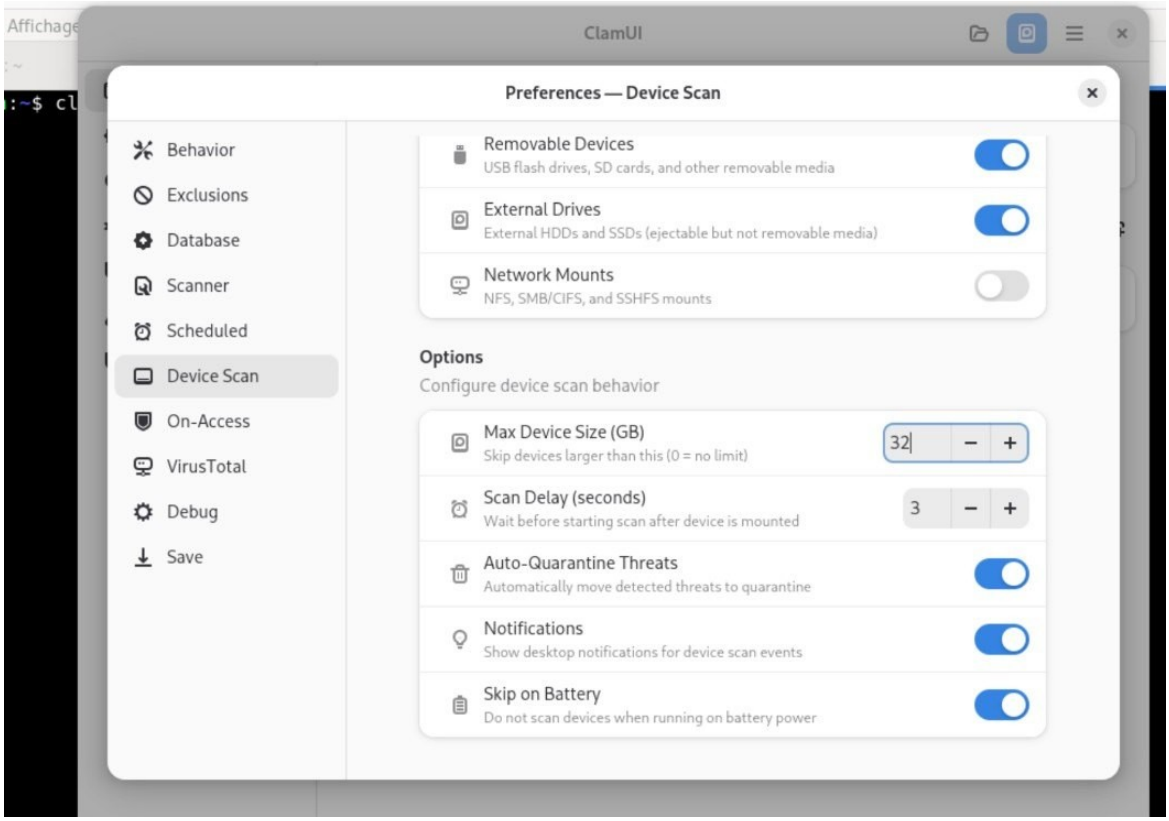


Figure 98 : *Options : taille maximale, délai de 3 s, quarantaine automatique et notifications*

Limite constatée et solution. Un disque monté à la main ne déclenche pas forcément l'analyse automatique, et `clamonacc` ne surveille que les dossiers déclarés (voir l'étape 6). Au lieu de monter directement le disque, on utilise donc un script qui **monte ET lance l'analyse**.

1. Créer le script d'analyse, puis le script de montage.

Sur Debian Client
```
sudo nano /usr/local/bin/usb-scan.sh
```

/usr/local/bin/usb-scan.sh
```
#!/bin/bash
# Analyse le contenu de /media/usb avec ClamAV et journalise le résultat
LOG="/var/log/clamav/usb_scan.log"
TARGET="/media/usb"

echo "[$(date)] Debut de l'analyse de ${TARGET}" >> "$LOG"
clamscan -r --infected "$TARGET" >> "$LOG" 2>&1
RC=$?
echo "[$(date)] Fin de l'analyse (code retour ${RC} : 0 = sain, 1 = virus trouve)" >> "$LOG"
```

Sur Debian Client
```
sudo nano /usr/local/bin/mount-usb.sh
```

/usr/local/bin/mount-usb.sh
```
#!/bin/bash
# Monte le disque USB et lance un scan automatique
if [ -z "$1" ]; then
    echo "Usage: mount-usb.sh <peripherique>   (exemple : /dev/sda)"
    exit 1
fi

DEVICE="$1"
MOUNTPOINT="/media/usb"
mkdir -p "$MOUNTPOINT"

# Monter
mount "$DEVICE" "$MOUNTPOINT" || { echo "Erreur de montage"; exit 1; }
echo "[$(date)] USB monte sur $MOUNTPOINT" >> /var/log/clamav/usb_scan.log

# Lancer le scan en arrière-plan
/usr/local/bin/usb-scan.sh &
echo "Montage reussi. Scan en cours (voir /var/log/clamav/usb_scan.log)"
```

2. Rendre les scripts exécutables :

Sur Debian Client
```
sudo chmod +x /usr/local/bin/usb-scan.sh /usr/local/bin/mount-usb.sh
```

3. Utiliser :

Sur Debian Client (en root)
```
umount /media/usb 2>/dev/null
/usr/local/bin/mount-usb.sh /dev/sda
sleep 40
cat /var/log/clamav/usb_scan.log
```

(Le délai est de l'ordre de 35 secondes, car `clamscan` doit d'abord charger la base de signatures.) Le même résultat s'obtient à la main :

Sur Debian Client (en root)
```
clamscan -r /media/usb/
```

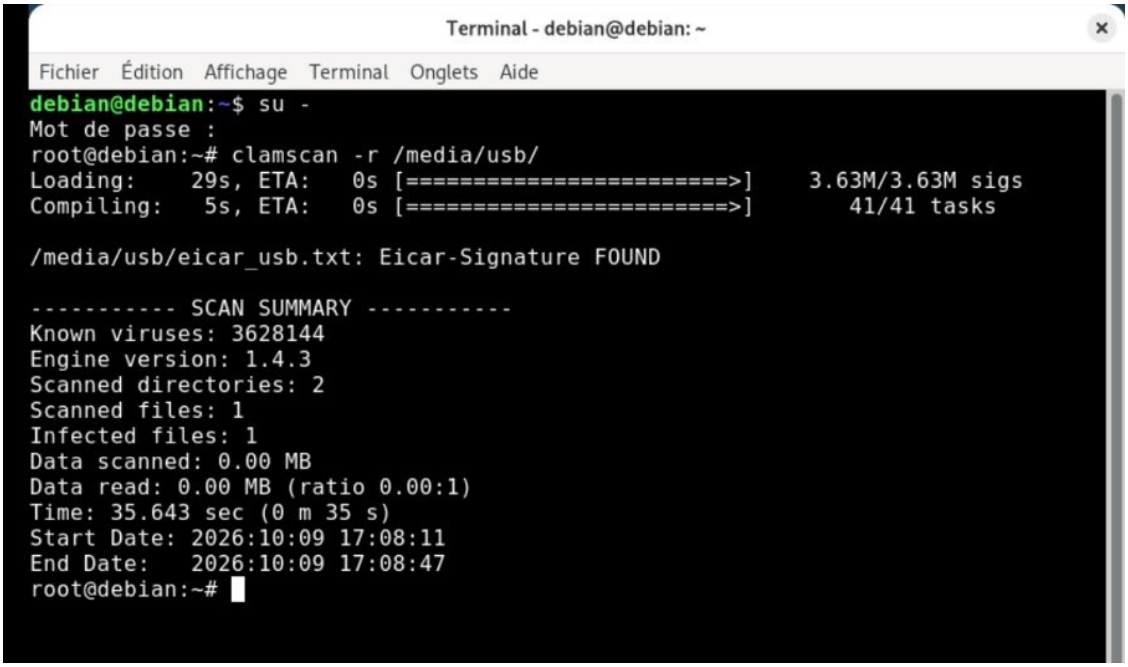


Figure 99 : *clamscan -r /media/usb/ : « eicar_usb.txt: Eicar-Signature FOUND » (1 fichier infecté sur 1 analysé)*

Nettoyage rapide :

Sur Debian Client

```
# 1. Retirer le fichier EICAR (optionnel)
rm -f /media/usb/eicar_usb.txt
umount /media/usb
# 2. Vérifier que clamonacc tourne toujours
ps aux | grep clamonacc | grep -v grep
```

Étape 8 : Planification des analyses

Dans ClamUI : **Preferences > Scheduled**.

Paramètre	Valeur
Enable Scheduled Scans	Activé
Fréquence (Scan Frequency)	Hebdomadaire (Weekly)
Jour (Day of Week)	Lundi
Heure (Scan Time)	02:00
Cible (Scan Targets)	/home/debian
Skip on Battery	Activé
Auto-Quarantine	Activé

Puis cliquer sur **Save & Apply**.

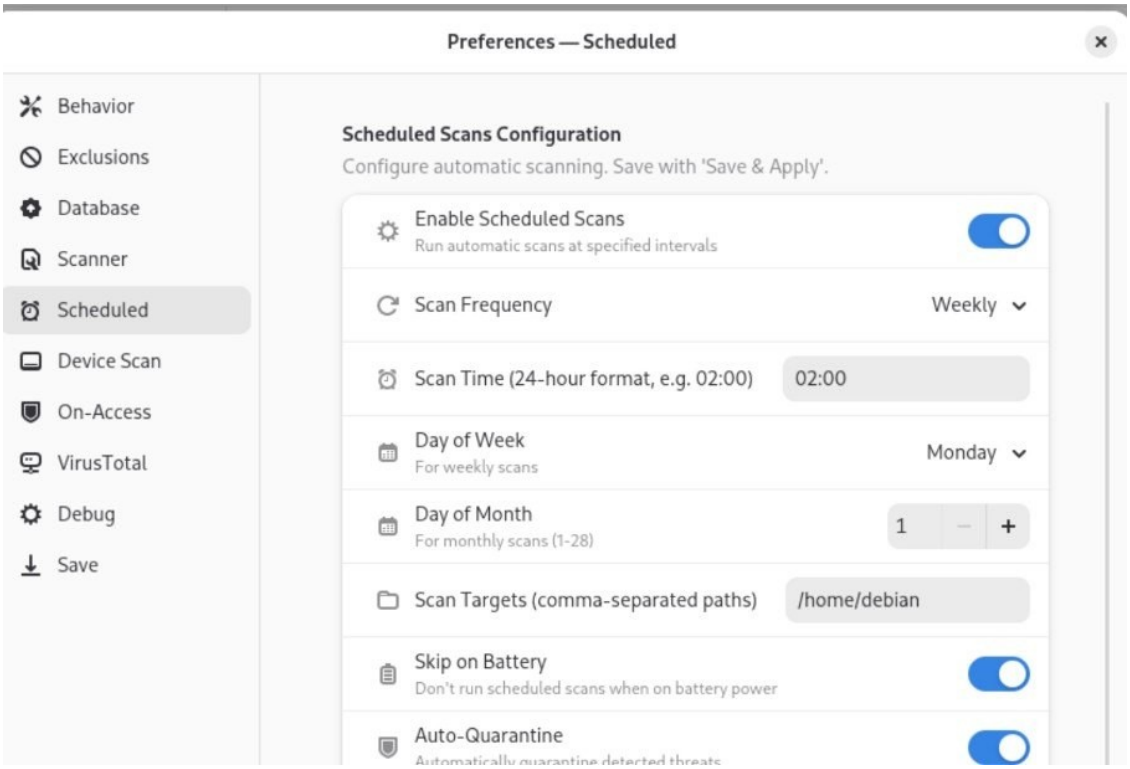


Figure 100 : *Analyses planifiées : chaque lundi à 02:00 sur /home/debian*

Étape 9 : Intégration au gestionnaire de fichiers (Nautilus)

Sur Debian Client

```
sudo apt install -y nautilus
nautilus
```

1. Dans Nautilus, naviguer vers `/home/debian`.
2. Faire un clic droit sur un fichier (n'importe lequel) ou sur un dossier.
3. Chercher dans le menu l'entrée **« Scan with ClamUI »**.
4. Deux cas possibles :

Cas A : l'entrée existe. On clique dessus : ClamUI s'ouvre avec le fichier prêt à scanner et l'analyse démarre.

Cas B : l'entrée n'existe pas. L'intégration n'est pas installée ou dépend d'une version de ClamUI plus récente que la version 0.4.0 utilisée ici. On le **consigne dans le rapport** comme une limite constatée, sans que cela remette en cause le reste de la protection.

Étape 10 : Éditeur de configuration

1. Regarder s'il existe un onglet **Configuration Editor** (ou similaire) dans ClamUI.
2. En parallèle, dans un terminal (utilisateur debian), afficher les fichiers de configuration sans les commentaires ni les lignes vides :

Sur Debian Client

```
grep -v "^#" /etc/clamav/clamd.conf | grep -v "^$"
grep -v "^#" /etc/clamav/freshclam.conf | grep -v "^$"
```

1. Chercher dans `freshclam.conf` le paramètre `Checks`, par exemple `Checks 24` : **24 vérifications de mise à jour par jour**, soit une toutes les heures.
2. Dans ClamUI > Preferences > Database, trouver l'équivalent : « Checks Per Day ».

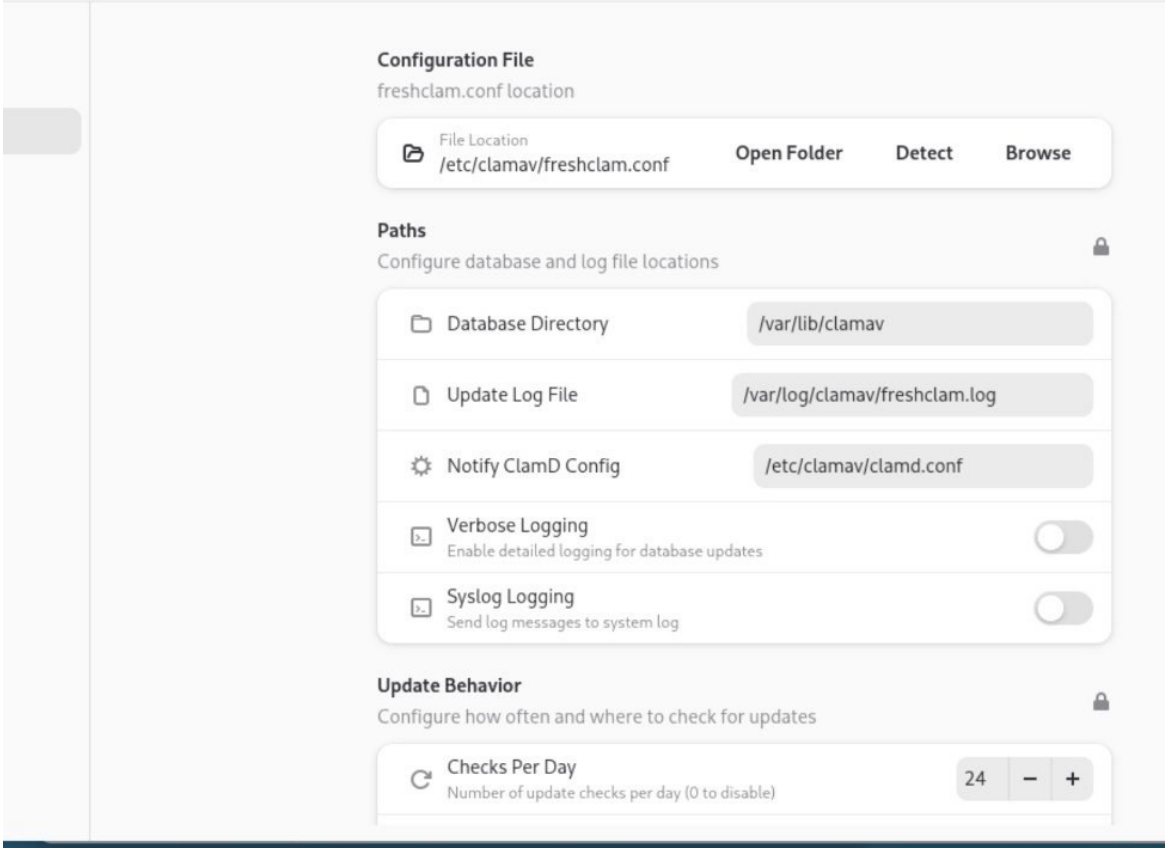


Figure 101 : *Preferences > Database : « Checks Per Day » vaut 24, comme dans freshclam.conf*

Questions de synthèse

1. Quelle est la différence entre ClamAV et ClamUI ?

- ClamAV est le moteur antivirus en ligne de commande (`clamscan`, `clamd`, `freshclam`, `clamonacc`).
- ClamUI est une interface graphique GTK4 qui pilote les outils ClamAV.
- ClamUI n'est pas un nouveau moteur : c'est une surcouche.

2. Quel est le résultat du test EICAR ?

Mode de détection	Fichier	Résultat
Scan manuel avec ClamUI	/home/debian/eicar.txt	Eicar-Signature détecté, mise en quarantaine proposée
Bouton « EICAR Test » de ClamUI	test_eicar.txt	Eicar-Signature détecté pendant l'analyse du dossier personnel
Scan d'un périphérique USB	/media/usb/eicar_usb.txt	« Eicar-Signature FOUND » avec clamscan -r /media/usb/
Protection en temps réel (clamonacc)	fichier du dossier surveillé	Détection à l'ouverture du fichier

Le fichier EICAR est donc détecté dans **tous les modes** testés : analyse manuelle, analyse d'un périphérique et protection en temps réel.

3. Comment fonctionne la quarantaine ?

- ClamUI isole le fichier infecté dans un dossier de quarantaine qu'il gère lui-même (l'emplacement exact est indiqué dans l'interface).
- Le fichier est mis à l'écart, mais n'est pas détruit.
- Deux actions sont possibles : **Restore** (restaurer) ou **Delete** (supprimer définitivement).

CONCLUSION

Bilan et perspectives

Ce laboratoire a permis de construire, de durcir, d'auditer et de sauvegarder une petite infrastructure d'entreprise, en suivant la démarche d'un administrateur sécurité. Les mesures sont claires : un réseau segmenté derrière un pare-feu périmétrique, un serveur dont la surface d'attaque est réduite à un seul port, une authentification par clés, des attaques par force brute bloquées automatiquement, des actions sensibles tracées par Auditd, et des sauvegardes vérifiées dont la restauration a été testée. Les audits Lynis (72 puis 73 sur 100) et OpenVAS (aucune vulnérabilité au-dessus de « faible ») confirment le niveau de sécurité atteint.

Le travail a aussi révélé des **difficultés réelles**, utiles à documenter : le conflit entre UFW et iptables-persistent, un espace insécable copié qui faussait une commande Fail2Ban, un faux test de verrouillage PAM (SSH par clé n'interroge jamais le mot de passe), et surtout un script de sauvegarde qui annonçait un succès alors que `rsync` était absent. Ces erreurs rappellent qu'**une mesure de sécurité n'existe que si elle est vérifiée**.

Perspectives d'amélioration

- Rétablir et contrôler la politique DROP du pare-feu après chaque redémarrage, puis planifier un scan de contrôle.
- Corriger les trois résultats OpenVAS (algorithmes MAC, horodatages TCP et ICMP).
- Ajouter une copie hors site des sauvegardes pour respecter la règle 3-2-1.
- Centraliser les journaux sur une machine dédiée, hors de portée d'un attaquant qui compromettrait le serveur.
- Renforcer OPNsense : règles de filtrage entre segments, détection d'intrusion (Suricata), mises à jour régulières.
- Ajouter une double authentification pour les accès d'administration.

ANNEXE

Glossaire

Terme	Définition
ACL	Access Control List : liste de droits fins, par utilisateur, qui complète les permissions Unix classiques
ASLR	Disposition aléatoire de l'espace d'adressage : place les programmes à des endroits imprévisibles en mémoire
Auditd	Système d'audit de Linux : il enregistre les événements de sécurité demandés par des règles
Brute force	Attaque qui essaie un très grand nombre de mots de passe jusqu'à trouver le bon
CVE / CVSS	Identifiant public d'une vulnérabilité (CVE) et note de gravité de 0 à 10 (CVSS)
Defense in depth	Défense en profondeur : plusieurs couches de protection successives
EICAR	Fichier de test inoffensif reconnu comme virus par tous les antivirus
Fail2Ban	Outil qui bannit automatiquement les adresses IP qui échouent trop souvent à se connecter
Hardening	Durcissement : réduction de la surface d'attaque d'un système
ICMP	Protocole des messages de contrôle réseau (le « ping » en fait partie)
iptables / Netfilter	Pare-feu intégré au noyau Linux et son outil de configuration
NAT	Traduction d'adresses : permet à un réseau privé de sortir sur Internet avec une adresse partagée
PAM	Modules d'authentification de Linux : règles communes pour vérifier un mot de passe
RST	Paquet réseau qui répond « rien ici » sur un port fermé
SHA256	Empreinte numérique d'un fichier : si le fichier change d'un seul bit, l'empreinte change
SSH	Connexion d'administration chiffrée à distance
sudo	Commande qui exécute une action avec les droits administrateur, en laissant une trace
Moindre privilège	Principe : chacun reçoit uniquement les droits nécessaires à sa tâche